

```
In [ ]: #PART-1(Data Import & Inspection )
```

```
In [1]: import pandas as pd
```

```
In [2]: import numpy as np
```

```
In [3]: pip install matplotlib
```

```
Requirement already satisfied: matplotlib in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (3.11.0)
Requirement already satisfied: contourpy>=1.0.1 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (1.5.0)
Requirement already satisfied: numpy>=1.25 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (2.5.1)
Requirement already satisfied: packaging>=20.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (26.2)
Requirement already satisfied: pillow>=9 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (12.3.0)
Requirement already satisfied: pyparsing>=3 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
```

```
In [4]: import matplotlib.pyplot as plt
```

```
In [5]: pip install seaborn
```

Requirement already satisfied: seaborn in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (0.13.2)

Requirement already satisfied: numpy!=1.24.0,>=1.20 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from seaborn) (2.5.1)

Requirement already satisfied: pandas>=1.2 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from seaborn) (3.0.3)

Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from seaborn) (3.11.0)

Requirement already satisfied: contourpy>=1.0.1 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.3)

Requirement already satisfied: cycler>=0.10 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.63.0)

Requirement already satisfied: kiwisolver>=1.3.1 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.5.0)

Requirement already satisfied: packaging>=20.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (26.2)

Requirement already satisfied: pillow>=9 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (12.3.0)

Requirement already satisfied: pyparsing>=3 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.3.2)

Requirement already satisfied: python-dateutil>=2.7 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)

Requirement already satisfied: tzdata in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from pandas>=1.2->seaborn) (2026.3)

Requirement already satisfied: six>=1.5 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)

Note: you may need to restart the kernel to use updated packages.

In [6]: `import seaborn as sns`

In [7]: `pip install scikit-learn`

Requirement already satisfied: scikit-learn in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (1.9.0)

Requirement already satisfied: numpy>=1.24.1 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from scikit-learn) (2.5.1)

Requirement already satisfied: scipy>=1.10.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from scikit-learn) (1.18.0)

Requirement already satisfied: joblib>=1.4.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from scikit-learn) (1.5.3)

Requirement already satisfied: narwhals>=2.0.1 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from scikit-learn) (2.24.0)

Requirement already satisfied: threadpoolctl>=3.5.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from scikit-learn) (3.6.0)

Note: you may need to restart the kernel to use updated packages.

In [8]: `from sklearn import preprocessing`

In [9]: `'''The required libraries were successfully imported`
`- Pandas is used for data manipulation`
`- NumPy is used for numerical computations`
`- Matplotlib and Seaborn are used for data visualization`
`- Scikit-learn provides machine learning algorithms and evaluation metrics'''`

```
Out[9]: 'The required libraries were successfully imported\n\n- Pandas is used for data
manipulation\n- NumPy is used for numerical computations\n- Matplotlib and Seab
orn are used for data visualization\n- Scikit-learn provides machine learning a
lgorithms and evaluation metrics'
```

```
In [10]: df=pd.read_csv("housing.csv")#Dataset Loaded successfully!
```

```
In [11]: print(df)
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	\
0	-122.23	37.88	41.0	880.0	129.0	
1	-122.22	37.86	21.0	7099.0	1106.0	
2	-122.24	37.85	52.0	1467.0	190.0	
3	-122.25	37.85	52.0	1274.0	235.0	
4	-122.25	37.85	52.0	1627.0	280.0	
...	...	...	...	...	...	
20635	-121.09	39.48	25.0	1665.0	374.0	
20636	-121.21	39.49	18.0	697.0	150.0	
20637	-121.22	39.43	17.0	2254.0	485.0	
20638	-121.32	39.43	18.0	1860.0	409.0	
20639	-121.24	39.37	16.0	2785.0	616.0	

	population	households	median_income	median_house_value	\
0	322.0	126.0	8.3252	452600.0	
1	2401.0	1138.0	8.3014	358500.0	
2	496.0	177.0	7.2574	352100.0	
3	558.0	219.0	5.6431	341300.0	
4	565.0	259.0	3.8462	342200.0	
...	...	...	...	...	
20635	845.0	330.0	1.5603	78100.0	
20636	356.0	114.0	2.5568	77100.0	
20637	1007.0	433.0	1.7000	92300.0	
20638	741.0	349.0	1.8672	84700.0	
20639	1387.0	530.0	2.3886	89400.0	

	ocean_proximity
0	NEAR BAY
1	NEAR BAY
2	NEAR BAY
3	NEAR BAY
4	NEAR BAY
...	...
20635	INLAND
20636	INLAND
20637	INLAND
20638	INLAND
20639	INLAND

[20640 rows x 10 columns]

```
In [12]: df.head(10) #Displays the first 10 rows of the dataset.
```

Out[12]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>longitude</th> <th>latitude</th> <th>housing_median_age</th>
<th>total_rooms</th> <th>total_bedrooms</th> <th>population</th>
<th>households</th> <th>median_income</th> <th>median_house_value</th>
<th>ocean_proximity</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th>
<th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th>
<th>9</th> </tbody> <tbody></tbody>

-122.23	37.88	41.0	880.0	129.0	322.0	126.0	8.3252	452600.0	NEAR BAY
-122.22	37.86	21.0	7099.0	1106.0	2401.0	1138.0	8.3014	358500.0	NEAR BAY
-122.24	37.85	52.0	1467.0	190.0	496.0	177.0	7.2574	352100.0	NEAR BAY
-122.25	37.85	52.0	1274.0	235.0	558.0	219.0	5.6431	341300.0	NEAR BAY
-122.25	37.85	52.0	1627.0	280.0	565.0	259.0	3.8462	342200.0	NEAR BAY
-122.25	37.85	52.0	919.0	213.0	413.0	193.0	4.0368	269700.0	NEAR BAY
-122.25	37.84	52.0	2535.0	489.0	1094.0	514.0	3.6591	299200.0	NEAR BAY
-122.25	37.84	52.0	3104.0	687.0	1157.0	647.0	3.1200	241400.0	NEAR BAY
-122.26	37.84	42.0	2555.0	665.0	1206.0	595.0	2.0804	226700.0	NEAR BAY
-122.25	37.84	52.0	3549.0	707.0	1551.0	714.0	3.6912	261100.0	NEAR BAY

In [13]: `df.tail(5)` *#Displays the last 5 rows*

Out[13]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>longitude</th> <th>latitude</th> <th>housing_median_age</th>
<th>total_rooms</th> <th>total_bedrooms</th> <th>population</th>
<th>households</th> <th>median_income</th> <th>median_house_value</th>
<th>ocean_proximity</th> </thead> <tbody> <th>20635</th> <th>20636</th>
<th>20637</th> <th>20638</th> <th>20639</th> </tbody> <tbody></tbody>

-121.09	39.48	25.0	1665.0	374.0	845.0	330.0	1.5603	78100.0	INLAND
-121.21	39.49	18.0	697.0	150.0	356.0	114.0	2.5568	77100.0	INLAND
-121.22	39.43	17.0	2254.0	485.0	1007.0	433.0	1.7000	92300.0	INLAND
-121.32	39.43	18.0	1860.0	409.0	741.0	349.0	1.8672	84700.0	INLAND
-121.24	39.37	16.0	2785.0	616.0	1387.0	530.0	2.3886	89400.0	INLAND

In [14]: `df.head(5)`

```
Out[14]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>longitude</th> <th>latitude</th> <th>housing_median_age</th>
<th>total_rooms</th> <th>total_bedrooms</th> <th>population</th>
<th>households</th> <th>median_income</th> <th>median_house_value</th>
<th>ocean_proximity</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th>
<th>3</th> <th>4</th> </tbody> <tbody></tbody>
```

-122.23	37.88	41.0	880.0	129.0	322.0	126.0	8.3252	452600.0	NEAR BAY
-122.22	37.86	21.0	7099.0	1106.0	2401.0	1138.0	8.3014	358500.0	NEAR BAY
-122.24	37.85	52.0	1467.0	190.0	496.0	177.0	7.2574	352100.0	NEAR BAY
-122.25	37.85	52.0	1274.0	235.0	558.0	219.0	5.6431	341300.0	NEAR BAY
-122.25	37.85	52.0	1627.0	280.0	565.0	259.0	3.8462	342200.0	NEAR BAY

```
In [15]: df.head(5)
```

```
Out[15]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>longitude</th> <th>latitude</th> <th>housing_median_age</th>
<th>total_rooms</th> <th>total_bedrooms</th> <th>population</th>
<th>households</th> <th>median_income</th> <th>median_house_value</th>
<th>ocean_proximity</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th>
<th>3</th> <th>4</th> </tbody> <tbody></tbody>
```

-122.23	37.88	41.0	880.0	129.0	322.0	126.0	8.3252	452600.0	NEAR BAY
-122.22	37.86	21.0	7099.0	1106.0	2401.0	1138.0	8.3014	358500.0	NEAR BAY
-122.24	37.85	52.0	1467.0	190.0	496.0	177.0	7.2574	352100.0	NEAR BAY
-122.25	37.85	52.0	1274.0	235.0	558.0	219.0	5.6431	341300.0	NEAR BAY
-122.25	37.85	52.0	1627.0	280.0	565.0	259.0	3.8462	342200.0	NEAR BAY

```
In [16]: df.dtypes
```

```
Out[16]: longitude          float64
latitude          float64
housing_median_age  float64
total_rooms        float64
total_bedrooms     float64
population         float64
households         float64
median_income      float64
median_house_value float64
ocean_proximity    str
dtype: object
```

```
In [17]: '''The dataset contains:
- Float variables
- Integer variables
- One categorical variable (object type)'''
```

```
Out[17]: 'The dataset contains:\n- Float variables\n- Integer variables\n- One categorical variable (object type)'
```

```
In [18]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   longitude              20640 non-null  float64
1   latitude               20640 non-null  float64
2   housing_median_age     20640 non-null  float64
3   total_rooms            20640 non-null  float64
4   total_bedrooms         20433 non-null  float64
5   population             20640 non-null  float64
6   households              20640 non-null  float64
7   median_income          20640 non-null  float64
8   median_house_value     20640 non-null  float64
9   ocean_proximity        20640 non-null  str
dtypes: float64(9), str(1)
memory usage: 1.7 MB
```

```
In [19]: '''The dataset contains both numerical and categorical variables
Most columns are numeric, while ocean_proximity is the only categorical feature'
```

```
Out[19]: 'The dataset contains both numerical and categorical variables\nMost columns are numeric, while ocean_proximity is the only categorical feature'
```

```
In [20]: df.shape
```

```
Out[20]: (20640, 10)
```

```
In [21]: '''The dataset contains 20,640 observations and 10 features'''
```

```
Out[21]: 'The dataset contains 20,640 observations and 10 features'
```

```
In [22]: df.describe()
```

```
Out[22]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>longitude</th> <th>latitude</th> <th>housing_median_age</th>
<th>total_rooms</th> <th>total_bedrooms</th> <th>population</th>
<th>households</th> <th>median_income</th> <th>median_house_value</th>
</thead> <tbody> <th>count</th> <th>mean</th> <th>std</th> <th>min</th>
<th>25%</th> <th>50%</th> <th>75%</th> <th>max</th> </tbody> <tbody>
20640.000000  20640.000000  20640.000000  20640.000000  20433.000000  20640.000000  20
-119.569704    35.631861    28.639486    2635.763081    537.870553    1425.476744
    2.003532     2.135952    12.585558    2181.615252    421.385070    1132.462122
-124.350000    32.540000     1.000000     2.000000     1.000000     3.000000
-121.800000    33.930000    18.000000    1447.750000    296.000000    787.000000
-118.490000    34.260000    29.000000    2127.000000    435.000000    1166.000000
-118.010000    37.710000    37.000000    3148.000000    647.000000    1725.000000
-114.310000    41.950000    52.000000   39320.000000   6445.000000   35682.000000  6
```



```
In [23]: '''
The summary statistics provide an overview of the distribution of each numerical
Some variables, such as total_rooms, population, and total_bedrooms, have large
These features may also contain outliers, which will be explored during the Expl
```

```
Out[23]: '\n\nThe summary statistics provide an overview of the distribution of each numerical feature\n\nSome variables, such as total_rooms, population, and total_bedrooms, have large ranges and high standard deviations, indicating considerable variation\n\nThese features may also contain outliers, which will be explored during the Exploratory Data Analysis (EDA) stage'
```

```
In [24]: '''## Description of Features
longitude - Geographic longitude of the housing block
latitude - Geographic latitude of the housing block
housing_median_age - Median age of houses in the block (years)
total_rooms -Total number of rooms within the block
total_bedrooms - Total number of bedrooms within the block
population - Total population living in the block
households - Total number of households in the block
median_income -Median household income
median_house_value -Median house value (Target Variable)
ocean_proximity - Distance category from the ocean (categorical feature)'''
```

```
Out[24]: '## Description of Features\nlongitude - Geographic longitude of the housing bl
ock\nlatitude - Geographic latitude of the housing block\nhousing_median_age -
Median age of houses in the block (years)\ntotal_rooms -Total number of rooms w
ithin the block\ntotal_bedrooms - Total number of bedrooms within the block\npo
pulation - Total population living in the block\nhouseholds - Total number of h
ouseholds in the block\nmedian_income -Median household income \nmedian_house_
value -Median house value (Target Variable\n)\nocean_proximity - Distance categ
ory from the ocean (categorical feature)'
```

```
In [25]: categorical_cols = df.select_dtypes(include=['object', 'str']).columns.tolist()
```

```
In [26]: numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns.tolist()
```

```
In [27]: print("Numerical:", numerical_cols)
print("Categorical:", categorical_cols)
```

```
Numerical: ['longitude', 'latitude', 'housing_median_age', 'total_rooms', 'total_
bedrooms', 'population', 'households', 'median_income', 'median_house_value']
Categorical: ['ocean_proximity']
```

```
In [28]: df.describe(include='str')
```

```
Out[28]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>ocean_proximity</th> </thead> <tbody> <th>count</th>
<th>unique</th> <th>top</th> <th>freq</th> </tbody> <tbody></tbody>

20640

5

<1H OCEAN

9136
```

```
In [29]: #PART-2(Data Cleaning )
```

```
In [30]: #checking missing values
df.isnull().sum()
```

```
Out[30]: longitude          0
latitude          0
housing_median_age      0
total_rooms          0
total_bedrooms      207
population          0
households          0
median_income        0
median_house_value    0
ocean_proximity      0
dtype: int64
```

```
In [31]: '''The dataset contains missing values only in the otal_bedrooms column (207 mis
All other columns are complete'''
```

```
Out[31]: 'The dataset contains missing values only in the otal_bedrooms column (207 miss
ing values)\nAll other columns are complete'
```



```
In [32]: #handling missing values ,#Fill mising value with meidan
df["total_bedrooms"] = df["total_bedrooms"].fillna(df["total_bedrooms"].median())
```

```
In [33]: #verify missing values

print(df.isnull().sum())
```

```
longitude          0
latitude           0
housing_median_age  0
total_rooms         0
total_bedrooms      0
population          0
households          0
median_income       0
median_house_value  0
ocean_proximity     0
dtype: int64
```

```
In [34]: '''The median was chosen because total_bedrooms contains extreme values and poss
        Unlike the mean, the median is not affected by unusually large or small values,
```

```
Out[34]: 'The median was chosen because total_bedrooms contains extreme values and possi
        ble outliers\n\nUnlike the mean, the median is not affected by unusually large
        or small values, making it a more reliable replacement for missing values'
```

```
In [35]: #checking duplicates
duplicates = df.duplicated().sum()

print("Duplicate Rows:", duplicates)
```

```
Duplicate Rows: 0
```

```
In [36]: '''No duplicate rows were found in the dataset'''
```

```
Out[36]: 'No duplicate rows were found in the dataset'
```

```
In [37]: #removing duplicates
df.drop_duplicates(inplace=True)
print("Duplicates after removal:", df.duplicated().sum())
```

```
Duplicates after removal: 0
```

```
In [38]: '''No duplicate rows were found in the dataset'''
```

```
Out[38]: 'No duplicate rows were found in the dataset'
```

```
In [39]: #check incorreccct values
print("Housing Age <=0 :", (df["housing_median_age"] <= 0).sum())

print("Median Income <=0 :", (df["median_income"] <= 0).sum())

print("House Price <=0 :", (df["median_house_value"] <= 0).sum())

print("Total Rooms <=0 :", (df["total_rooms"] <= 0).sum())

print("Population <=0 :", (df["population"] <= 0).sum())
```

```
print("Households <=0 :", (df["households"] <= 0).sum())
```

```
Housing Age <=0 : 0
Median Income <=0 : 0
House Price <=0 : 0
Total Rooms <=0 : 0
Population <=0 : 0
Households <=0 : 0
```

```
In [40]: '''No impossible values were detected.
```

```
All numerical features contain valid positive values suitable for analysis'''
```

```
Out[40]: 'No impossible values were detected.\n\nAll numerical features contain valid po
sitive values suitable for analysis'
```

```
In [41]: #checking negative values
numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns

for col in numerical_cols:
    print(col, (df[col] < 0).sum())
```

```
longitude 20640
latitude 0
housing_median_age 0
total_rooms 0
total_bedrooms 0
population 0
households 0
median_income 0
median_house_value 0
```

```
In [42]: for col in numerical_cols:
        print(col, (df[col] == 0).sum())
```

```
longitude 0
latitude 0
housing_median_age 0
total_rooms 0
total_bedrooms 0
population 0
households 0
median_income 0
median_house_value 0
```

```
In [43]: numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns
print(numerical_cols)
```

```
Index(['longitude', 'latitude', 'housing_median_age', 'total_rooms',
      'total_bedrooms', 'population', 'households', 'median_income',
      'median_house_value'],
      dtype='str')
```

```
In [44]: print(df.dtypes)
```

```

longitude      float64
latitude       float64
housing_median_age  float64
total_rooms    float64
total_bedrooms float64
population     float64
households     float64
median_income  float64
median_house_value float64
ocean_proximity str
dtype: object

```

In [45]: `'''All columns have appropriate data types`

`No conversion was required because:`

- `- Numerical variables are stored as integers or floating-point numbers.`
- `- ocean_proximity is correctly stored as a categorical (object) variable'''`

Out[45]: 'All columns have appropriate data types\n\nNo conversion was required because:\n\n- Numerical variables are stored as integers or floating-point numbers.\n- ocean_proximity is correctly stored as a categorical (object) variable'

In [46]: `print("Final Dataset Shape:", df.shape)`

Final Dataset Shape: (20640, 10)

In [47]: `print(df.isnull().sum())`

```

longitude      0
latitude       0
housing_median_age  0
total_rooms    0
total_bedrooms 0
population     0
households     0
median_income  0
median_house_value 0
ocean_proximity 0
dtype: int64

```

In [48]: `### Interpretation`

`'''The dataset is now completely clean`

- `- No missing values remain`
- `- No duplicate records exist`
- `- No invalid values were detected`
- `- All variables have appropriate data types`

`The dataset is now ready for Exploratory Data Analysis (EDA) and machine learning`

Out[48]: 'The dataset is now completely clean\n\n- No missing values remain\n- No duplicate records exist\n- No invalid values were detected\n- All variables have appropriate data types\n\nThe dataset is now ready for Exploratory Data Analysis (EDA) and machine learning'

In [49]: `#PART -3(Exploratory Data Analysis (EDA))`

In [50]: `#calculate mean,median.mode
mean = df[numerical_cols].mean()`

```
median = df[numerical_cols].median()
mode = df[numerical_cols].mode().iloc[0]
```

```
In [51]: print(mean)
         print(median)
         print(mode)
```

```
longitude      -119.569704
latitude        35.631861
housing_median_age  28.639486
total_rooms     2635.763081
total_bedrooms   536.838857
population      1425.476744
households       499.539680
median_income     3.870671
median_house_value 206855.816909
dtype: float64
longitude      -118.4900
latitude        34.2600
housing_median_age  29.0000
total_rooms     2127.0000
total_bedrooms   435.0000
population      1166.0000
households       409.0000
median_income     3.5348
median_house_value 179700.0000
dtype: float64
longitude      -118.310
latitude        34.060
housing_median_age  52.000
total_rooms     1527.000
total_bedrooms   435.000
population       891.000
households       306.000
median_income     3.125
median_house_value 500001.000
Name: 0, dtype: float64
```

```
In [52]: '''- Mean represents the average value.
         - Median represents the middle value.
         - Moderepresents the most frequently occurring value.
```

Comparing these measures helps identify whether a feature is symmetrically distr

```
Out[52]: '- Mean represents the average value.\n- Median represents the middle value.\n-
Moderepresents the most frequently occurring value.\n\nComparing these measures
helps identify whether a feature is symmetrically distributed or skewed'
```

```
In [53]: #calculating variamce ,SD,IQR
         variance = df[numerical_cols].var()

         std = df[numerical_cols].std()

         Q1 = df[numerical_cols].quantile(0.25)

         Q3 = df[numerical_cols].quantile(0.75)

         IQR = Q3 - Q1
```

```
In [54]: summary_table = pd.DataFrame({
    "Mean": mean,
    "Median": median,
    "Mode": mode,
    "Variance": variance,
    "Standard Deviation": std,
    "Q1": Q1,
    "Q3": Q3,
    "IQR": IQR
})

summary_table
```

```
Out[54]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>Mean</th> <th>Median</th> <th>Mode</th> <th>Variance</th>
<th>Standard Deviation</th> <th>Q1</th> <th>Q3</th> <th>IQR</th> </thead>
<tbody> <th>longitude</th> <th>latitude</th> <th>housing_median_age</th>
<th>total_rooms</th> <th>total_bedrooms</th> <th>population</th>
<th>households</th> <th>median_income</th> <th>median_house_value</th>
</tbody> <tbody></tbody>
```

-119.569704	-118.4900	-118.310	4.014139e+00	2.003532	-121.8000	-1
35.631861	34.2600	34.060	4.562293e+00	2.135952	33.9300	
28.639486	29.0000	52.000	1.583963e+02	12.585558	18.0000	
2635.763081	2127.0000	1527.000	4.759445e+06	2181.615252	1447.7500	31
536.838857	435.0000	435.000	1.758895e+05	419.391878	297.0000	6
1425.476744	1166.0000	891.000	1.282470e+06	1132.462122	787.0000	17
499.539680	409.0000	306.000	1.461760e+05	382.329753	280.0000	6
3.870671	3.5348	3.125	3.609323e+00	1.899822	2.5634	
206855.816909	179700.0000	500001.000	1.331615e+10	115395.615874	119600.0000	2647

```
In [55]: #identify skewed features
for col in numerical_cols:
    if mean[col] > median[col]:
        print(f"{col}: Positively (Right) Skewed")
    elif mean[col] < median[col]:
        print(f"{col}: Negatively (Left) Skewed")
    else:
        print(f"{col}: Approximately Symmetric")
```

```

longitude: Negatively (Left) Skewed
latitude: Positively (Right) Skewed
housing_median_age: Negatively (Left) Skewed
total_rooms: Positively (Right) Skewed
total_bedrooms: Positively (Right) Skewed
population: Positively (Right) Skewed
households: Positively (Right) Skewed
median_income: Positively (Right) Skewed
median_house_value: Positively (Right) Skewed

```

```

In [56]: '''
A feature is:

- Right-skewed when Mean > Median
- Left-skewed when Mean < Median
- Approximately symmetric when Mean ≈ Median

This comparison provides a simple indication of the distribution shape'''

```

```

Out[56]: '\nA feature is:\n\n- Right-skewed when Mean > Median\n- Left-skewed when Mean
< Median\n- Approximately symmetric when Mean ≈ Median\n\nThis comparison provi
des a simple indication of the distribution shape'

```

```

In [57]: #calculating Actual Skewnwss
skewness = df[numerical_cols].skew()

print(skewness)

```

```

longitude          -0.297801
latitude           0.465953
housing_median_age  0.060331
total_rooms        4.147343
total_bedrooms     3.481141
population         4.935858
households         3.410438
median_income      1.646657
median_house_value 0.977763
dtype: float64

```

```

In [58]: #final; Summary table including skewness
summary_table["Skewness"] = df[numerical_cols].skew()

summary_table

```

```
Out[58]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>Mean</th> <th>Median</th> <th>Mode</th> <th>Variance</th>
<th>Standard Deviation</th> <th>Q1</th> <th>Q3</th> <th>IQR</th>
<th>Skewness</th> </thead> <tbody> <th>longitude</th> <th>latitude</th>
<th>housing_median_age</th> <th>total_rooms</th> <th>total_bedrooms</th>
<th>population</th> <th>households</th> <th>median_income</th>
<th>median_house_value</th> </tbody> <tbody></tbody>
```

-119.569704	-118.4900	-118.310	4.014139e+00	2.003532	-121.8000	-1
35.631861	34.2600	34.060	4.562293e+00	2.135952	33.9300	
28.639486	29.0000	52.000	1.583963e+02	12.585558	18.0000	
2635.763081	2127.0000	1527.000	4.759445e+06	2181.615252	1447.7500	31
536.838857	435.0000	435.000	1.758895e+05	419.391878	297.0000	6
1425.476744	1166.0000	891.000	1.282470e+06	1132.462122	787.0000	17
499.539680	409.0000	306.000	1.461760e+05	382.329753	280.0000	6
3.870671	3.5348	3.125	3.609323e+00	1.899822	2.5634	
206855.816909	179700.0000	500001.000	1.331615e+10	115395.615874	119600.0000	2647



In [59]:

```
'''
###
```

The summary statistics table combines all important descriptive measures into a single table. It provides a comprehensive overview of the dataset and highlights differences in central tendency and variability across features.

```
Out[59]: '\n### \n\nThe summary statistics table combines all important descriptive measures into a single table\nIt provides a comprehensive overview of the dataset and highlights differences in central tendency and variability across features'
```

In [60]:

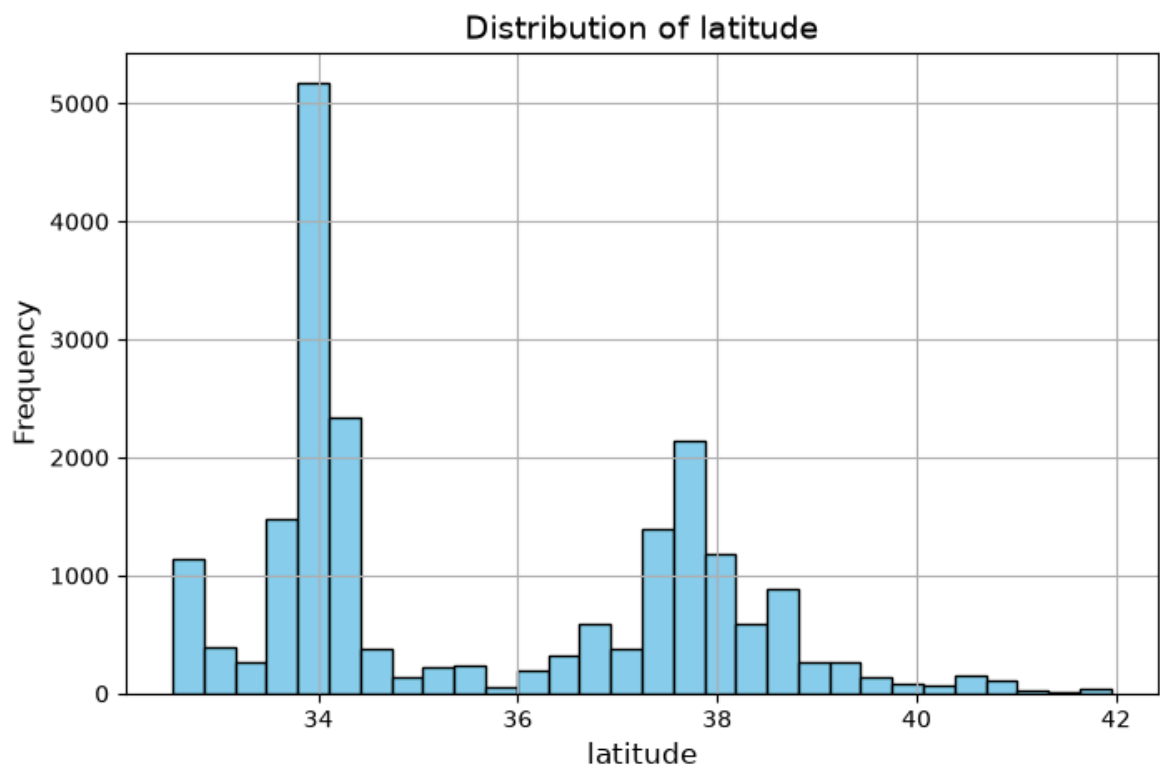
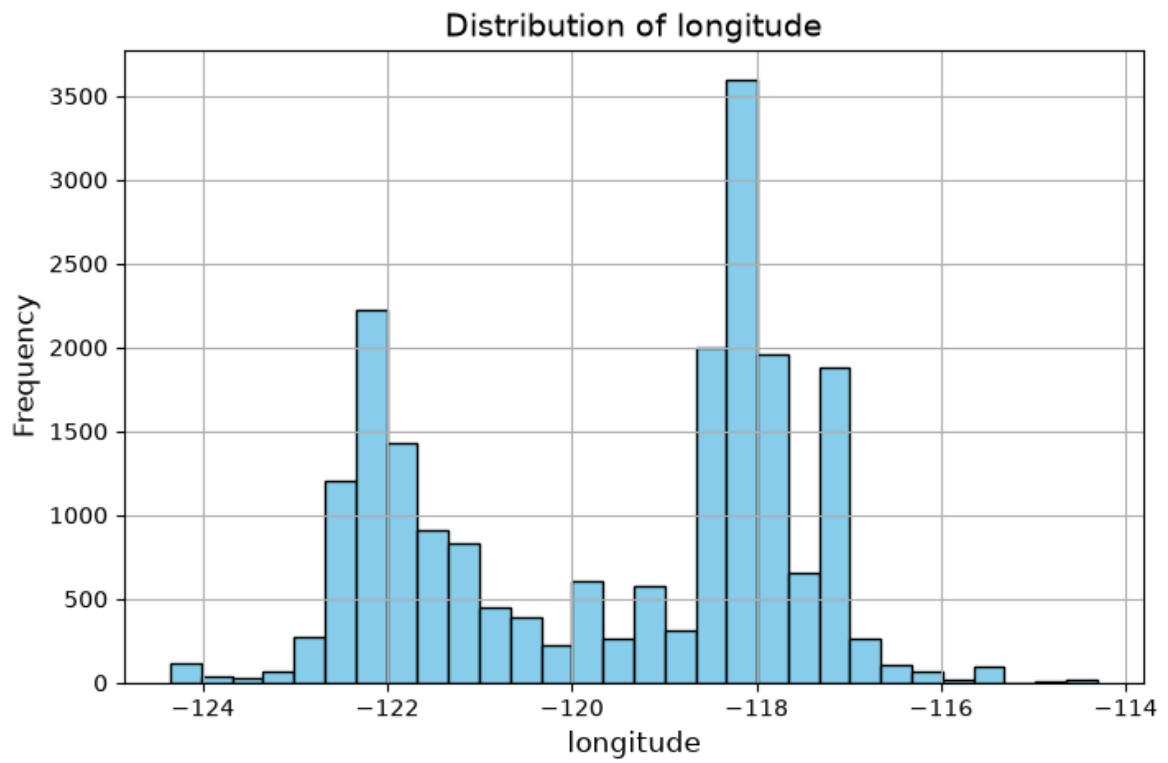
```
#Histogram
numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns

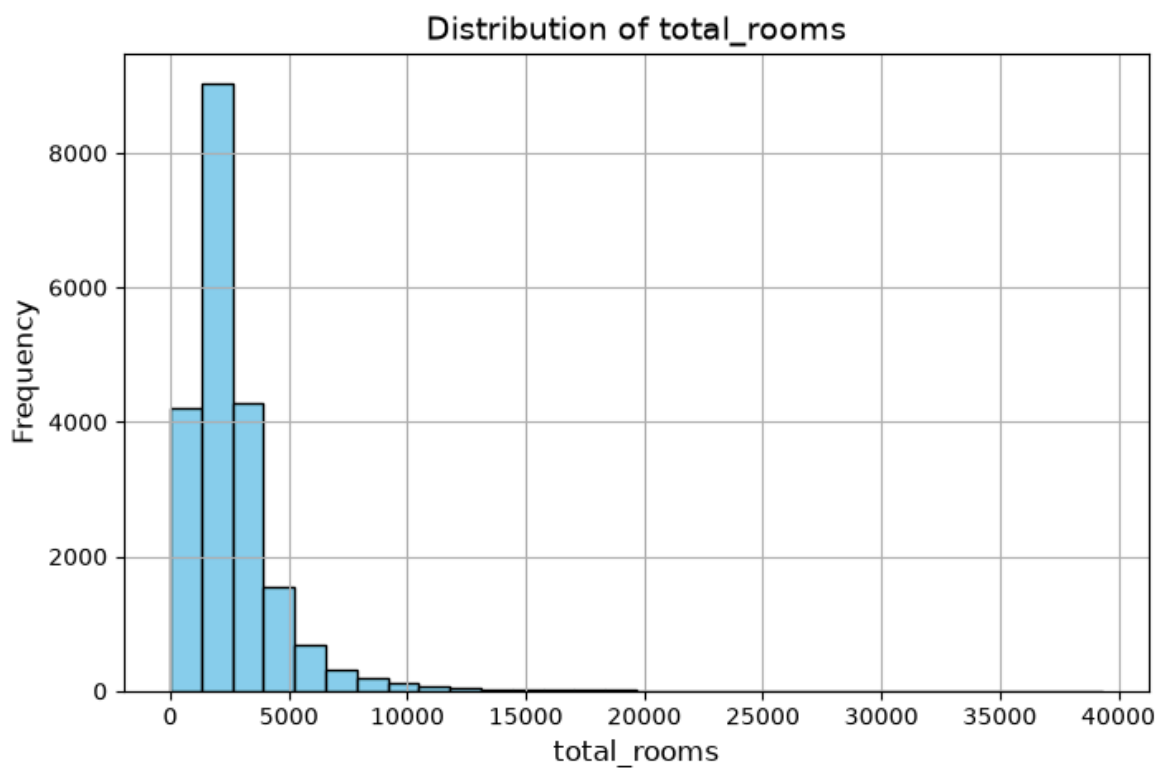
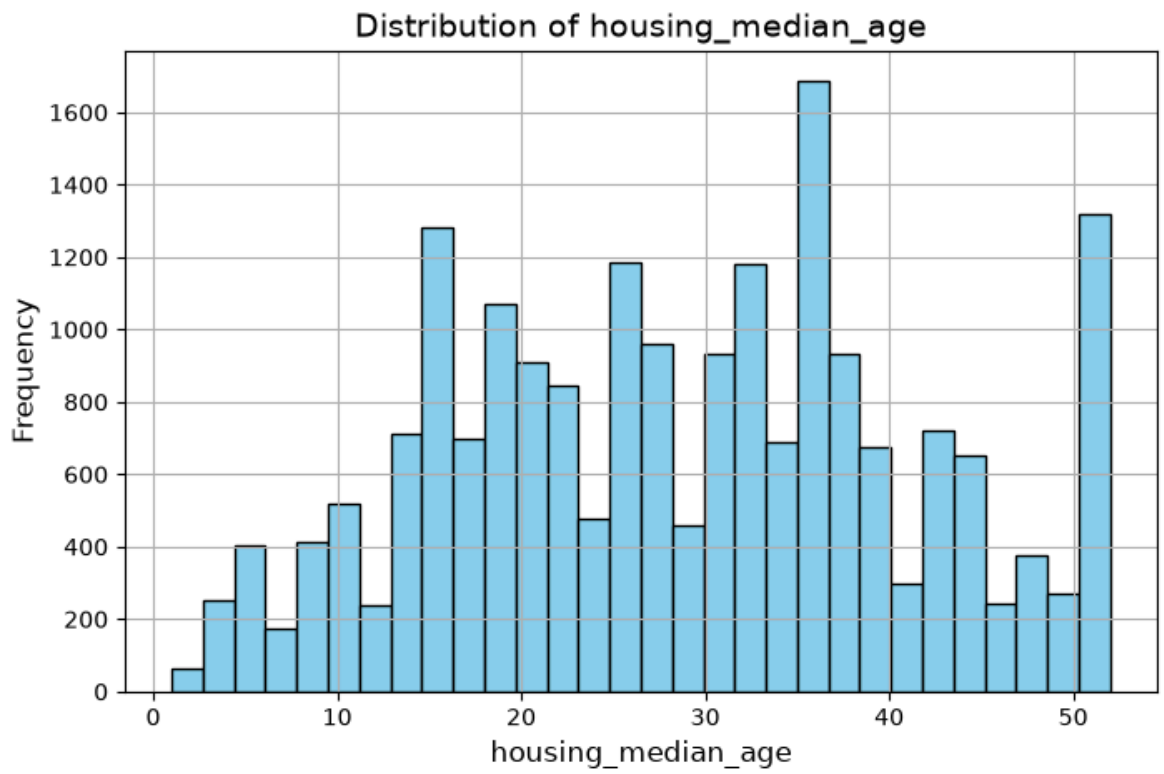
for col in numerical_cols:
    plt.figure(figsize=(8,5))

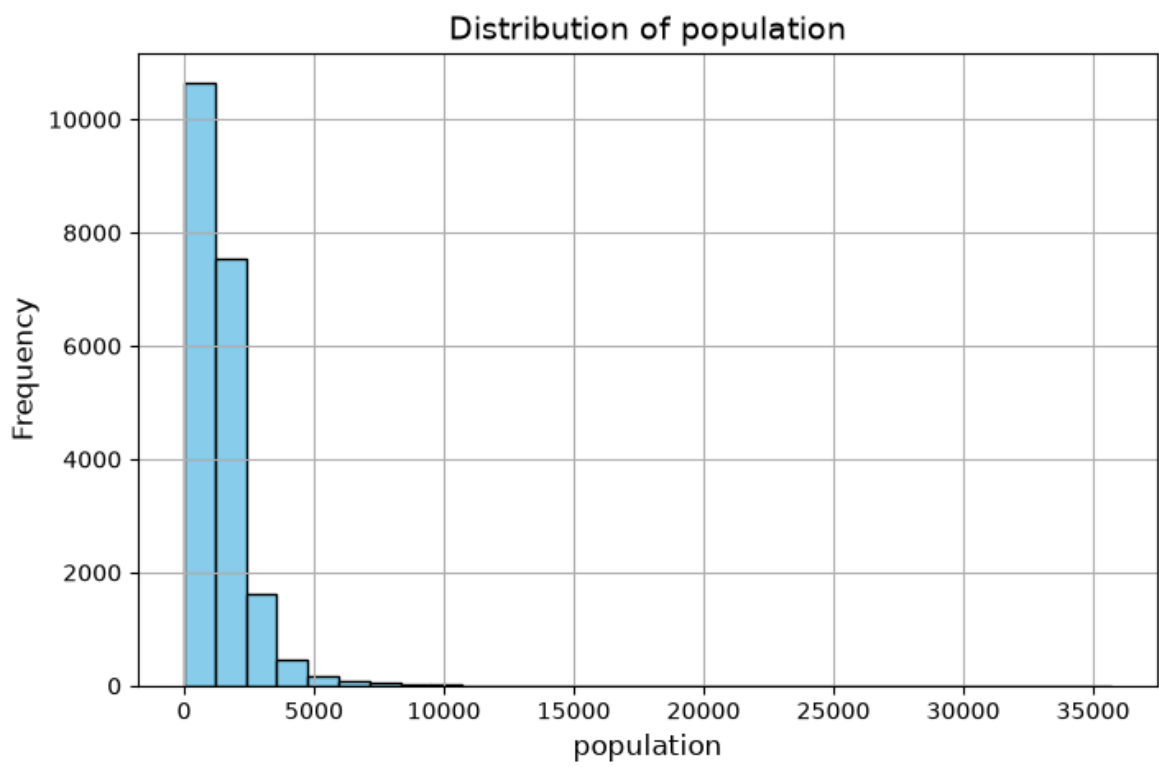
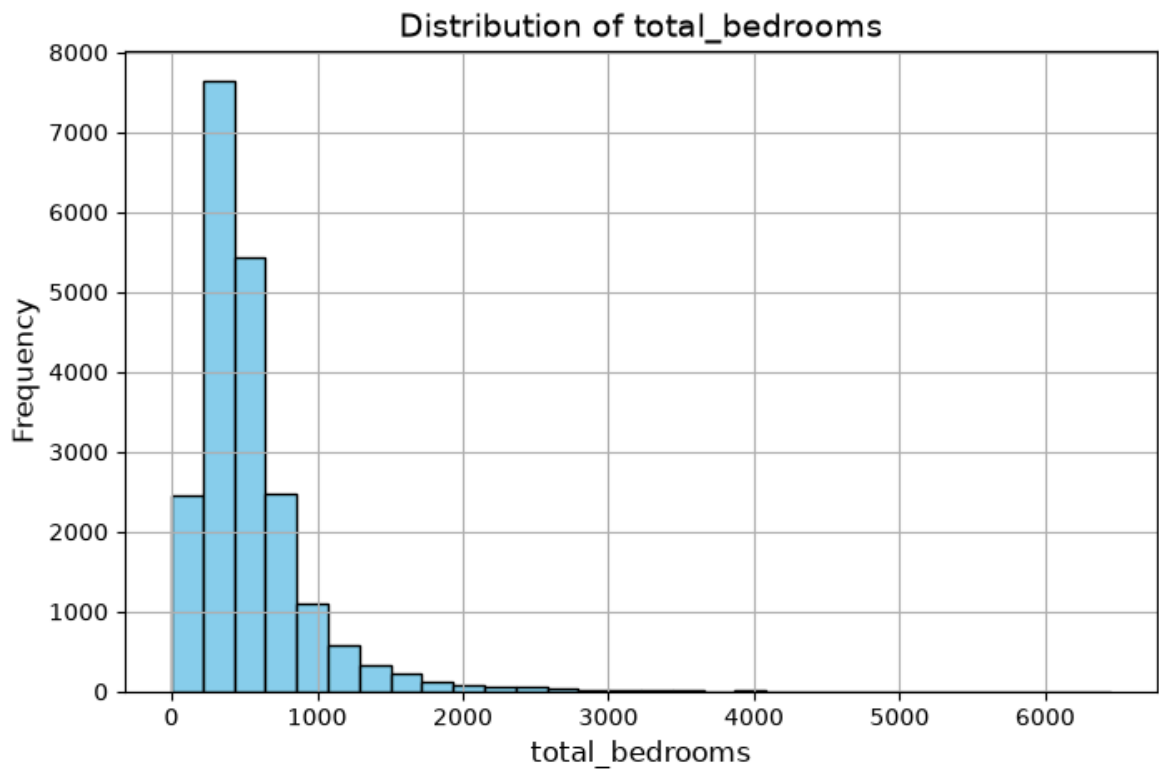
    plt.hist(df[col], bins=30, edgecolor='black', color='skyblue')

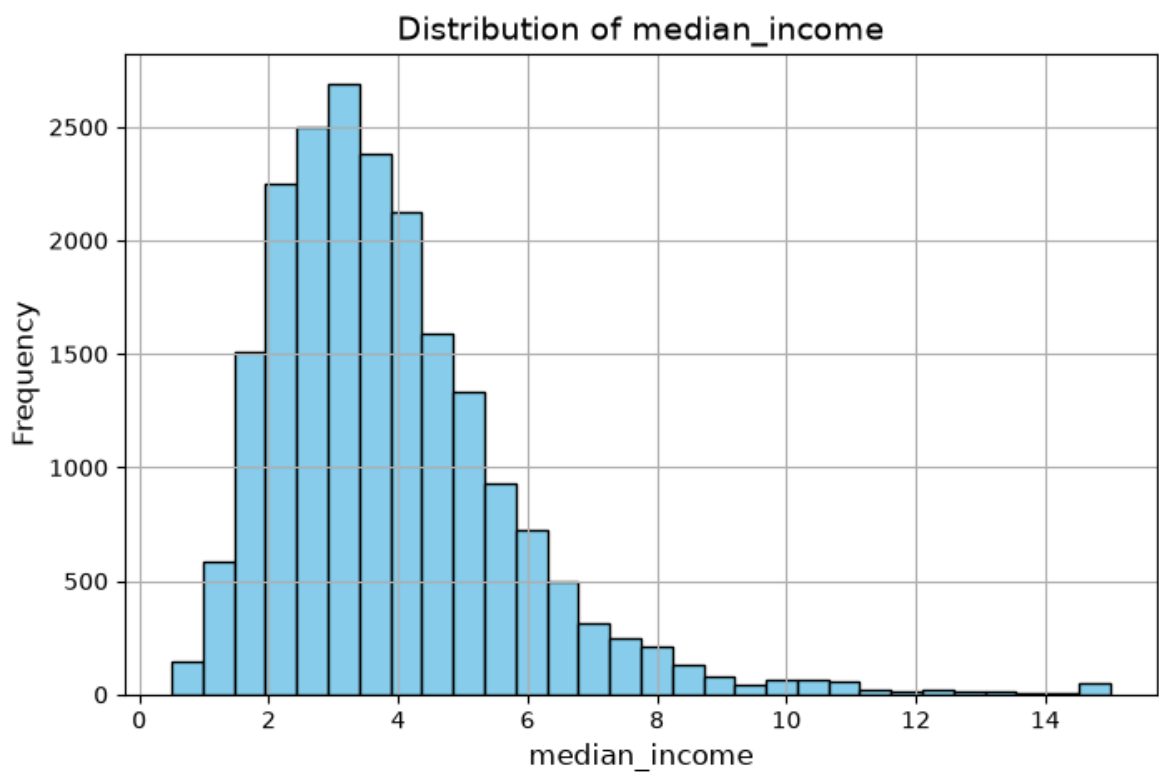
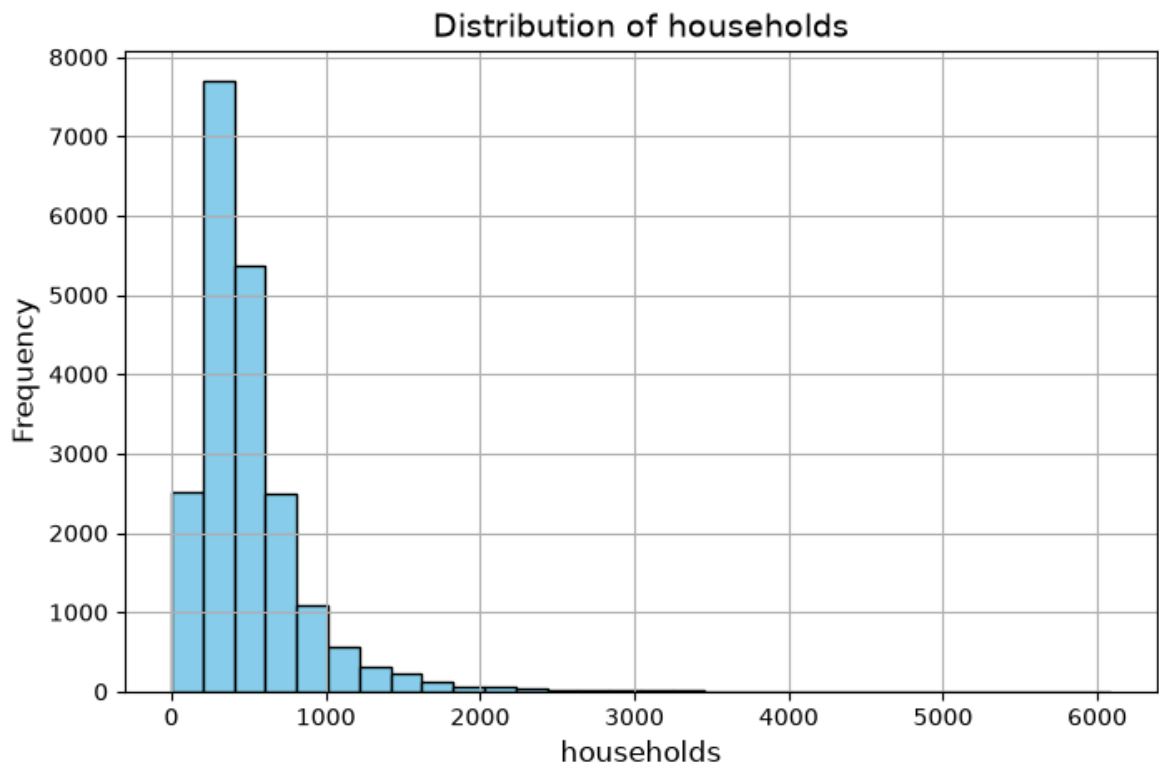
    plt.title(f"Distribution of {col}", fontsize=14)
    plt.xlabel(col, fontsize=12)
    plt.ylabel("Frequency", fontsize=12)

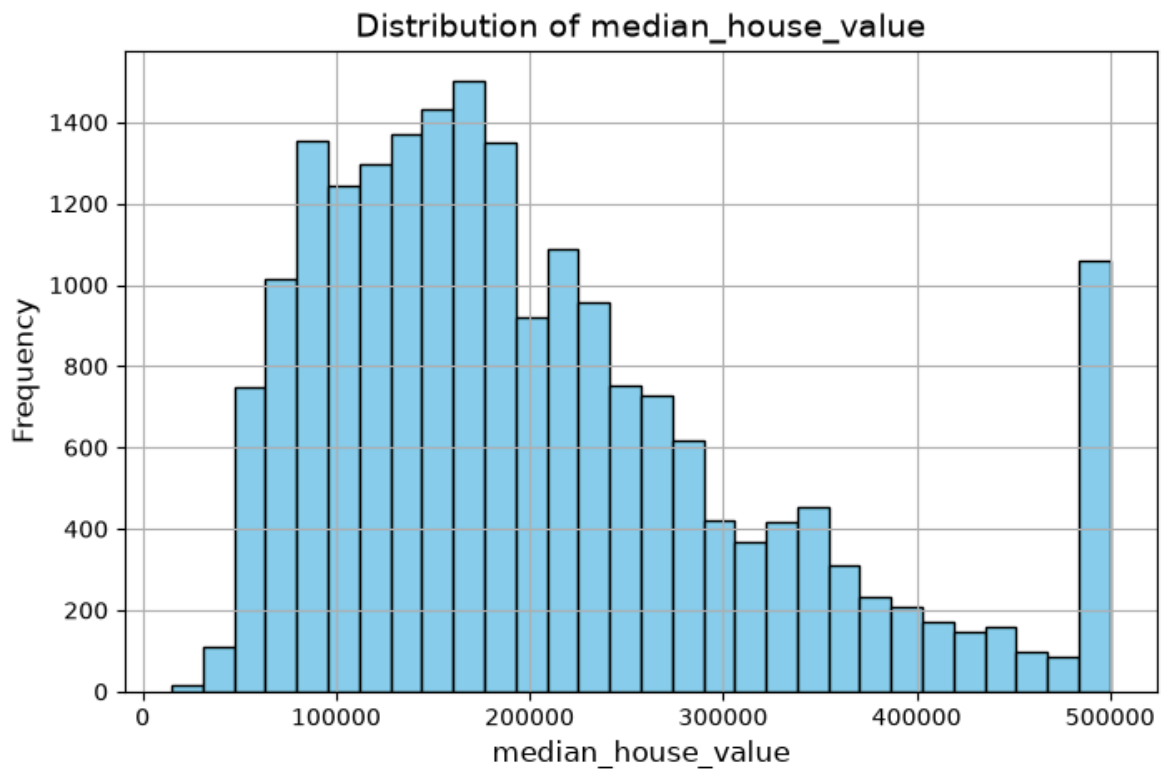
    plt.grid(True)
    plt.show()
```











```
In [61]: '''
1] Longitude:
The longitude distribution is not uniform, indicating that houses are concentrat
2] Latitude:
The latitude distribution shows that most housing blocks are concentrated within
3]Housing Median Age:
The housing median age is moderately distributed, with many houses falling betwe
4] Total Rooms:
The distribution of total rooms is highly right-skewed. Most housing blocks cont
These extreme values may represent densely populated neighborhoods or unusually
5] Total Bedrooms:
The histogram of total bedrooms is also right-skewed. Most observations have few
These may be considered potential outliers.
6] Population:
The population distribution is strongly right-skewed. Most housing blocks have r
7] Households:
The households feature follows a right-skewed distribution. Most blocks have a m
8] Median Income:
Median income is positively skewed, with most housing blocks having moderate inc
9]Median House Value (Target Variable):
The target variable, median house value, is positively skewed. Most houses are p
The sharp cutoff near the maximum value indicates that house prices were capped
```

```
Out[61]: '\n1] Longitude:\nThe longitude distribution is not uniform, indicating that ho
uses are concentrated in specific regions of California rather than being evenl
y distributed. This reflects the geographical clustering of residential areas\n
2] Latitude:\nThe latitude distribution shows that most housing blocks are conc
entrated within certain latitude ranges. This suggests that the population is u
nevenly distributed across California.\n3] Housing Median Age:\nThe housing medi
an age is moderately distributed, with many houses falling between 20 and 40 ye
ars old. The distribution indicates that California contains a mix of older and
newer residential areas.\n4] Total Rooms:\nThe distribution of total rooms is h
ighly right-skewed. Most housing blocks contain a relatively small number of ro
oms, while a few blocks have extremely large numbers of rooms. \nThese extreme
values may represent densely populated neighborhoods or unusually large residen
tial developments.\n5] Total Bedrooms:\nThe histogram of total bedrooms is also
right-skewed. Most observations have fewer bedrooms, while a small number of ho
using blocks contain exceptionally high bedroom counts.\nThese may be considere
d potential outliers.\n6] Population:\nThe population distribution is strongly
right-skewed. Most housing blocks have relatively small populations, whereas a
few blocks contain very large populations. This indicates considerable variatio
n in population size across different regions\n7] Households:\nThe households f
eature follows a right-skewed distribution. Most blocks have a moderate number
of households, while a few blocks contain a significantly larger number of hous
eholds.\n8] Median Income:\nMedian income is positively skewed, with most housi
ng blocks having moderate income levels and fewer blocks having very high incom
es. This suggests income inequality across different regions of California.\n9]
Median House Value (Target Variable):\nThe target variable, median house value,
is positively skewed. Most houses are priced in the lower to middle range, whil
e fewer houses have very high values. \nThe sharp cutoff near the maximum value
indicates that house prices were capped in the dataset, meaning values above th
is limit were not recorded'
```

```
In [62]: '''Overall Histogram Analysis (Good for Assignment):

The histograms reveal that most numerical features are positively (right) skewed

The median_income feature also shows a right-skewed distribution, suggesting tha

The target variable, median_house_value, is also right-skewed and shows an upper
```

```
Out[62]: 'Overall Histogram Analysis (Good for Assignment):\n\nThe histograms reveal tha
t most numerical features are positively (right) skewed , meaning that the majo
rity of observations are concentrated at lower values with a small number of ex
tremely high values. Features such as total_rooms, **total_bedrooms, populatio
n, and households exhibit noticeable outliers\n\nThe median_income feature also
shows a right-skewed distribution, suggesting that most households earn moderat
e incomes while only a small proportion have very high incomes\n\nThe target va
riable, median_house_value, is also right-skewed and shows an upper limit, indi
cating that house prices were capped in the dataset. These observations suggest
that the dataset contains non-normal distributions and outliers, which are comm
on in real-world housing data and should be considered during model developmen
t'
```

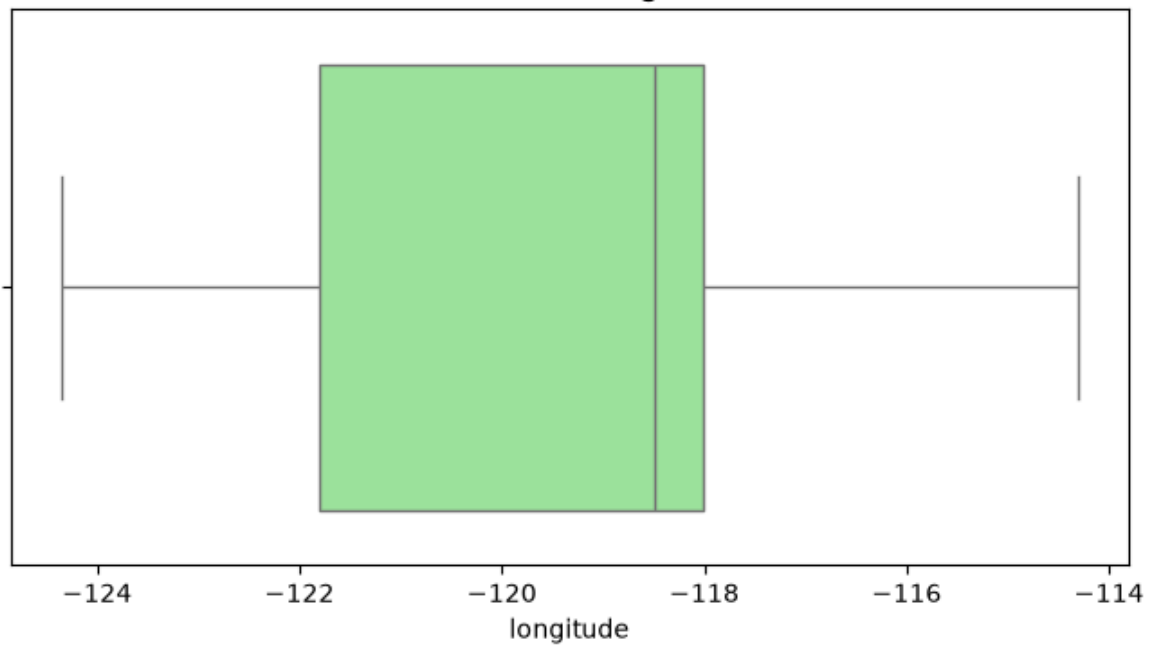
```
In [63]: #Box Plots to identify Outliers
for col in numerical_cols:
    plt.figure(figsize=(8,4))

    sns.boxplot(x=df[col], color="lightgreen")

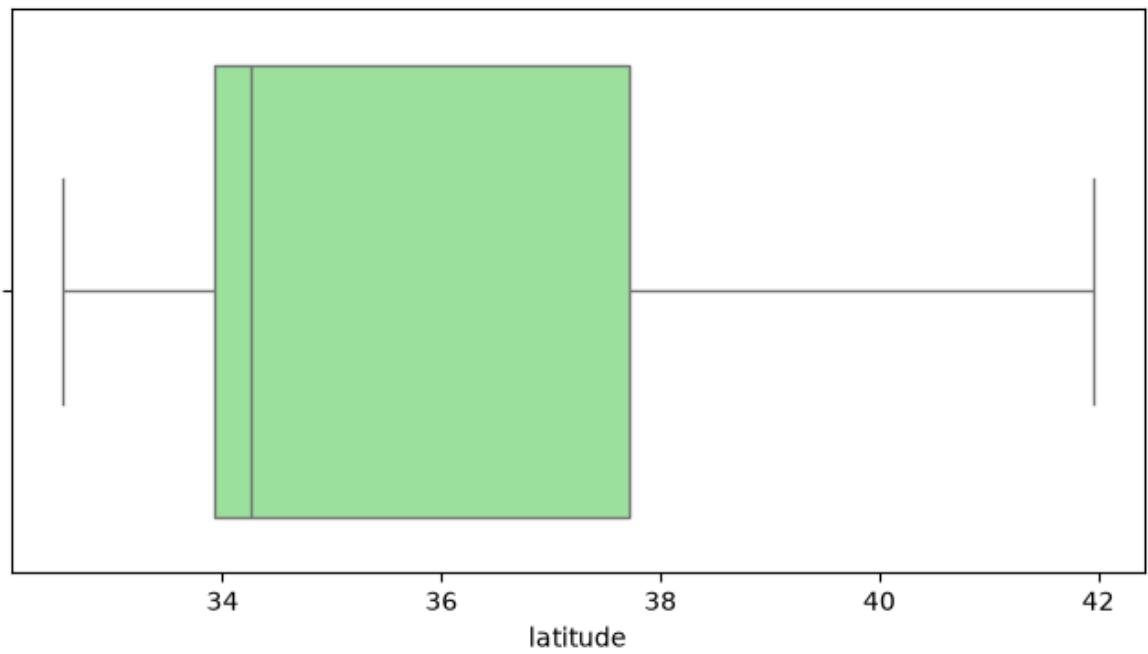
    plt.title(f"Box Plot of {col}", fontsize=14)
    plt.xlabel(col)
```

```
plt.show()
```

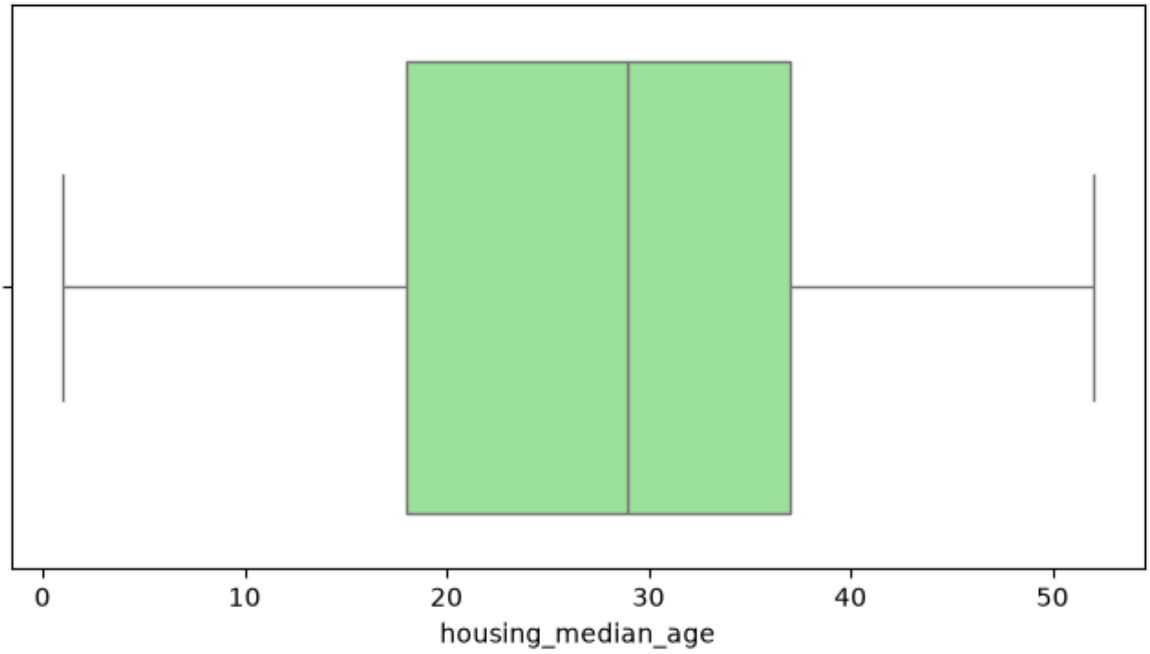
Box Plot of longitude



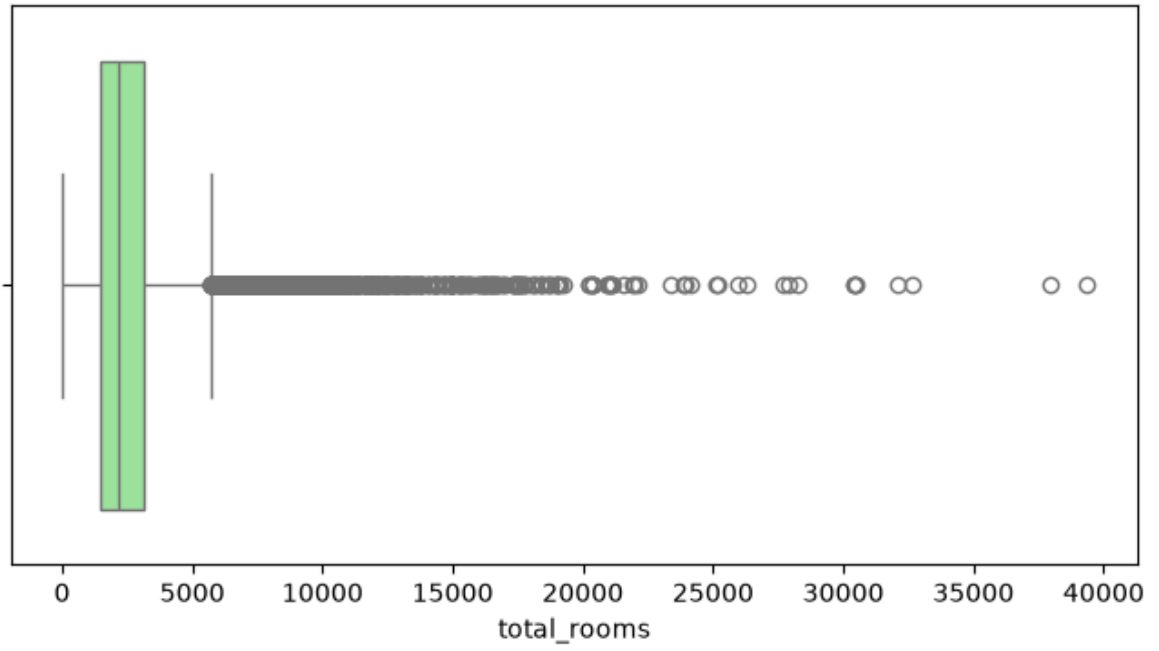
Box Plot of latitude



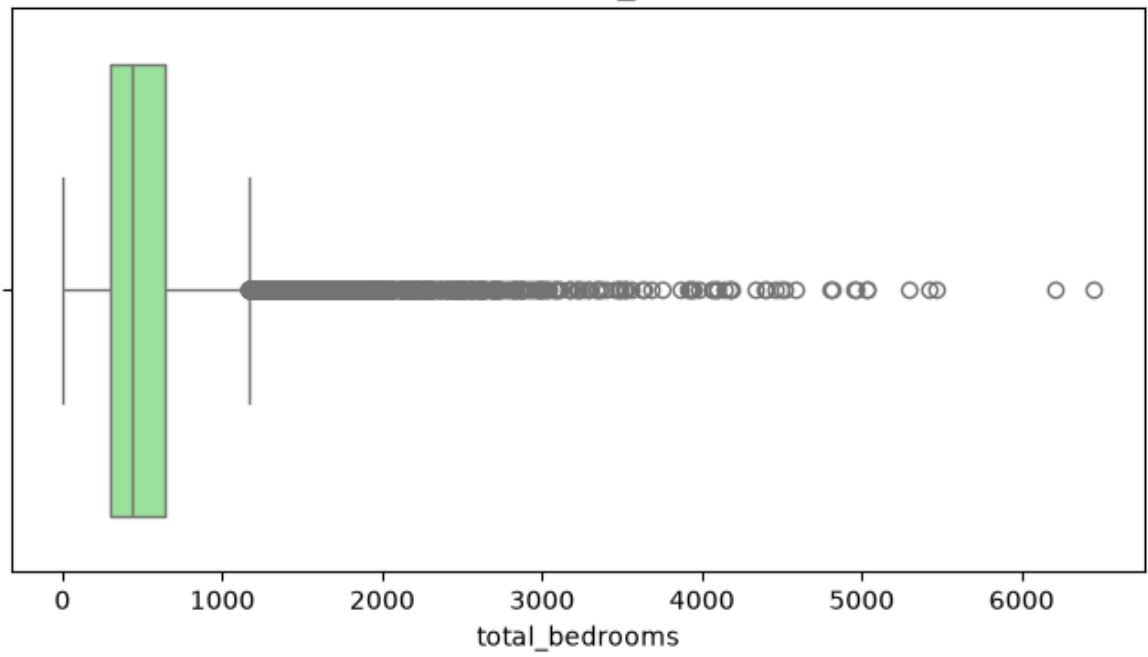
Box Plot of housing_median_age



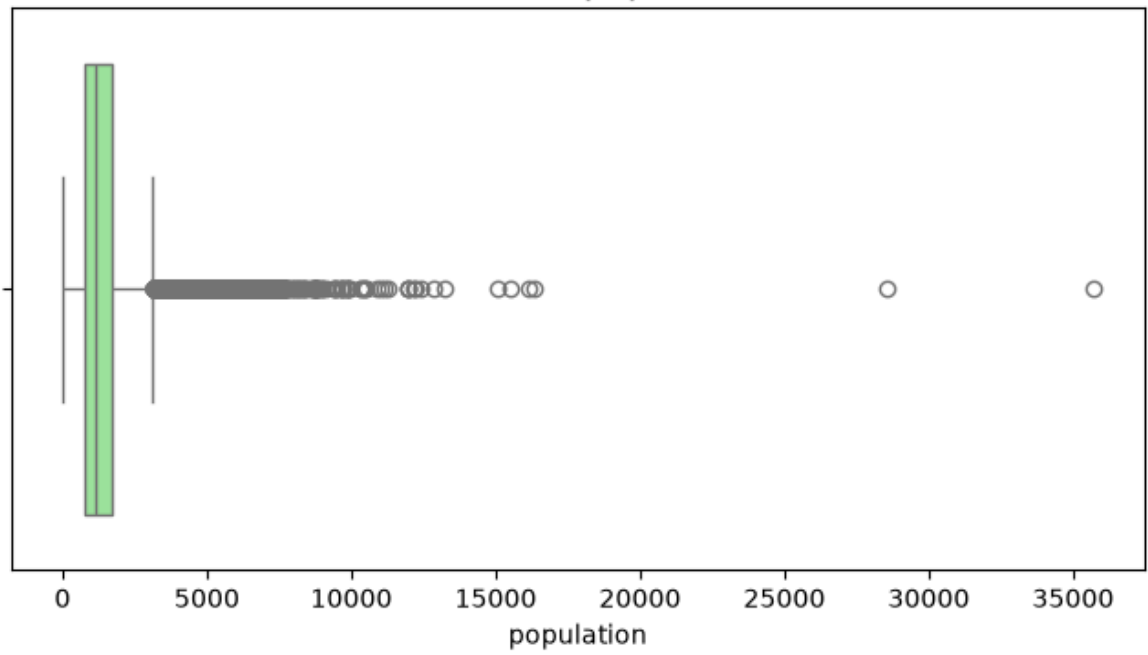
Box Plot of total_rooms



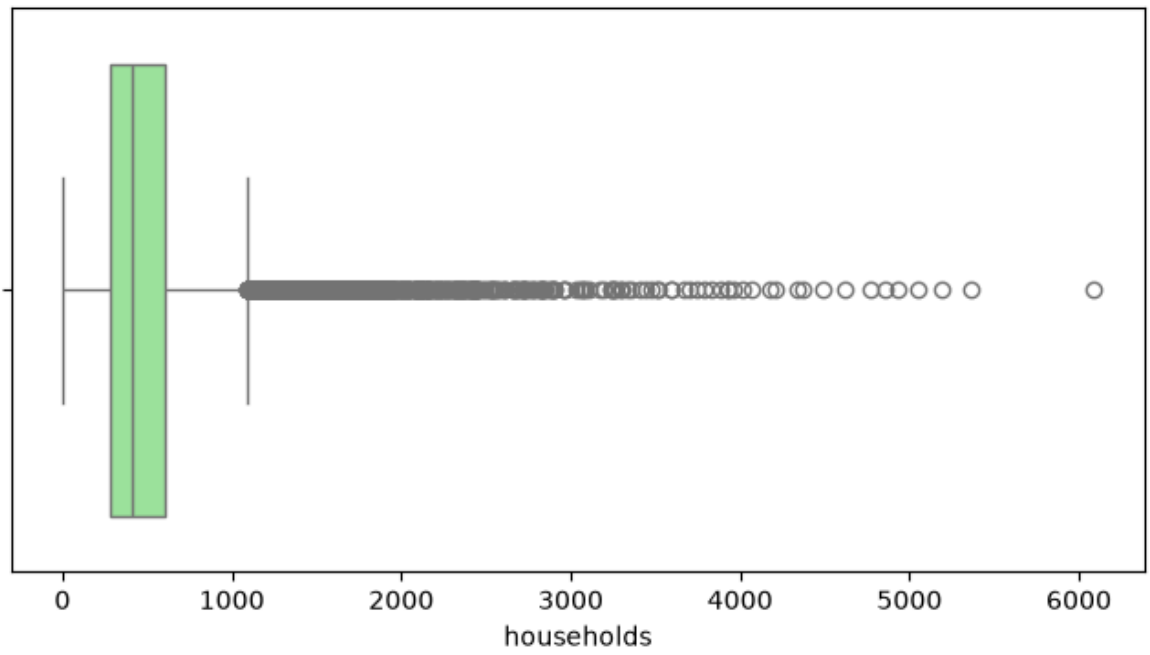
Box Plot of total_bedrooms



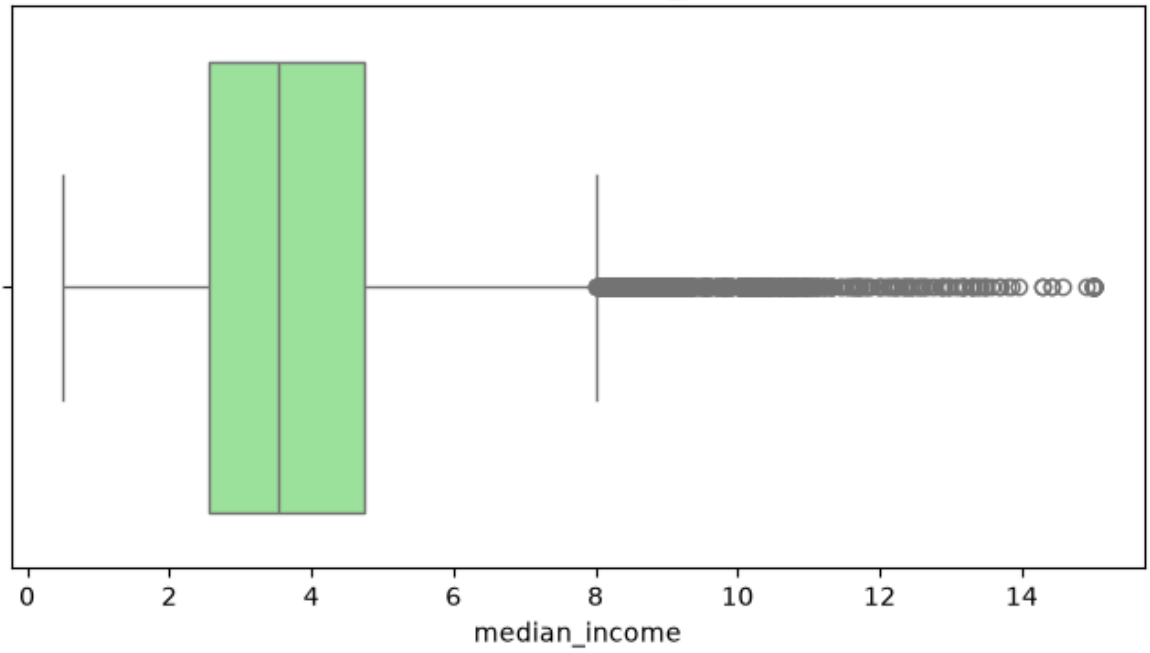
Box Plot of population

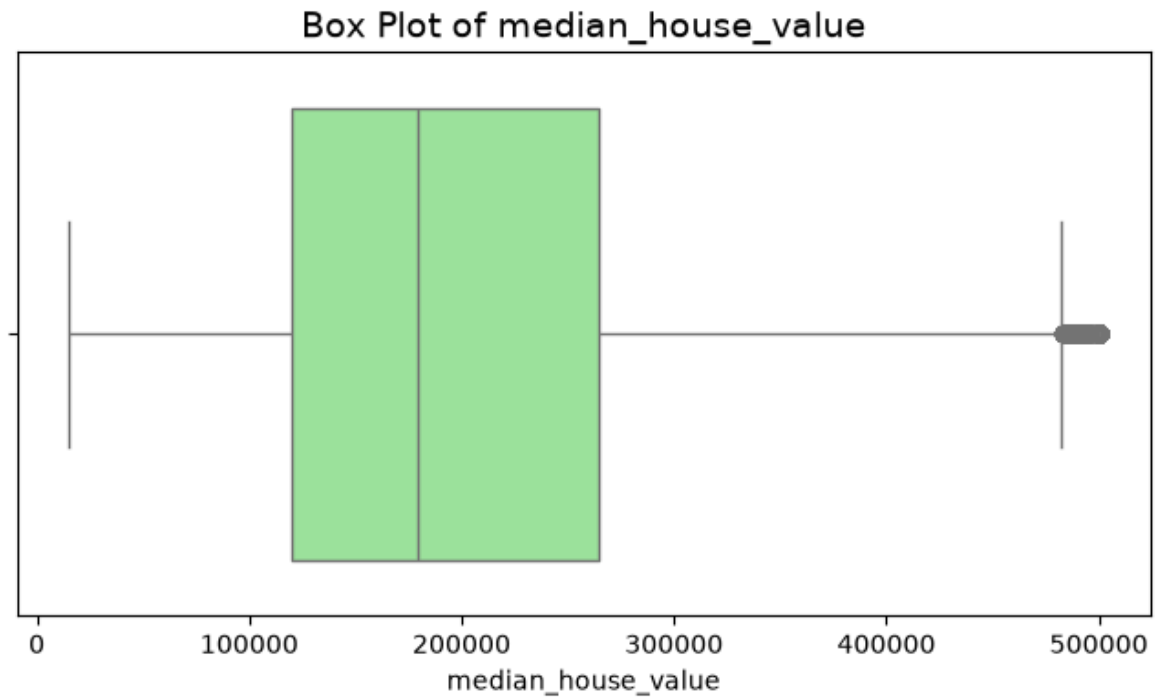


Box Plot of households



Box Plot of median_income





```
In [64]: '''  
1] Longitude:  
The box plot of longitude shows a moderate spread of values with a few outliers.  
  
2] Latitude:  
The latitude box plot indicates a reasonable distribution of housing locations w  
  
3] Housing Median Age:  
The housing median age has relatively few outliers, indicating that most houses  
  
4] Total Rooms:  
The total rooms feature contains a large number of outliers beyond the upper whi  
  
5] Total Bedrooms:  
The total bedrooms feature also shows numerous outliers. Most housing blocks hav  
  
6] Population:  
The population feature exhibits many extreme outliers. While most housing blocks  
  
7] Households:  
The households feature has several outliers above the upper whisker. These repre  
  
8] Median Income:  
The median income feature contains a few high-income outliers. Most housing bloc  
  
9] Median House Value (Target Variable):  
The median house value feature shows several high-value outliers and an upper li  
'''
```

```
Out[64]: '\n1] Longitude:\nThe box plot of longitude shows a moderate spread of values with a few outliers. These outliers represent housing locations that are farther east or west compared to the majority of California housing blocks.\n\n2] Latitude:\nThe latitude box plot indicates a reasonable distribution of housing locations with a few outliers. These outliers correspond to housing blocks located in less common northern or southern regions of California.\n\n3] Housing Median Age:\nThe housing median age has relatively few outliers, indicating that most houses fall within a similar age range. The majority of homes are between 20 and 40 years old, showing a balanced distribution of housing ages.\n\n4] Total Rooms:\nThe total rooms feature contains a large number of outliers beyond the upper whisker. These outliers represent housing blocks with exceptionally high numbers of rooms, indicating a highly right-skewed distribution.\n\n5] Total Bedrooms:\nThe total bedrooms feature also shows numerous outliers. Most housing blocks have a moderate number of bedrooms, while a small number contain exceptionally large bedroom counts, contributing to a positively skewed distribution.\n\n6] Population:\nThe population feature exhibits many extreme outliers. While most housing blocks have moderate population sizes, a few blocks contain very large populations, resulting in a long upper tail.\n\n7] Households:\nThe households feature has several outliers above the upper whisker. These represent housing blocks with unusually high numbers of households compared to the majority of the dataset.\n\n8] Median Income:\nThe median income feature contains a few high-income outliers. Most housing blocks have moderate income levels, while only a small number of areas report exceptionally high median incomes.\n\n9] Median House Value (Target Variable):\nThe median house value feature shows several high-value outliers and an upper limit near $500,000. This suggests that house prices were capped in the dataset, causing the highest-priced houses to be grouped at the maximum recorded value.\n'
```

```
In [65]: '''
Overall Analysis:
The box plots reveal that several numerical features contain significant outliers. Such outliers are common in real-world housing datasets and may represent densely populated neighborhoods or unusually large residential developments. Since these values appear to be valid observations rather than data entry errors, they were retained for further analysis and model building.\n'''
```

```
Out[65]: '\nOverall Analysis:\nThe box plots reveal that several numerical features contain significant outliers, particularly total_rooms, total_bedrooms, population, and households. These outliers indicate that some housing blocks have exceptionally large values compared to the majority of the dataset.\nSuch outliers are common in real-world housing datasets and may represent densely populated neighborhoods or unusually large residential developments. Since these values appear to be valid observations rather than data entry errors, they were retained for further analysis and model building.\n'
```

```
In [66]: #Scatter plots with Target Variable
target = "median_house_value"
features = numerical_cols.drop(target)

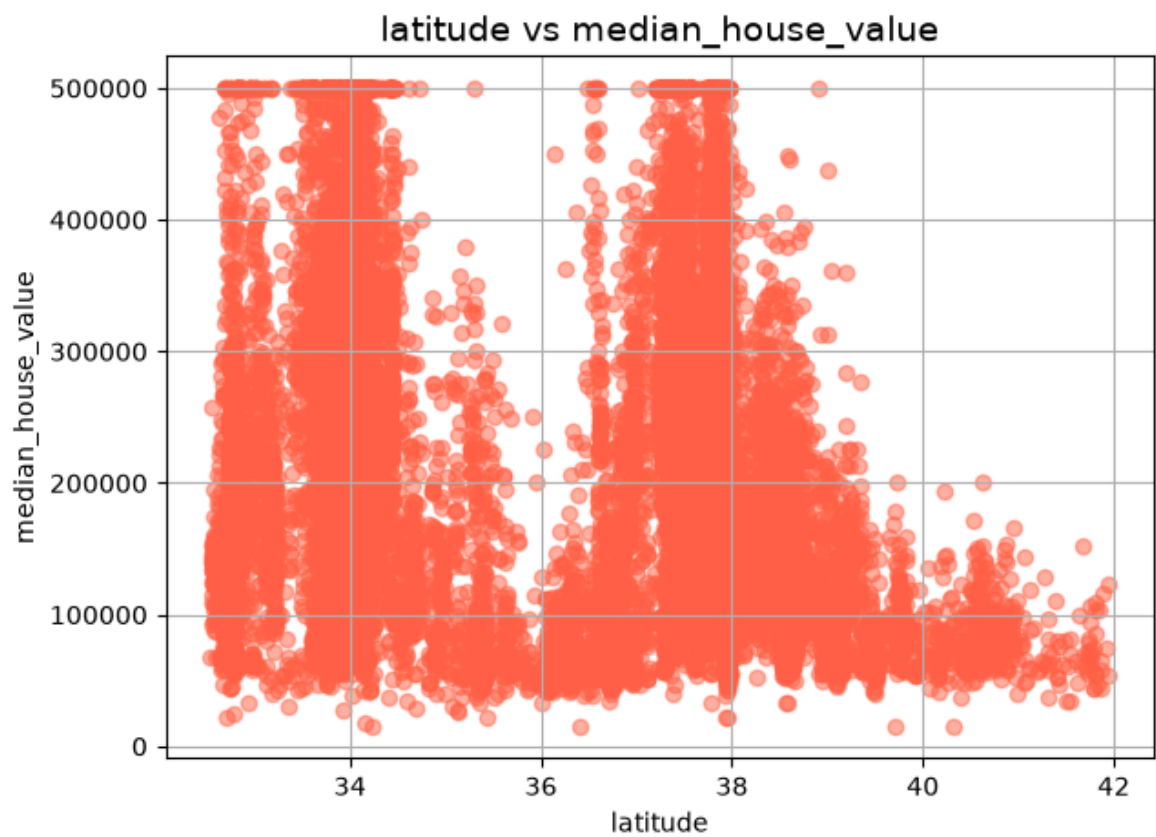
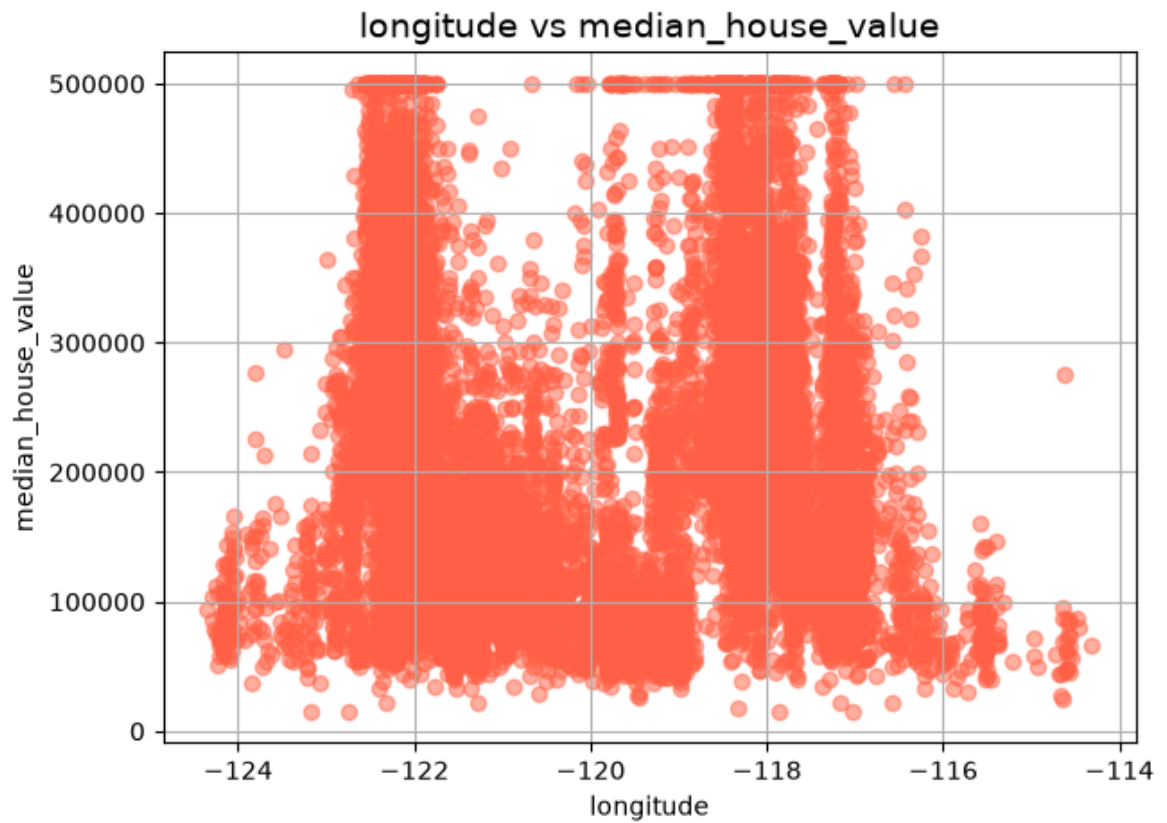
for col in features:
    plt.figure(figsize=(7,5))

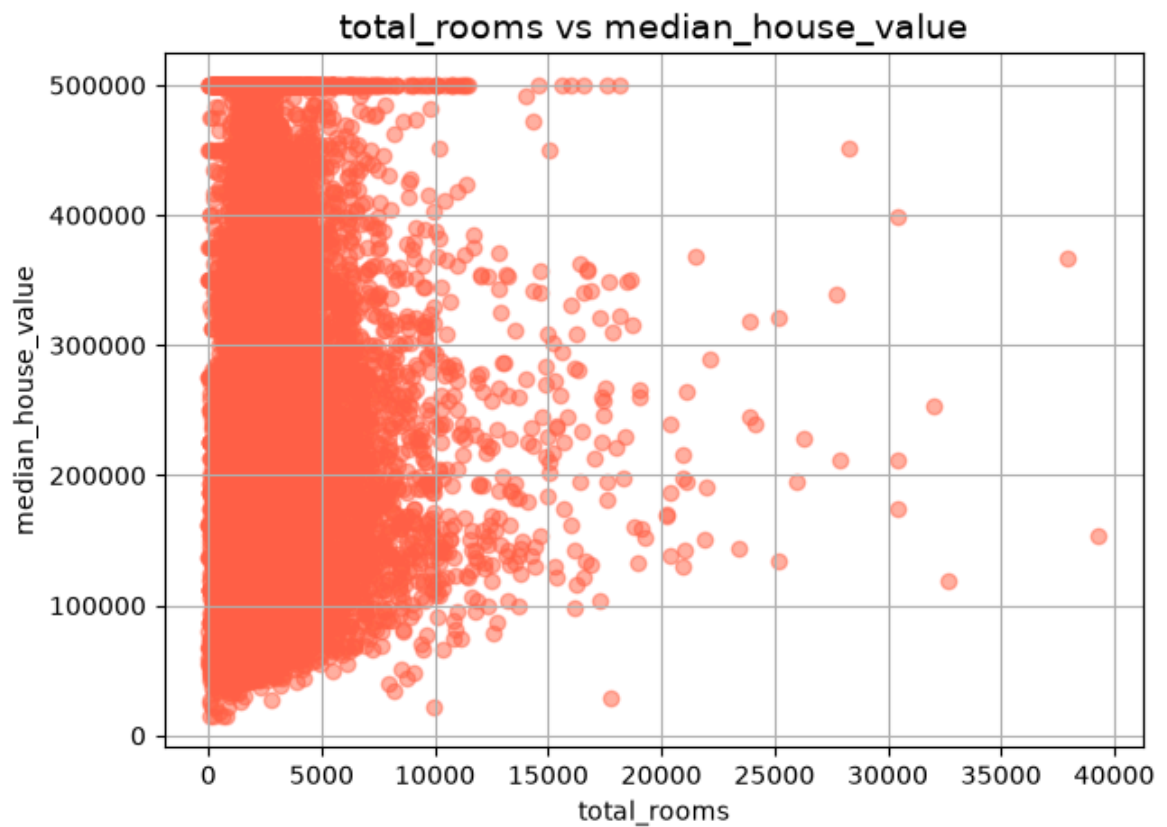
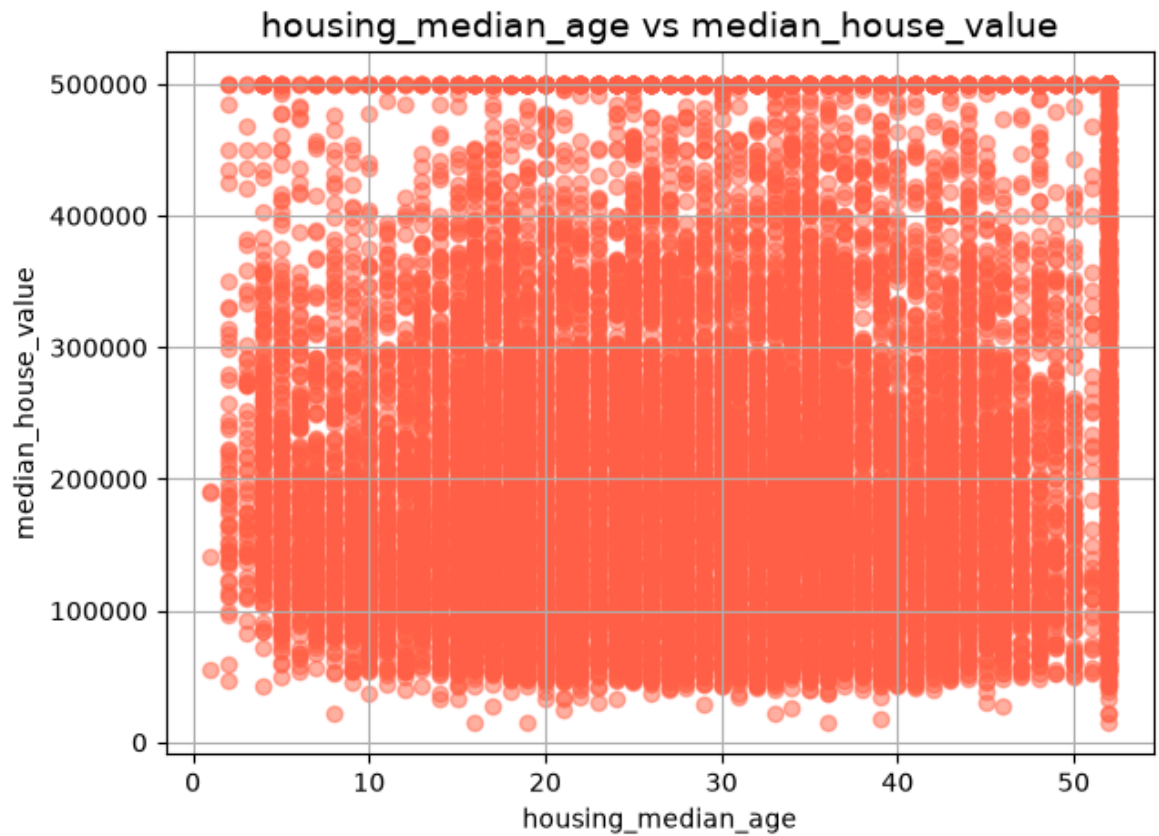
    plt.scatter(df[col],
                df[target],
                alpha=0.5,
                color="tomato")

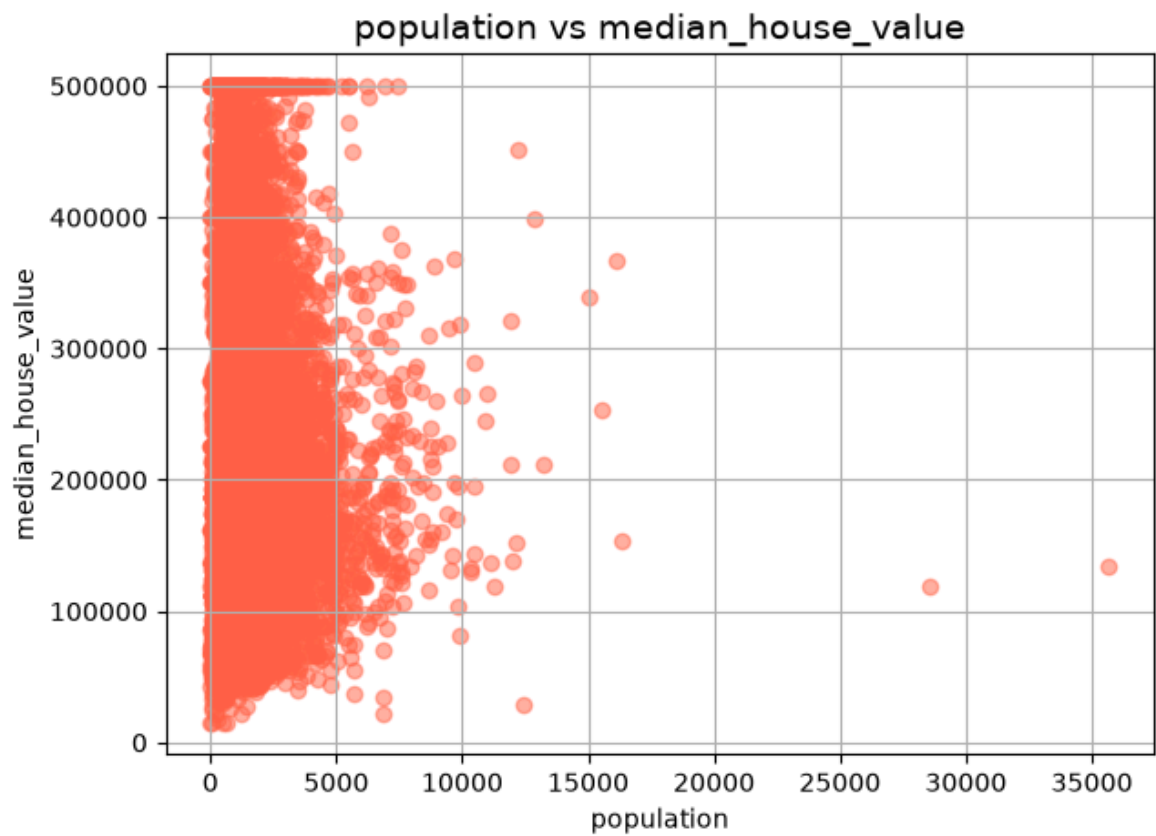
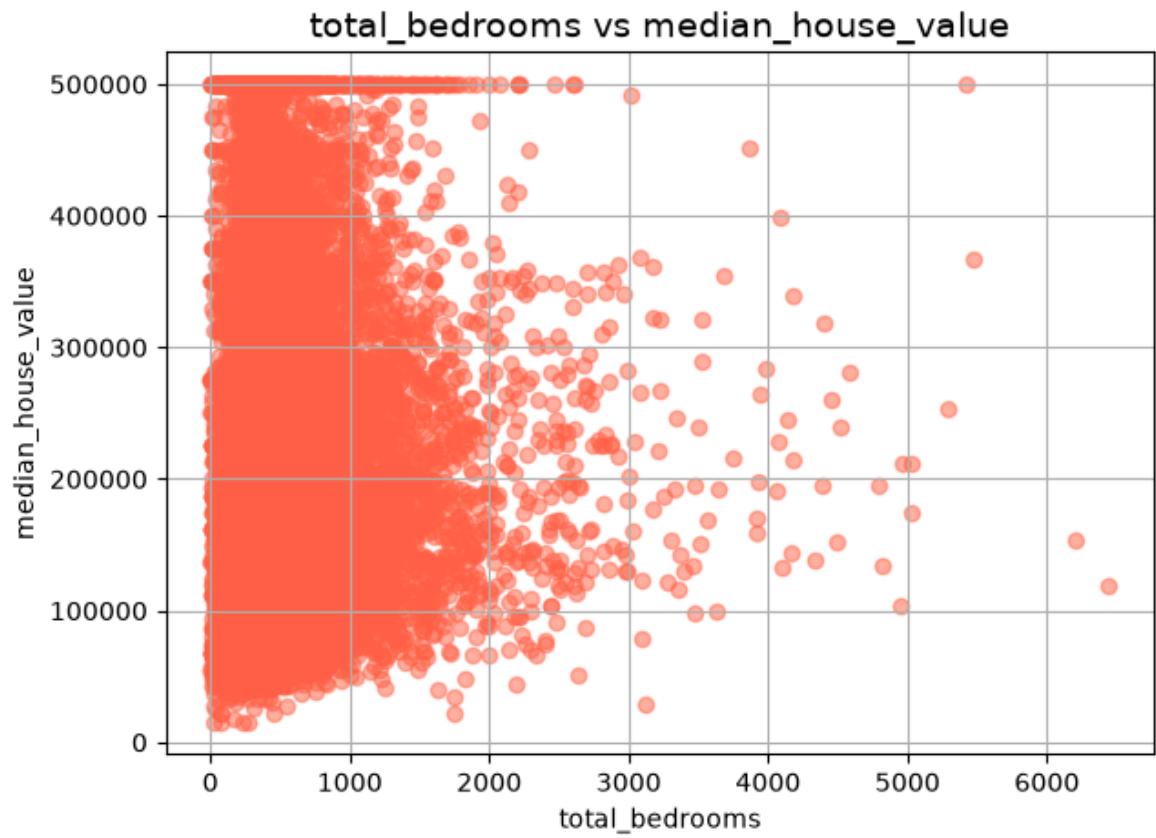
    plt.title(f"{col} vs {target}", fontsize=14)
    plt.xlabel(col)
    plt.ylabel(target)

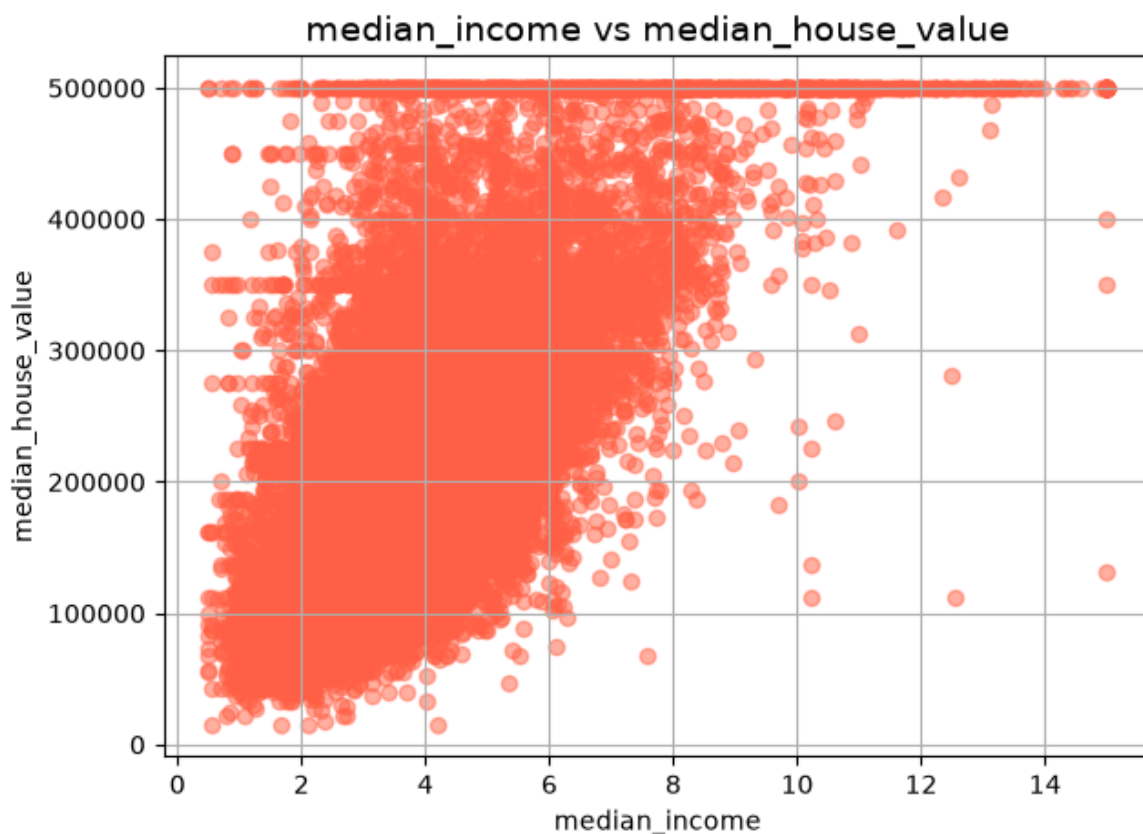
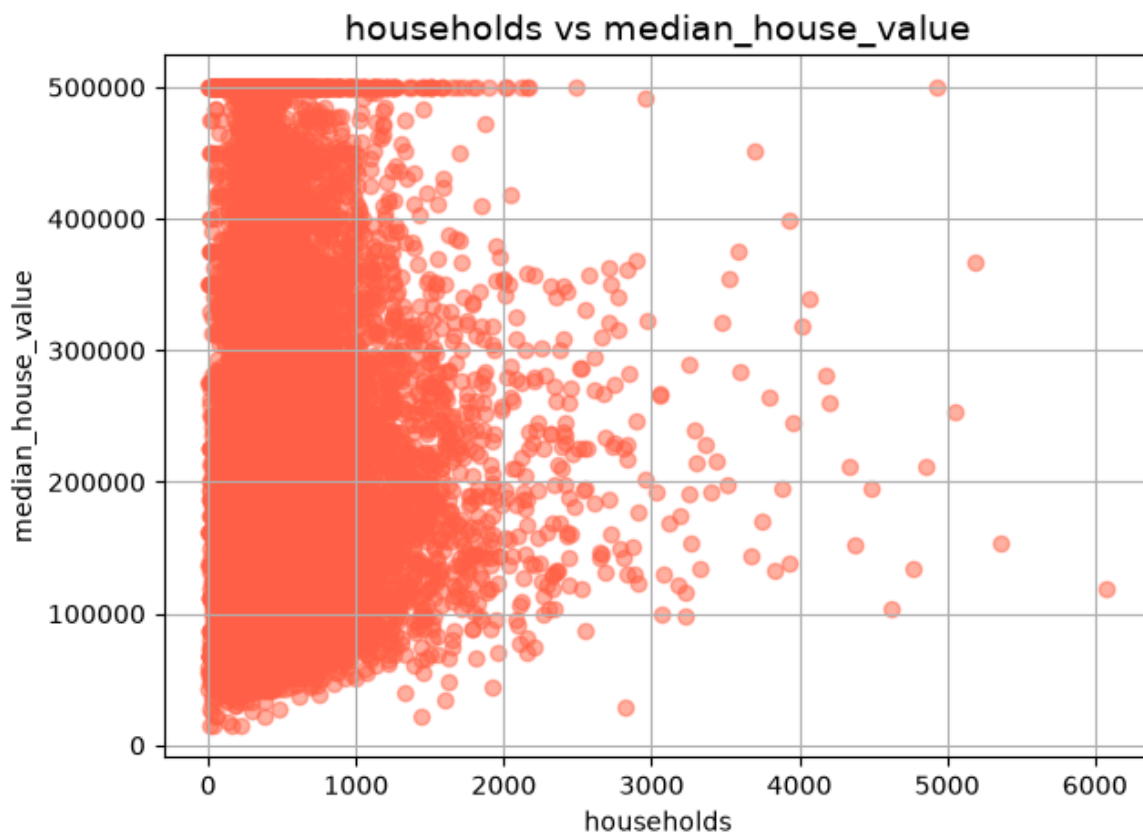
    plt.grid(True)
```

```
plt.show()
```









```
In [67]: '''
1] Longitude:
The scatter plot shows that house values vary across different longitudes. House

2] Latitude:
The latitude scatter plot indicates that house values change across different ge

3] Housing Median Age:
```


The relationship between housing median age and house value is weak to moderate.

4] Total Rooms:

The scatter plot shows a weak positive relationship between total rooms and median house value.

5] Total Bedrooms:

The total bedrooms feature shows a weak positive relationship with median house value.

6] Population:

The population feature exhibits a very weak relationship with house values. Housing blocks with similar populations have a wide range of house prices, indicating that population alone is not a reliable predictor.

7] Households:

The households feature shows a slight positive trend with house prices. However, the large variation in data points suggests that this feature has limited predictive power when used independently.

8] Median Income:

The scatter plot reveals a strong positive relationship between median income and median house value. As median income increases, house prices generally increase as well. This makes median income the strongest predictor of house prices in the dataset.

Overall Analysis:

The scatter plots indicate that median_income has the strongest positive relationship with the target variable (median_house_value), making it the best feature for the Simple Linear Regression model. Features such as population, households, total_rooms, and total_bedrooms exhibit weaker relationships, while longitude and latitude highlight the importance of geographical location in determining house prices.

```
Out[67]: '\n1] Longitude:\nThe scatter plot shows that house values vary across different longitudes. Houses located near the western coastal regions generally have higher median house values than those located inland, indicating that location significantly influences house prices.\n\n2] Latitude:\nThe latitude scatter plot indicates that house values change across different geographical regions. Higher house prices are observed in certain latitude ranges, suggesting that location plays an important role in determining property values.\n\n3] Housing Median Age:\nThe relationship between housing median age and house value is weak to moderate. Although some older houses have higher values, there is no clear linear trend, indicating that house age alone is not a strong predictor of house prices.\n\n4] Total Rooms:\nThe scatter plot shows a weak positive relationship between total rooms and median house value. Houses with more rooms tend to have higher prices, but the points are widely scattered, indicating that other factors also influence house values.\n\n5] Total Bedrooms:\nThe total bedrooms feature shows a weak positive relationship with median house value. While larger houses generally have higher prices, the wide spread of data points suggests that the relationship is not very strong.\n\n6] Population:\nThe population feature exhibits a very weak relationship with house values. Housing blocks with similar populations have a wide range of house prices, indicating that population alone is not a reliable predictor.\n\n7] Households:\nThe households feature shows a slight positive trend with house prices. However, the large variation in data points suggests that this feature has limited predictive power when used independently.\n\n8] Median Income:\nThe scatter plot reveals a strong positive relationship between median income and median house value. As median income increases, house prices generally increase as well. This makes median income the strongest predictor of house prices in the dataset.\n\nOverall Analysis:\n\nThe scatter plots indicate that median_income has the strongest positive relationship with the target variable (median_house_value), making it the best feature for the Simple Linear Regression model. Features such as population, households, total_rooms, and total_bedrooms exhibit weaker relationships, while longitude and latitude highlight the importance of geographical location in determining house prices.\n'
```

```
In [68]: #correlation heatmap
plt.figure(figsize=(10,8))

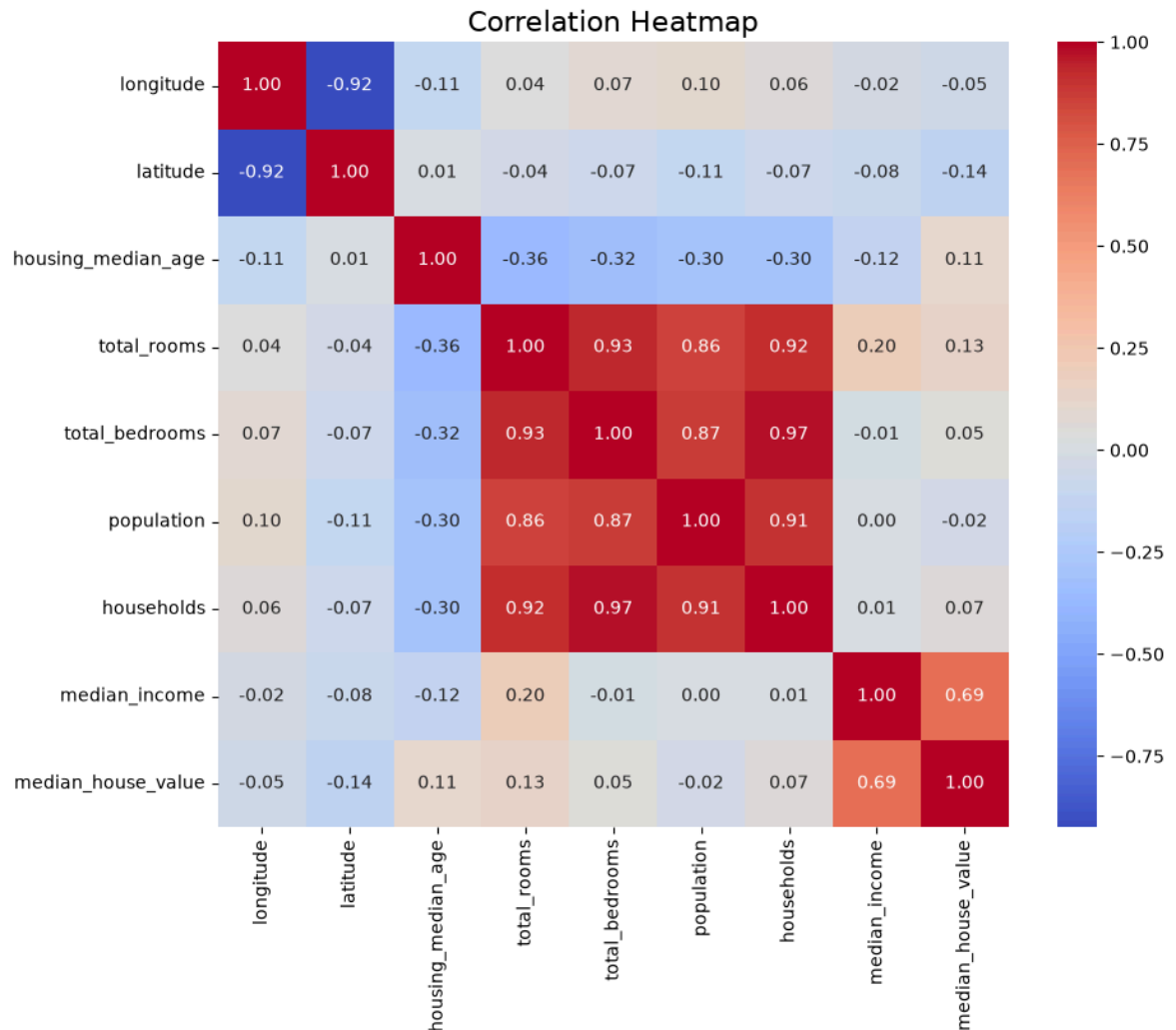
correlation = df[numerical_cols].corr()
```



```
sns.heatmap(correlation,
             annot=True,
             cmap="coolwarm",
             fmt=".2f")

plt.title("Correlation Heatmap", fontsize=16)

plt.show()
```



In [69]:

```
'''
Correlation Heatmap Analysis:

1] The correlation heatmap shows the strength and direction of the relationship
2] Median Income has the strongest positive correlation with Median House Value.
3] Longitude has a negative correlation with Median House Value, suggesting that
4] Latitude also shows a negative correlation with Median House Value. Certain 1
5] Total Rooms, Total Bedrooms, Population, and Households are strongly positive
6] Housing Median Age has a weak positive correlation with Median House Value, i
7] Population has a very weak correlation with Median House Value. This suggests
8] The heatmap helps identify the most useful features for machine learning. Bas
```

Overall Analysis:

The correlation heatmap provides a clear overview of relationships among the numerical features.

Out[69]: `'\nCorrelation Heatmap Analysis:\n\n1] The correlation heatmap shows the strength and direction of the relationship between all numerical features.\n\n2] Median Income has the strongest positive correlation with Median House Value. This indicates that areas with higher median incomes generally have higher house prices.\n\n3] Longitude has a negative correlation with Median House Value, suggesting that house prices tend to decrease as longitude increases. This reflects geographical differences in housing prices across California.\n\n4] Latitude also shows a negative correlation with Median House Value. Certain latitude regions have lower house values, while others, particularly coastal regions, have higher values.\n\n5] Total Rooms, Total Bedrooms, Population, and Households are strongly positively correlated with each other. This is expected because housing blocks with more rooms generally contain more bedrooms, households, and residents.\n\n6] Housing Median Age has a weak positive correlation with Median House Value, indicating that older houses may have slightly higher prices, but the relationship is not very strong.\n\n7] Population has a very weak correlation with Median House Value. This suggests that population alone is not a good predictor of house prices.\n\n8] The heatmap helps identify the most useful features for machine learning. Based on the correlation values, Median Income is selected as the best feature for the Simple Linear Regression model because it has the highest correlation with the target variable.\n\nOverall Analysis:\n\nThe correlation heatmap provides a clear overview of relationships among the numerical features. It confirms that Median Income is the strongest predictor of Median House Value, while geographical features such as Longitude and Latitude also influence housing prices. Features with strong correlations are useful for building predictive models and selecting important variables for regression analysis.'`

```
In [70]: import seaborn as sns
import matplotlib.pyplot as plt

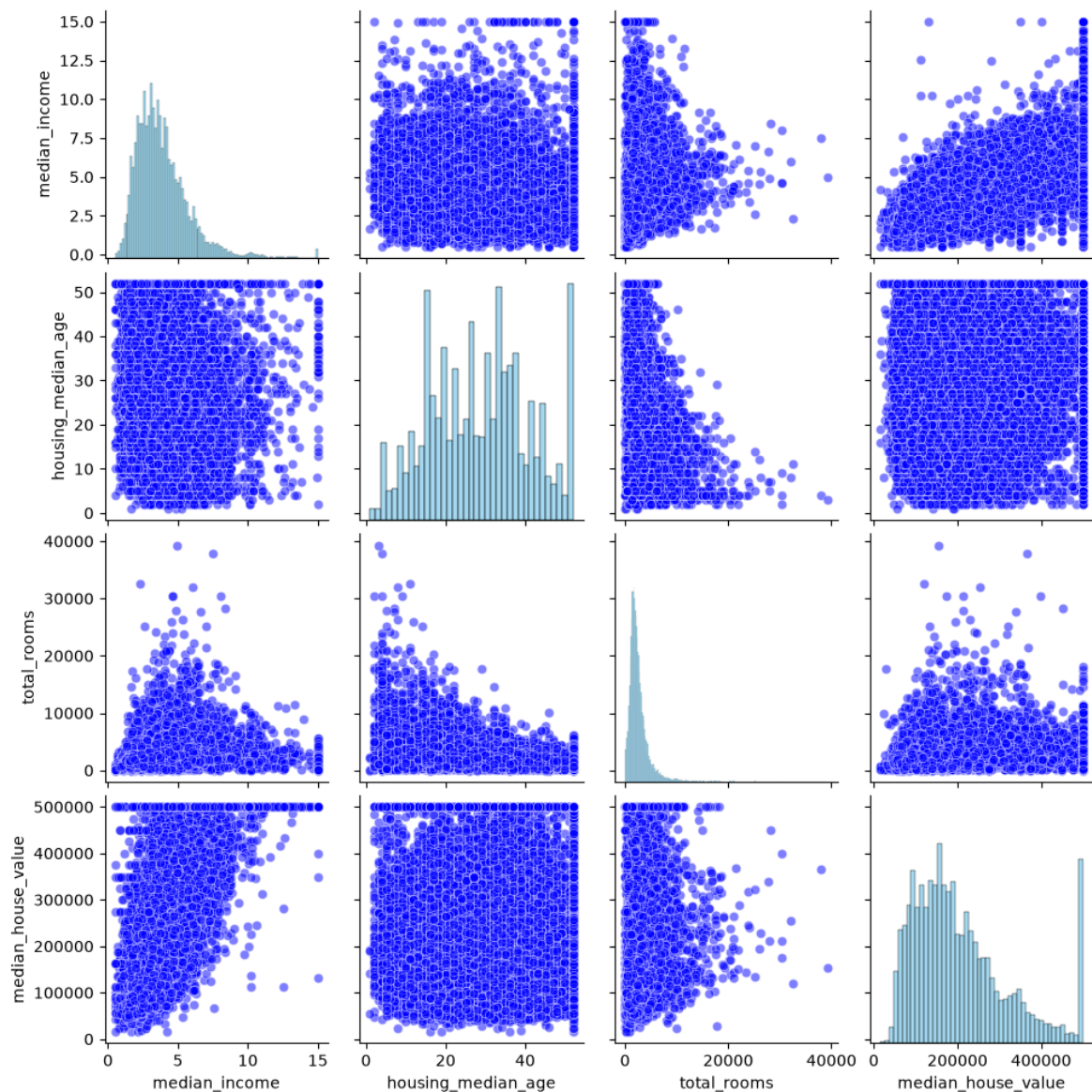
important_features = [
    "median_income",
    "housing_median_age",
    "total_rooms",
    "median_house_value"
]

sns.pairplot(
    df[important_features],
    diag_kind="hist",
    plot_kws={"alpha": 0.5, "color": "blue"},
    diag_kws={"color": "skyblue"}
)

plt.suptitle("Pair Plot of Important Housing Features", y=1.02, fontsize=16)

plt.show()
```

Pair Plot of Important Housing Features



In [71]: '''

Pair Plot Analysis:

1] Median Income:

The pair plot shows a strong positive relationship between median income and med
As the median income increases, the house prices generally increase, confirming

2] Housing Median Age:

Housing median age exhibits a weak to moderate relationship with median house va
Although some older houses have higher values, the relationship is not strongly

3] Total Rooms:

The total rooms feature shows a slight positive relationship with median house v
However, the data points are widely scattered, suggesting that the number of roo

4] Median House Value:

The target variable displays a positive association with median income but weake
The distribution also shows a concentration of values near the upper limit, indi

5] Distribution of Individual Features:

The diagonal histograms display the distribution of each feature. Median income

Overall Analysis:

```

The pair plot provides a comprehensive visualization of the relationships among
It confirms that median income has the strongest positive relationship with medi
Housing median age and total rooms show weaker relationships with the target var
'''

```

```

Out[71]: '\nPair Plot Analysis:\n\n1] Median Income:\nThe pair plot shows a strong posit
ive relationship between median income and median house value. \nAs the median
income increases, the house prices generally increase, confirming that median i
ncome is the strongest predictor of house prices.\n\n2] Housing Median Age:\nHo
using median age exhibits a weak to moderate relationship with median house val
ue.\nAlthough some older houses have higher values, the relationship is not str
ongly linear, indicating that age alone does not determine house prices.\n\n3]
Total Rooms:\nThe total rooms feature shows a slight positive relationship with
median house value.\nHowever, the data points are widely scattered, suggesting
that the number of rooms alone is not sufficient to accurately predict house pr
ices.\n\n4] Median House Value:\nThe target variable displays a positive associ
ation with median income but weaker relationships with housing median age and t
otal rooms. \nThe distribution also shows a concentration of values near the up
per limit, indicating that house prices were capped in the dataset.\n\n5] Distr
ibution of Individual Features:\nThe diagonal histograms display the distributi
on of each feature. Median income and median house value are positively skewed,
while total rooms contains several extreme values, indicating the presence of o
utliers.\n\nOverall Analysis:\n\nThe pair plot provides a comprehensive visuali
zation of the relationships among the selected features.\nIt confirms that medi
an income has the strongest positive relationship with median house value, maki
ng it the most suitable feature for Simple Linear Regression. \nHousing median
age and total rooms show weaker relationships with the target variable. The pai
r plot also highlights the presence of skewed distributions and outliers, which
are important considerations during model development.\n'

```

```

In [72]: # PART -4(Insights & Observations )

```

```

In [73]: '''
Observation 1
Median Income has the strongest positive correlation with Median House Value.
This indicates that areas with higher household incomes generally have higher ho

Observation 2
The distribution of Median House Value is positively (right) skewed, with most h
The distribution also shows an upper limit (around $500,000), indicating that ho

Observation 3
Features such as Total Rooms, Total Bedrooms, Population, and Households are hig
Most housing blocks have relatively small values, while a few blocks contain exc

Observation 4
The box plots reveal many outliers in Total Rooms, Total Bedrooms, Population, a
These outliers likely represent densely populated housing blocks or unusually la

Observation 5
Longitude and Latitude both influence house prices.
Houses located in coastal regions generally have higher median house values comp

Observation 6
Population has a very weak correlation with Median House Value. This suggests th

Observation 7
Housing Median Age has a weak positive relationship with house prices
While some older houses have higher values, house age alone is not a strong pred

```

Observation 8

The pair plot confirms that Median Income has the clearest positive linear relat

Observation 9

Several numerical features, particularly Total Rooms, Total Bedrooms, Population his suggests possible multicollinearity, which should be considered when buildin

Observation 10

the exploratory data analysis indicates that economic and geographical factors h
Therefore, features such as Median Income, Latitude, and Longitude are expect

```
Out[73]: '\nObservation 1\nMedian Income has the strongest positive correlation with Med
ian House Value. \nThis indicates that areas with higher household incomes gene
rally have higher house prices, making median income the best predictor for the
regression model.\n\nObservation 2\nThe distribution of Median House Value is p
ositively (right) skewed, with most houses priced in the low to middle range\nT
he distribution also shows an upper limit (around $500,000), indicating that ho
use prices were capped in the dataset\n\nObservation 3\nFeatures such as Total
Rooms, Total Bedrooms, Population, and Households are highly right-skewed\nMost
housing blocks have relatively small values, while a few blocks contain excepti
onally large values\n\nObservation 4\nThe box plots reveal many outliers in Tot
al Rooms, Total Bedrooms, Population, and Households\nThese outliers likely rep
resent densely populated housing blocks or unusually large residential areas ra
ther than data entry errors\nObservation 5\nLongitude and Latitude both influen
ce house prices. \nHouses located in coastal regions generally have higher medi
an house values compared to inland regions, highlighting the importance of geog
raphical location.\n\nObservation 6\nPopulation has a very weak correlation wit
h Median House Value. This suggests that simply having a larger population does
not necessarily result in higher housing prices.\n\nObservation 7\nHousing Medi
an Age has a weak positive relationship with house prices\nWhile some older hou
ses have higher values, house age alone is not a strong predictor of housing pr
ices\n\nObservation 8\nThe pair plot confirms that Median Income has the cleare
st positive linear relationship with Median House Value. Other features show we
aker and more scattered relationships with the target variable.\n\nObservation
9\nSeveral numerical features, particularly Total Rooms, Total Bedrooms, Popula
tion, and Households, are strongly correlated with one another \nhis suggests p
ossible multicollinearity, which should be considered when building a Multiple
Linear Regression model\n\nObservation 10\nthe exploratory data analysis indica
tes that economic and geographical factors have a greater influence on house pr
ices than demographic factors\n    Therefore, features such as Median Income, L
atitude, and Longitude are expected to improve prediction accuracy in regressio
n models'
```

```
In [74]: #PART-5(Simple Linear Regression )
```

```
In [75]: from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

from sklearn.metrics import (
    r2_score,
    mean_absolute_error,
    root_mean_squared_error
)
```

```
In [76]: from sklearn.metrics import mean_squared_error
```

```
In [77]: X = df[["median_income"]] #feature
y = df[["median_house_value"]] #target
```

```
In [78]: '''From the correlation heatmap, median_income showed the highest positive correlation with median_house_value\nThis indicates that neighborhoods with higher median incomes generally have higher house prices.\nTherefore, median_income was selected as the independent variable for the Simple Linear Regression model'
```

```
Out[78]: 'From the correlation heatmap, median_income showed the highest positive correlation with median_house_value\nThis indicates that neighborhoods with higher median incomes generally have higher house prices.\nTherefore, median_income was selected as the independent variable for the Simple Linear Regression model'
```

```
In [79]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)
```

```
In [80]: print("Training samples :", len(X_train))\nprint("Testing samples :", len(X_test))
```

Training samples : 16512

Testing samples : 4128

```
In [81]: #train model\nmodel = LinearRegression()\n\nmodel.fit(X_train, y_train)
```

```

Out[81]: <style>.sk-global { /* Definition of color scheme common for light and dark mode */ --
sklearn-color-text: #000; --sklearn-color-text-muted: #666; --sklearn-color-line: gray; /*
Definition of color scheme for unfitted estimators */ --sklearn-color-unfitted-level-0:
#fff5e6; --sklearn-color-unfitted-level-1: #f6e4d2; --sklearn-color-unfitted-level-2:
#ffe0b3; --sklearn-color-unfitted-level-3: chocolate; /* Definition of color scheme for
fitted estimators */ --sklearn-color-fitted-level-0: #f0f8ff; --sklearn-color-fitted-level-1:
#d4ebff; --sklearn-color-fitted-level-2: #b3dbfd; --sklearn-color-fitted-level-3:
cornflowerblue; } .sk-global.light { /* Specific color for light theme */ --sklearn-color-text-
on-default-background: black; --sklearn-color-background: white; --sklearn-color-border-
box: black; --sklearn-color-icon: #696969; } .sk-global.dark { --sklearn-color-text-on-
default-background: white; --sklearn-color-background: #111; --sklearn-color-border-
box: white; --sklearn-color-icon: #878787; } .sk-global { color: var(--sklearn-color-text); }
.sk-global pre { padding: 0; } .sk-global input.sk-hidden--visually { border: 0; clip-path:
inset(100%); height: 1px; margin: -1px; overflow: hidden; padding: 0; position: absolute;
width: 1px; } .sk-global div.sk-dashed-wrapped { border: 1px dashed var(--sklearn-color-
line); margin: 0 0.4em 0.5em 0.4em; box-sizing: border-box; padding-bottom: 0.4em;
background-color: var(--sklearn-color-background); } .sk-global div.sk-container { /*
jupyter's `normalize.less` sets `[hidden] { display: none; }` but bootstrap.min.css set
`[hidden] { display: none !important; }` so we also need the `!important` here to be able to
override the default hidden behavior on the sphinx rendered scikit-learn.org. See:
https://github.com/scikit-learn/scikit-learn/issues/21755 */ display: inline-block
!important; position: relative; } .sk-global div.sk-text-repr-fallback { display: none; } div.sk-
parallel-item, div.sk-serial, div.sk-item { /* draw centered vertical line to link estimators */
background-image: linear-gradient(var(--sklearn-color-text-on-default-background), var(-
sklearn-color-text-on-default-background)); background-size: 2px 100%; background-
repeat: no-repeat; background-position: center center; } /* Parallel-specific style estimator
block */ .sk-global div.sk-parallel-item::after { content: ""; width: 100%; border-bottom:
2px solid var(--sklearn-color-text-on-default-background); flex-grow: 1; } .sk-global div.sk-
parallel { display: flex; align-items: stretch; justify-content: center; background-color: var(--
sklearn-color-background); position: relative; } .sk-global div.sk-parallel-item { display: flex;
flex-direction: column; } .sk-global div.sk-parallel-item:first-child::after { align-self: flex-
end; width: 50%; } .sk-global div.sk-parallel-item:last-child::after { align-self: flex-start;
width: 50%; } .sk-global div.sk-parallel-item:only-child::after { width: 0; } /* Serial-specific
style estimator block */ .sk-global div.sk-serial { display: flex; flex-direction: column; align-
items: center; background-color: var(--sklearn-color-background); padding-right: 1em;
padding-left: 1em; } /* Toggleable style: style used for
estimator/Pipeline/ColumnTransformer box that is clickable and can be
expanded/collapsed. - Pipeline and ColumnTransformer use this feature and define the
default style - Estimators will overwrite some part of the style using the `sk-estimator`
class */ /* Pipeline and ColumnTransformer style (default) */ .sk-global div.sk-toggleable {
/* Default theme specific background. It is overwritten whether we have a specific
estimator or a Pipeline/ColumnTransformer */ background-color: var(--sklearn-color-
background); } /* Toggleable label */ .sk-global label.sk-toggleable__label { cursor: pointer;
display: flex; width: 100%; margin-bottom: 0; padding: 0.5em; box-sizing: border-box;
text-align: center; align-items: center; justify-content: center; gap: 0.5em; } .sk-global

```

```

label.sk-toggleable__label .caption { font-size: 0.6rem; font-weight: lighter; color: var(--
sklearn-color-text-muted); } .sk-global label.sk-toggleable__label-arrow:before { /* Arrow
on the left of the label */ content: " ▶ "; float: left; margin-right: 0.25em; color: var(--
sklearn-color-icon); } .sk-global label.sk-toggleable__label-arrow:hover:before { color: var(--
sklearn-color-text); } /* Toggleable content - dropdown */ .sk-global div.sk-
toggleable__content { display: none; text-align: left; /* unfitted */ background-color: var(--
sklearn-color-unfitted-level-0); } .sk-global div.sk-toggleable__content.fitted { /* fitted */
background-color: var(--sklearn-color-fitted-level-0); } .sk-global div.sk-
toggleable__content pre { margin: 0.2em; border-radius: 0.25em; color: var(--sklearn-
color-text); /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-
global div.sk-toggleable__content.fitted pre { /* unfitted */ background-color: var(--
sklearn-color-fitted-level-0); } .sk-global input.sk-toggleable__control:checked~div.sk-
toggleable__content { /* Expand drop-down */ display: block; width: 100%; overflow:
visible; } .sk-global input.sk-toggleable__control:checked~label.sk-toggleable__label-
arrow:before { content: " ▼ "; } /* Pipeline/ColumnTransformer-specific style */ .sk-global
div.sk-label input.sk-toggleable__control:checked~label.sk-toggleable__label { color: var(--
sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); } .sk-global
div.sk-label.fitted input.sk-toggleable__control:checked~label.sk-toggleable__label {
background-color: var(--sklearn-color-fitted-level-2); } /* Estimator-specific style */ /*
Colorize estimator box */ .sk-global div.sk-estimator input.sk-
toggleable__control:checked~label.sk-toggleable__label { /* unfitted */ background-color:
var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-estimator.fitted input.sk-
toggleable__control:checked~label.sk-toggleable__label { /* fitted */ background-color:
var(--sklearn-color-fitted-level-2); } .sk-global div.sk-label label.sk-toggleable__label, .sk-
global div.sk-label label { /* The background is the default theme color */ color: var(--
sklearn-color-text-on-default-background); } /* On hover, darken the color of the
background */ .sk-global div.sk-label:hover label.sk-toggleable__label { color: var(--
sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); } /* Label box,
darken color on hover, fitted */ .sk-global div.sk-label.fitted:hover label.sk-
toggleable__label.fitted { color: var(--sklearn-color-text); background-color: var(--sklearn-
color-fitted-level-2); } /* Estimator label */ .sk-global div.sk-label label { font-family:
monospace; font-weight: bold; line-height: 1.2em; } .sk-global div.sk-label-container {
text-align: center; } /* Estimator-specific */ .sk-global div.sk-estimator { font-family:
monospace; border: 1px dotted var(--sklearn-color-border-box); border-radius: 0.25em;
box-sizing: border-box; margin-bottom: 0.5em; /* unfitted */ background-color: var(--
sklearn-color-unfitted-level-0); } .sk-global div.sk-estimator.fitted { /* fitted */
background-color: var(--sklearn-color-fitted-level-0); } /* on hover */ .sk-global div.sk-
estimator:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-2); }
.sk-global div.sk-estimator.fitted:hover { /* fitted */ background-color: var(--sklearn-color-
fitted-level-2); } /* Specification for estimator info (e.g. "i" and "?") */ /* Common style for
"i" and "?" */ .sk-estimator-doc-link, a:link.sk-estimator-doc-link, a:visited.sk-estimator-
doc-link { float: right; font-size: smaller; line-height: 1em; font-family: monospace;
background-color: var(--sklearn-color-unfitted-level-0); border-radius: 1em; height: 1em;
width: 1em; text-decoration: none !important; margin-left: 0.5em; text-align: center; /*
unfitted */ border: var(--sklearn-color-unfitted-level-3) 1pt solid; color: var(--sklearn-

```



```

color-unfitted-level-3); } .sk-estimator-doc-link.fitted, a:link.sk-estimator-doc-link.fitted,
a:visited.sk-estimator-doc-link.fitted { /* fitted */ background-color: var(--sklearn-color-
fitted-level-0); border: var(--sklearn-color-fitted-level-3) 1pt solid; color: var(--sklearn-
color-fitted-level-3); } /* On hover */ div.sk-estimator: hover .sk-estimator-doc-link: hover,
.sk-estimator-doc-link: hover, div.sk-label-container: hover .sk-estimator-doc-link: hover,
.sk-estimator-doc-link: hover { /* unfitted */ background-color: var(--sklearn-color-
unfitted-level-3); border: var(--sklearn-color-fitted-level-0) 1pt solid; color: var(--sklearn-
color-unfitted-level-0); text-decoration: none; } div.sk-estimator.fitted: hover .sk-estimator-
doc-link.fitted: hover, .sk-estimator-doc-link.fitted: hover, div.sk-label-container: hover .sk-
estimator-doc-link.fitted: hover, .sk-estimator-doc-link.fitted: hover { /* fitted */
background-color: var(--sklearn-color-fitted-level-3); border: var(--sklearn-color-fitted-
level-0) 1pt solid; color: var(--sklearn-color-fitted-level-0); text-decoration: none; } /* Span,
style for the box shown on hovering the info icon */ .sk-estimator-doc-link span { display:
none; z-index: 9999; position: relative; font-weight: normal; right: .2ex; padding: .5ex;
margin: .5ex; width: min-content; min-width: 20ex; max-width: 50ex; color: var(--sklearn-
color-text); box-shadow: 2pt 2pt 4pt #999; /* unfitted */ background: var(--sklearn-color-
unfitted-level-0); border: .5pt solid var(--sklearn-color-unfitted-level-3); } .sk-estimator-
doc-link.fitted span { /* fitted */ background: var(--sklearn-color-fitted-level-0); border:
var(--sklearn-color-fitted-level-3); } .sk-estimator-doc-link: hover span { display: block; } /*
"-specific style due to the `` HTML tag */ .sk-global a.estimator_doc_link { float: right;
font-size: 1rem; line-height: 1em; font-family: monospace; background-color: var(--
sklearn-color-unfitted-level-0); border-radius: 1rem; height: 1rem; width: 1rem; text-
decoration: none; /* unfitted */ color: var(--sklearn-color-unfitted-level-1); border: var(--
sklearn-color-unfitted-level-1) 1pt solid; } .sk-global a.estimator_doc_link.fitted { /* fitted
*/ background-color: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-
level-1) 1pt solid; color: var(--sklearn-color-fitted-level-1); } /* On hover */ .sk-global
a.estimator_doc_link: hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-
level-3); color: var(--sklearn-color-background); text-decoration: none; } .sk-global
a.estimator_doc_link.fitted: hover { /* fitted */ background-color: var(--sklearn-color-fitted-
level-3); } .sk-top-container.sk-global { /* pydata-sphinx-theme hides overflow, so
scrolling is disabled. We need to set it to !important and add tabindex="0" in the HTML to
allow keyboard-only users to navigate the display. */ overflow-x: scroll !important; max-
width: 100%; } .estimator-table { font-family: monospace; } .estimator-table summary {
padding: .5rem; cursor: pointer; } .estimator-table summary::marker { font-size: 0.7rem; }
.estimator-table details[open] { padding-left: 0.1rem; padding-right: 0.1rem; padding-
bottom: 0.3rem; } .estimator-table .parameters-table { margin-left: auto !important;
margin-right: auto !important; margin-top: 0; } .estimator-table .parameters-table tr:nth-
child(odd) { background-color: #fff; } .estimator-table .parameters-table tr:nth-child(even)
{ background-color: #f6f6f6; } .estimator-table .parameters-table tr: hover td { background-
color: #e0e0e0; } .estimator-table table :is(td, th) { border: 1px solid rgba(106, 105, 104,
0.232); } /* `table td` is set in notebook with right text-align. We need to overwrite it. */
.estimator-table table td.param { text-align: left; position: relative; padding: 0; } .user-set td
{ color: rgb(255, 94, 0); text-align: left !important; } .user-set td.value { color: rgb(255, 94, 0);
background-color: transparent; } .default td, .estimator-table th { color: black; text-align:
left !important; } .user-set td i, .default td i { color: black; } td.fitted-att-type { white-space:

```

```

preserve nowrap; } /* Styles for parameter documentation links We need styling for visited
so jupyter doesn't overwrite it */ a.param-doc-link, a.param-doc-link:link, a.param-doc-
link:visited { text-decoration: underline dashed; text-underline-offset: .3em; color: inherit;
display: block; padding: .5em; } @supports(anchor-name: --doc-link) { a.param-doc-link,
a.param-doc-link:link, a.param-doc-link:visited { anchor-name: --doc-link; } } /* "hack" to
make the entire area of the cell containing the link clickable */ a.param-doc-link::before {
position: absolute; content: ""; inset: 0; } .param-doc-description { display: none; position:
absolute; z-index: 9999; left: 0; padding: .5ex; margin-left: 1.5em; color: var(--sklearn-
color-text); box-shadow: .3em .3em .4em #999; width: max-content; text-align: left; max-
height: 10em; overflow-y: auto; /* unfitted */ background: var(--sklearn-color-unfitted-
level-0); border: thin solid var(--sklearn-color-unfitted-level-3); } @supports(position-area:
center right) { .param-doc-description { position-area: center right; position: fixed; margin-
left: 0; } } /* Fitted state for parameter tooltips */ .fitted .param-doc-description { /* fitted
*/ background: var(--sklearn-color-fitted-level-0); border: thin solid var(--sklearn-color-
fitted-level-3); } .param-doc-link:hover .param-doc-description { display: block; } .copy-
paste-icon { background-image:
url(data:image/svg+xml;base64,PHN2ZyB4bWxucz0iaHR0cDovL3d3dy53My5vcmcvMjAwMC9:
background-repeat: no-repeat; background-size: 14px 14px; background-position: 0;
display: inline-block; width: 14px; height: 14px; cursor: pointer; } .features { font-family:
monospace; cursor: pointer; background-color: var(--sklearn-color-unfitted-level-0);
border: 1px dotted var(--sklearn-color-border-box); border-radius: .20em; margin-bottom:
0.5em; font-size: inherit; /* Needed for jupyter */ } .features.fitted { background-color:
var(--sklearn-color-fitted-level-0); } .features summary { cursor: pointer; display: flex;
margin-bottom: 0; text-align: center; align-items: center; justify-content: center; gap:
0.5em; padding: .25em; } .features details[open] > summary { color: var(--sklearn-color-
text); background-color: var(--sklearn-color-unfitted-level-2); border-radius: .20em 0 0 0; }
.features.fitted details[open] > summary { background-color: var(--sklearn-color-fitted-
level-2); border-radius: .20em 0 0 0; } .features details > summary .arrow::before { content:
" ▶ "; color: grey; } .features details[open] > summary .arrow::before { content: " ▼ "; }
.features details:hover > summary { margin: 0; background-color: var(--sklearn-color-
unfitted-level-2); } .features.fitted details:hover > summary { margin: 0; background-color:
var(--sklearn-color-fitted-level-2); } .features .features-container { max-width: 15em; max-
height: 10em; overflow: auto; scrollbar-width: thin; padding: .25em 0.1rem; background-
color: var(--sklearn-color-unfitted-level-0); border-radius: 0 0 .5em .5em; } .features.fitted
.features-container { background-color: var(--sklearn-color-fitted-level-0); } .features
.image-container { block-size: 1em; inline-size: 1em; padding: 0; margin: 0%; display: flex;
justify-content: center; align-items: center; } .features .copy-paste-icon { background-size:
1em 1em; width: 1em; height: 1em; filter: grayscale(100%) opacity(60%); } .features
.features-container table { width: 100%; margin: 0.01em; } .features .features-container
table tr:nth-child(odd) { background-color: #fff; } .features .features-container table tr:nth-
child(even) { background-color: #f6f6f6; } .features .features-container table tr:hover {
background-color: #e0e0e0; } .features .features-container table { table-layout: inherit; }
.features .features-container table td { text-align: left; padding: 0 0.5em; border: 1px solid
rgba(106, 105, 104, 0.232); white-space: nowrap; color: var(--sklearn-color-text); }
.total_features { display: flex; justify-content: center; margin-top: 0.5em; } </style> <body>

```

LinearRegression()

**In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.**

☒ sk-estimator-id-1 sk-toggleable__label sk-toggleable__label-arrow

LinearRegression

[?Documentation for LinearRegression](#)

sk-estimator-id-1

sk-estimator-id-1 sk-toggleable__label sk-toggleable__label-arrow

fit_intercept **fit_intercept**: bool, default=True
Whether to calculate the intercept for this model. If set to False, no intercept will be used in calculations (i.e. data is expected to be centered).

True

copy_X **copy_X**: bool, default=True
If True, X will be copied; else, it may be overwritten.

True

tol **tol**: float, default=1e-6
The precision of the solution (coef_) is determined by 'tol' which specifies the convergence criterion of the underlying solver. 'tol' is set as 'atol' and 'btol' of :func:`scipy.sparse.linalg.lsqr` when fitting on sparse training data. 'tol' is set as 'cond' of :func:`scipy.linalg.lstsq` when fitting on dense training data.
.. versionadded:: 1.7
.. versionchanged:: 1.9
Now supported on dense data, interpreted as the 'cond' parameter.

1e-06

n_jobs **n_jobs**: int, default=None
The number of jobs to use for the computation. This will only provide speedup in case of sufficiently large problems, that is if firstly 'n_targets > 1' and secondly 'X' is sparse or if 'positive' is set to 'True'. 'None' means 1 unless in a :obj:`joblib.parallel\_backend` context. '-1' means using all processors. See :term:`Glossary` **n_jobs** for more details.

None

positive **positive**: bool, default=False
When set to "True", forces the coefficients to be positive. This option is only supported for dense arrays.
For a comparison between a linear regression model with positive constraints on the regression coefficients and a linear regression without such constraints, see :ref:`sphx\_glr\_auto\_examples\_linear\_model\_plot\_nnls.py`.
.. versionadded:: 0.24

False

sk-estimator-id-1

sk-estimator-id-1 sk-toggleable__label sk-toggleable__label-arrow

sk-estimator-id-1 sk-toggleable__label sk-toggleable__label-arrow

coef_ **coef_**: array of shape (n_features,) or (n_targets, n_features)
Estimated coefficients for the linear regression problem. If multiple targets are passed during the fit (y 2D), this is a 2D array of shape (n_targets, n_features), while if only one target is passed, this is a 1D array of length n_features.

ndarray[float64]
(1,)

[41933.85]

feature_names_in_ **feature_names_in_**: ndarray of shape (n_features_in_,)
Names of features seen during :term:`fit`. Defined only when 'X' has feature names that are all strings.
.. versionadded:: 1.0

ndarray[object]
(1,)

['median_income']

<code>intercept_</code> : float or array of shape (n_targets,) Independent term in the linear model. Set to 0.0 if <code>fit_intercept = False</code> .	float64	4.446e+04
<code>n_features_in_</code> : int Number of features seen during :term:`fit`. versionadded:: 0.24	int	1
<code>rank_</code> : int Rank of matrix `X`. Only available when `X` is dense.	int64	np.int64(1)
<code>singular_</code> : array of shape (min(X, y)) Singular values of `X`. Only available when `X` is dense.	ndarray[float64] (1,)	[244.69]

</details>

```

<script> /* Authors: The scikit-learn developers
SPDX-License-Identifier: BSD-3-Clause */
function copyToClipboard(text, element) { // Get the parameter prefix from the closest
toggleable content
const toggleableContent = element.closest('.sk-toggleable__content');
const paramPrefix = toggleableContent ? toggleableContent.dataset.paramPrefix : '';
const fullParamName = paramPrefix ? `paramPrefix{text}` : text;
const originalStyle = element.style;
const computedStyle = window.getComputedStyle(element);
const originalWidth = computedStyle.width;
const originalHTML = element.innerHTML.replace('Copied!', '');
navigator.clipboard.writeText(fullParamName).then(() => {
element.style.width = originalWidth;
element.style.color = 'green';
element.innerHTML = "Copied!";
setTimeout(() => {
element.innerHTML = originalHTML;
element.style = originalStyle;
}, 2000);
}).catch(err => {
console.error('Failed to copy:', err);
element.style.color = 'red';
element.innerHTML = "Failed!";
setTimeout(() => {
element.innerHTML = originalHTML;
element.style = originalStyle;
}, 2000);
});
return false;
}
document.querySelectorAll('.copy-paste-icon').forEach(function(element) {
const toggleableContent = element.closest('.sk-toggleable__content');
const paramPrefix = toggleableContent ? toggleableContent.dataset.paramPrefix : '';
const parent = element.parentElement;
if (!parent || !parent.nextElementSibling) {
console.warn('Expected copy-paste icon is missing from the DOM structure');
return;
}
const paramName = element.parentElement.nextElementSibling.textContent.trim().split(' ')[0];
const fullParamName = paramPrefix ? `paramPrefix{paramName}` : paramName;
element.setAttribute('title', fullParamName);
});
/** * Copy the list of feature names formatted as a Python list.
* * @param {HTMLElement} element - The copy button inside a `features` block;
its siblings * contain a `details` element and a table containing feature
named.
* @returns {boolean} Always returns `false` so callers can prevent the default
click behavior.
*/
function copyFeatureNamesToClipboard(element) {
var detailsElem = element.closest('.features').querySelector('details');
var wasOpen = detailsElem.open;
detailsElem.open = true;
var content = element.closest('.features').querySelector('tbody').innerHTML.trim();
if (!wasOpen) detailsElem.open = false;
const rows = content.split('\n').map(row => `row`);
const formattedText = `[\n${rows.join(',\n')},\n]`;
const originalHTML = element.innerHTML.replace('âœ“', '');
const originalStyle = element.style;
const copyMark = document.createElement('span');
copyMark.innerHTML = 'âœ“';
copyMark.style.color = 'blue';
copyMark.style.fontSize = '1em';
navigator.clipboard.writeText(formattedText).then(() => {
element.style.display =

```

```
'none'; element.parentElement.appendChild(copyMark); setTimeout(() => {
copyMark.remove(); element.innerHTML = originalHTML; element.style = originalStyle; },
1000); }) .catch(err => { console.error('Failed to copy:', err); element.style.color = 'orange';
element.innerHTML = "Failed!"; setTimeout(() => { element.innerHTML = originalHTML;
element.style = originalStyle; }, 1000); }); return false; } /** * Adapted from Skrub *
https://github.com/skrub-
data/skrub/blob/403466d1d5d4dc76a7ef569b3f8228db59a31dc3/skrub/_reporting/_data/tem
* @returns "light" or "dark" */ function detectTheme(element) { const body =
document.querySelector('body'); // Check VSCode theme const themeKindAttr =
body.getAttribute('data-vscode-theme-kind'); const themeNameAttr =
body.getAttribute('data-vscode-theme-name'); if (themeKindAttr && themeNameAttr) {
const themeKind = themeKindAttr.toLowerCase(); const themeName =
themeNameAttr.toLowerCase(); if (themeKind.includes("dark") ||
themeName.includes("dark")) { return "dark"; } if (themeKind.includes("light") ||
themeName.includes("light")) { return "light"; } } // Check Jupyter theme if
(body.getAttribute('data-jp-theme-light') === 'false') { return 'dark'; } else if
(body.getAttribute('data-jp-theme-light') === 'true') { return 'light'; } // Guess based on a
parent element's color const color = window.getComputedStyle(element.parentNode,
null).getPropertyValue('color'); const match = color.match(/^rgb\s*
\s* (\d+) \s*, \s* (\d+) \s*, \s* (\d+) \s*$/i); if (match) { const [r, g, b] = [
parseFloat(match[1]), parseFloat(match[2]), parseFloat(match[3]) ]; //
https://en.wikipedia.org/wiki/HSL_and_HSV#Lightness const luma = 0.299 * r + 0.587 * g
+ 0.114 * b; if (luma > 180) { // If the text is very bright we have a dark theme return
'dark'; } if (luma < 75) { // If the text is very dark we have a light theme return 'light'; } //
Otherwise fall back to the next heuristic. } // Fallback to system preference return
window.matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' : 'light'; } function
forceTheme(elementId) { const estimatorElement =
document.querySelector(`#${elementId}`); if (estimatorElement === null) {
console.error(`Element with id ${elementId} not found.`); } else { const theme =
detectTheme(estimatorElement); estimatorElement.classList.add(theme); } }
forceTheme('sk-container-id-1');
```

```
In [82]: #display slope and intercept
print("Slope (Coefficient):", model.coef_[0])

print("Intercept:", model.intercept_)
```

```
Slope (Coefficient): 41933.84939381274
Intercept: 44459.729169078666
```

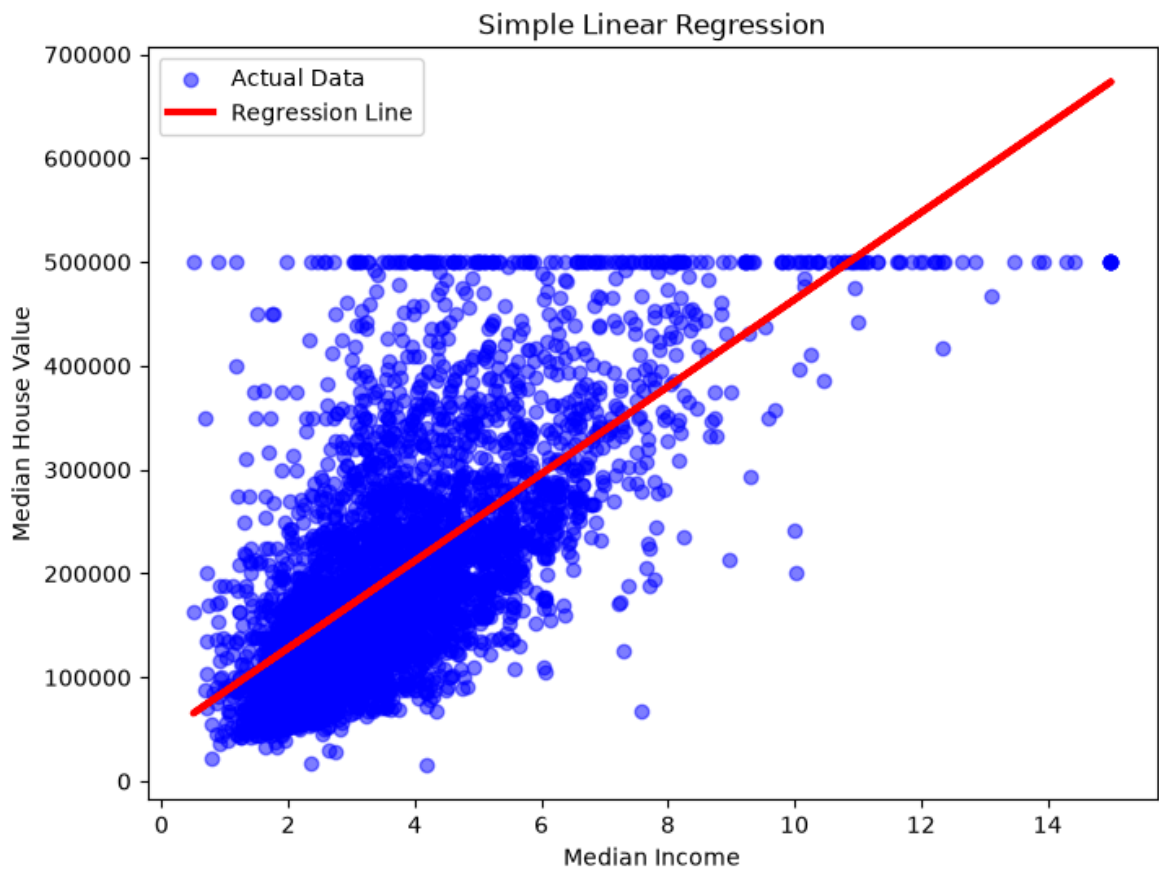
```
In [83]: #prediction
y_pred = model.predict(X_test)
```

```
In [84]: #scatter plot with regression
plt.figure(figsize=(8,6))

plt.scatter(X_test, y_test, color="blue", alpha=0.5, label="Actual Data")
plt.plot(X_test, y_pred, color="red", linewidth=3, label="Regression Line")
```

```
plt.title("Simple Linear Regression")
plt.xlabel("Median Income")
plt.ylabel("Median House Value")
plt.legend()

plt.show()
```



In [85]: '''The regression line shows a positive relationship between median income and median house value. Although the data points are scattered around the line, an increasing trend is clearly visible. This indicates that higher median income is generally associated with higher house prices.'''

Out[85]: 'The regression line shows a positive relationship between median income and median house value\n\nAlthough the data points are scattered around the line, an increasing trend is clearly visible. \nThis indicates that higher median income is generally associated with higher house prices'

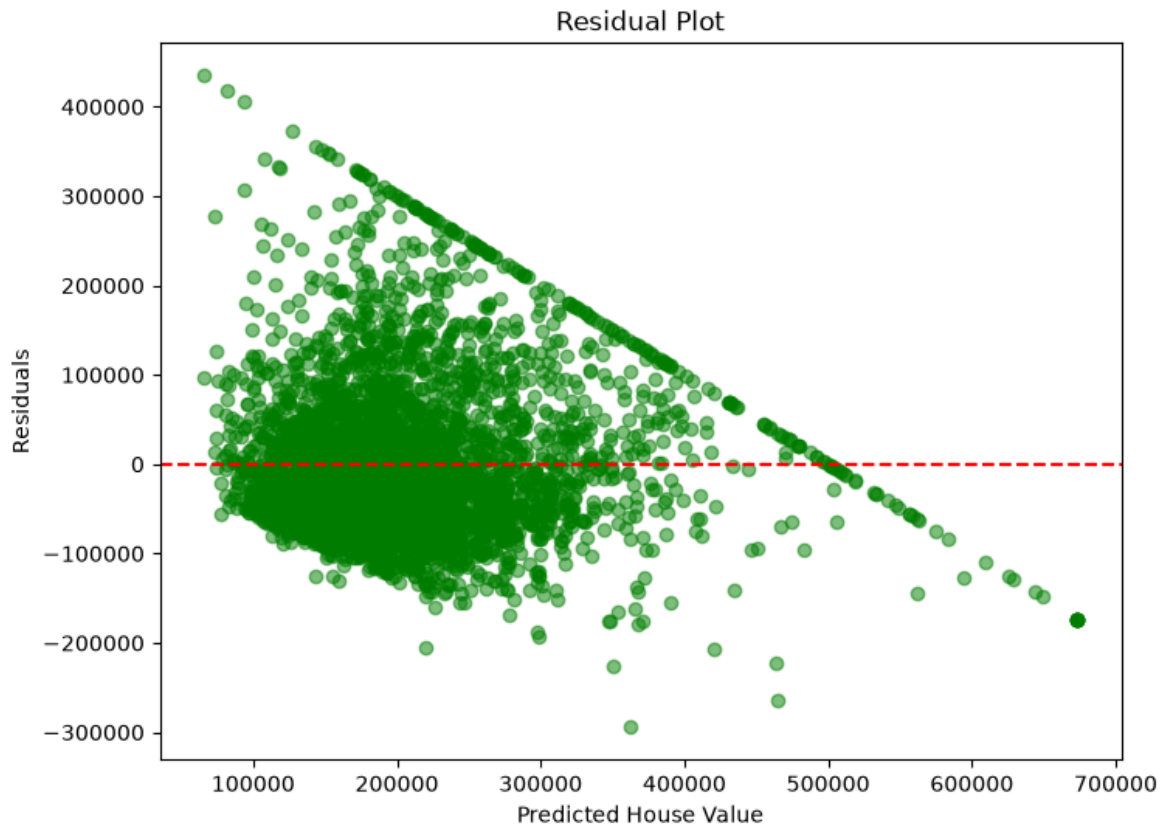
```
In [86]: #Residual plot
residuals = y_test - y_pred

plt.figure(figsize=(8,6))
plt.scatter(y_pred, residuals, color="green", alpha=0.5)

plt.axhline(y=0, color="red", linestyle="--")

plt.title("Residual Plot")
plt.xlabel("Predicted House Value")
plt.ylabel("Residuals")

plt.show()
```



In [87]: `'''Most residuals are randomly distributed around zero, indicating that the model captures the general trend of the data. \nHowever, some larger residuals suggest that a single predictor cannot explain all variations in house prices'`

Out[87]: 'Most residuals are randomly distributed around zero, indicating that the model captures the general trend of the data. \nHowever, some larger residuals suggest that a single predictor cannot explain all variations in house prices'

In [88]: `#evaluation metrics
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
print("R2 Score:", r2)
print("MAE:", mae)
print("RMSE:", rmse)`

R² Score: 0.45885918903846656

MAE: 62990.86530093761

RMSE: 84209.01241414454

In [89]: `#PART 6(Multiple Linear Regression)`

In [90]: `features = [
 "median_income",
 "housing_median_age",
 "total_rooms",
 "latitude",
 "longitude"
]

X = df[features]
y = df["median_house_value"]`

In [91]: `'''Feature Selection`

The following features were selected because they have meaningful relationships with the target variable (median_house_value).

- median_income - Highest positive correlation with house value.
- housing_median_age - Older neighborhoods may have higher property values.
- total_rooms - Represents the size of houses.
- latitude - Captures geographical location.
- longitude - Captures geographical location.

These features combine economic, structural, and geographical information, which are important factors affecting house prices.'

Out[91]: 'Feature Selection\n\nThe following features were selected because they have meaningful relationships\nwith the target variable (median_house_value).\n\n• median_income - Highest positive correlation with house value.\n• housing_median_age - Older neighborhoods may have higher property values.\n• total_rooms - Represents the size of houses.\n• latitude - Captures geographical location.\n• longitude - Captures geographical location.\n\nThese features combine economic, structural, and geographical information,\nwhich are important factors affecting house prices.'

In [92]: *## Conclusion*

'''The Simple Linear Regression model successfully predicts median house values\n\nMedian income was selected because it had the highest positive correlation with the target variable.\n\nThe model demonstrates a clear positive relationship between income and house prices.\n\nA Multiple Linear Regression model using additional features such as longitude,

Out[92]: 'The Simple Linear Regression model successfully predicts median house values using median income.\n\nMedian income was selected because it had the highest positive correlation with the target variable.\n\nThe model demonstrates a clear positive relationship between income and house prices. However, because only one feature is used, the model cannot capture all factors influencing house values. Therefore, its prediction accuracy is moderate.\n\nA Multiple Linear Regression model using additional features such as longitude, latitude, housing median age, and total rooms is expected to achieve better performance'

In [93]: **from** sklearn.model_selection **import** train_test_split

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

In [94]: **from** sklearn.linear_model **import** LinearRegression

```
model = LinearRegression()

model.fit(X_train, y_train)
```



```

Out[94]: <style>.sk-global { /* Definition of color scheme common for light and dark mode */ --
sklearn-color-text: #000; --sklearn-color-text-muted: #666; --sklearn-color-line: gray; /*
Definition of color scheme for unfitted estimators */ --sklearn-color-unfitted-level-0:
#fff5e6; --sklearn-color-unfitted-level-1: #f6e4d2; --sklearn-color-unfitted-level-2:
#ffe0b3; --sklearn-color-unfitted-level-3: chocolate; /* Definition of color scheme for
fitted estimators */ --sklearn-color-fitted-level-0: #f0f8ff; --sklearn-color-fitted-level-1:
#d4ebff; --sklearn-color-fitted-level-2: #b3dbfd; --sklearn-color-fitted-level-3:
cornflowerblue; } .sk-global.light { /* Specific color for light theme */ --sklearn-color-text-
on-default-background: black; --sklearn-color-background: white; --sklearn-color-border-
box: black; --sklearn-color-icon: #696969; } .sk-global.dark { --sklearn-color-text-on-
default-background: white; --sklearn-color-background: #111; --sklearn-color-border-
box: white; --sklearn-color-icon: #878787; } .sk-global { color: var(--sklearn-color-text); }
.sk-global pre { padding: 0; } .sk-global input.sk-hidden--visually { border: 0; clip-path:
inset(100%); height: 1px; margin: -1px; overflow: hidden; padding: 0; position: absolute;
width: 1px; } .sk-global div.sk-dashed-wrapped { border: 1px dashed var(--sklearn-color-
line); margin: 0 0.4em 0.5em 0.4em; box-sizing: border-box; padding-bottom: 0.4em;
background-color: var(--sklearn-color-background); } .sk-global div.sk-container { /*
jupyter's `normalize.less` sets `[hidden] { display: none; }` but bootstrap.min.css set
`[hidden] { display: none !important; }` so we also need the `!important` here to be able to
override the default hidden behavior on the sphinx rendered scikit-learn.org. See:
https://github.com/scikit-learn/scikit-learn/issues/21755 */ display: inline-block
!important; position: relative; } .sk-global div.sk-text-repr-fallback { display: none; } div.sk-
parallel-item, div.sk-serial, div.sk-item { /* draw centered vertical line to link estimators */
background-image: linear-gradient(var(--sklearn-color-text-on-default-background), var(-
sklearn-color-text-on-default-background)); background-size: 2px 100%; background-
repeat: no-repeat; background-position: center center; } /* Parallel-specific style estimator
block */ .sk-global div.sk-parallel-item::after { content: ""; width: 100%; border-bottom:
2px solid var(--sklearn-color-text-on-default-background); flex-grow: 1; } .sk-global div.sk-
parallel { display: flex; align-items: stretch; justify-content: center; background-color: var(--
sklearn-color-background); position: relative; } .sk-global div.sk-parallel-item { display: flex;
flex-direction: column; } .sk-global div.sk-parallel-item:first-child::after { align-self: flex-
end; width: 50%; } .sk-global div.sk-parallel-item:last-child::after { align-self: flex-start;
width: 50%; } .sk-global div.sk-parallel-item:only-child::after { width: 0; } /* Serial-specific
style estimator block */ .sk-global div.sk-serial { display: flex; flex-direction: column; align-
items: center; background-color: var(--sklearn-color-background); padding-right: 1em;
padding-left: 1em; } /* Toggleable style: style used for
estimator/Pipeline/ColumnTransformer box that is clickable and can be
expanded/collapsed. - Pipeline and ColumnTransformer use this feature and define the
default style - Estimators will overwrite some part of the style using the `sk-estimator`
class */ /* Pipeline and ColumnTransformer style (default) */ .sk-global div.sk-toggleable {
/* Default theme specific background. It is overwritten whether we have a specific
estimator or a Pipeline/ColumnTransformer */ background-color: var(--sklearn-color-
background); } /* Toggleable label */ .sk-global label.sk-toggleable__label { cursor: pointer;
display: flex; width: 100%; margin-bottom: 0; padding: 0.5em; box-sizing: border-box;
text-align: center; align-items: center; justify-content: center; gap: 0.5em; } .sk-global

```

```

label.sk-toggleable__label .caption { font-size: 0.6rem; font-weight: lighter; color: var(--sklearn-color-text-muted); } .sk-global label.sk-toggleable__label-arrow:before { /* Arrow on the left of the label */ content: " ▶ "; float: left; margin-right: 0.25em; color: var(--sklearn-color-icon); } .sk-global label.sk-toggleable__label-arrow:hover:before { color: var(--sklearn-color-text); } /* Toggleable content - dropdown */ .sk-global div.sk-toggleable__content { display: none; text-align: left; /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-toggleable__content.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); } .sk-global div.sk-toggleable__content pre { margin: 0.2em; border-radius: 0.25em; color: var(--sklearn-color-text); /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-toggleable__content.fitted pre { /* unfitted */ background-color: var(--sklearn-color-fitted-level-0); } .sk-global input.sk-toggleable__control:checked~div.sk-toggleable__content { /* Expand drop-down */ display: block; width: 100%; overflow: visible; } .sk-global input.sk-toggleable__control:checked~label.sk-toggleable__label-arrow:before { content: " ▼ "; } /* Pipeline/ColumnTransformer-specific style */ .sk-global div.sk-label input.sk-toggleable__control:checked~label.sk-toggleable__label { color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-label.fitted input.sk-toggleable__control:checked~label.sk-toggleable__label { background-color: var(--sklearn-color-fitted-level-2); } /* Estimator-specific style */ /* Colorize estimator box */ .sk-global div.sk-estimator input.sk-toggleable__control:checked~label.sk-toggleable__label { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-estimator.fitted input.sk-toggleable__control:checked~label.sk-toggleable__label { /* fitted */ background-color: var(--sklearn-color-fitted-level-2); } .sk-global div.sk-label label.sk-toggleable__label, .sk-global div.sk-label label { /* The background is the default theme color */ color: var(--sklearn-color-text-on-default-background); } /* On hover, darken the color of the background */ .sk-global div.sk-label:hover label.sk-toggleable__label { color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); } /* Label box, darken color on hover, fitted */ .sk-global div.sk-label.fitted:hover label.sk-toggleable__label.fitted { color: var(--sklearn-color-text); background-color: var(--sklearn-color-fitted-level-2); } /* Estimator label */ .sk-global div.sk-label label { font-family: monospace; font-weight: bold; line-height: 1.2em; } .sk-global div.sk-label-container { text-align: center; } /* Estimator-specific */ .sk-global div.sk-estimator { font-family: monospace; border: 1px dotted var(--sklearn-color-border-box); border-radius: 0.25em; box-sizing: border-box; margin-bottom: 0.5em; /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-estimator.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); } /* on hover */ .sk-global div.sk-estimator:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-estimator.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-2); } /* Specification for estimator info (e.g. "i" and "?") */ /* Common style for "i" and "?" */ .sk-estimator-doc-link, a.link.sk-estimator-doc-link, a.visited.sk-estimator-doc-link { float: right; font-size: smaller; line-height: 1em; font-family: monospace; background-color: var(--sklearn-color-unfitted-level-0); border-radius: 1em; height: 1em; width: 1em; text-decoration: none !important; margin-left: 0.5em; text-align: center; /* unfitted */ border: var(--sklearn-color-unfitted-level-3) 1pt solid; color: var(--sklearn-

```

```

color-unfitted-level-3); } .sk-estimator-doc-link.fitted, a:link.sk-estimator-doc-link.fitted,
a:visited.sk-estimator-doc-link.fitted { /* fitted */ background-color: var(--sklearn-color-
fitted-level-0); border: var(--sklearn-color-fitted-level-3) 1pt solid; color: var(--sklearn-
color-fitted-level-3); } /* On hover */ div.sk-estimator: hover .sk-estimator-doc-link: hover,
.sk-estimator-doc-link: hover, div.sk-label-container: hover .sk-estimator-doc-link: hover,
.sk-estimator-doc-link: hover { /* unfitted */ background-color: var(--sklearn-color-
unfitted-level-3); border: var(--sklearn-color-fitted-level-0) 1pt solid; color: var(--sklearn-
color-unfitted-level-0); text-decoration: none; } div.sk-estimator.fitted: hover .sk-estimator-
doc-link.fitted: hover, .sk-estimator-doc-link.fitted: hover, div.sk-label-container: hover .sk-
estimator-doc-link.fitted: hover, .sk-estimator-doc-link.fitted: hover { /* fitted */
background-color: var(--sklearn-color-fitted-level-3); border: var(--sklearn-color-fitted-
level-0) 1pt solid; color: var(--sklearn-color-fitted-level-0); text-decoration: none; } /* Span,
style for the box shown on hovering the info icon */ .sk-estimator-doc-link span { display:
none; z-index: 9999; position: relative; font-weight: normal; right: .2ex; padding: .5ex;
margin: .5ex; width: min-content; min-width: 20ex; max-width: 50ex; color: var(--sklearn-
color-text); box-shadow: 2pt 2pt 4pt #999; /* unfitted */ background: var(--sklearn-color-
unfitted-level-0); border: .5pt solid var(--sklearn-color-unfitted-level-3); } .sk-estimator-
doc-link.fitted span { /* fitted */ background: var(--sklearn-color-fitted-level-0); border:
var(--sklearn-color-fitted-level-3); } .sk-estimator-doc-link: hover span { display: block; } /*
"-specific style due to the `` HTML tag */ .sk-global a.estimator_doc_link { float: right;
font-size: 1rem; line-height: 1em; font-family: monospace; background-color: var(--
sklearn-color-unfitted-level-0); border-radius: 1rem; height: 1rem; width: 1rem; text-
decoration: none; /* unfitted */ color: var(--sklearn-color-unfitted-level-1); border: var(--
sklearn-color-unfitted-level-1) 1pt solid; } .sk-global a.estimator_doc_link.fitted { /* fitted
*/ background-color: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-
level-1) 1pt solid; color: var(--sklearn-color-fitted-level-1); } /* On hover */ .sk-global
a.estimator_doc_link: hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-
level-3); color: var(--sklearn-color-background); text-decoration: none; } .sk-global
a.estimator_doc_link.fitted: hover { /* fitted */ background-color: var(--sklearn-color-fitted-
level-3); } .sk-top-container.sk-global { /* pydata-sphinx-theme hides overflow, so
scrolling is disabled. We need to set it to !important and add tabindex="0" in the HTML to
allow keyboard-only users to navigate the display. */ overflow-x: scroll !important; max-
width: 100%; } .estimator-table { font-family: monospace; } .estimator-table summary {
padding: .5rem; cursor: pointer; } .estimator-table summary::marker { font-size: 0.7rem; }
.estimator-table details[open] { padding-left: 0.1rem; padding-right: 0.1rem; padding-
bottom: 0.3rem; } .estimator-table .parameters-table { margin-left: auto !important;
margin-right: auto !important; margin-top: 0; } .estimator-table .parameters-table tr:nth-
child(odd) { background-color: #fff; } .estimator-table .parameters-table tr:nth-child(even)
{ background-color: #f6f6f6; } .estimator-table .parameters-table tr: hover td { background-
color: #e0e0e0; } .estimator-table table :is(td, th) { border: 1px solid rgba(106, 105, 104,
0.232); } /* `table td` is set in notebook with right text-align. We need to overwrite it. */
.estimator-table table td.param { text-align: left; position: relative; padding: 0; } .user-set td
{ color: rgb(255, 94, 0); text-align: left !important; } .user-set td.value { color: rgb(255, 94, 0);
background-color: transparent; } .default td, .estimator-table th { color: black; text-align:
left !important; } .user-set td i, .default td i { color: black; } td.fitted-att-type { white-space:

```

```

preserve nowrap; } /* Styles for parameter documentation links We need styling for visited
so jupyter doesn't overwrite it */ a.param-doc-link, a.param-doc-link:link, a.param-doc-
link:visited { text-decoration: underline dashed; text-underline-offset: .3em; color: inherit;
display: block; padding: .5em; } @supports(anchor-name: --doc-link) { a.param-doc-link,
a.param-doc-link:link, a.param-doc-link:visited { anchor-name: --doc-link; } } /* "hack" to
make the entire area of the cell containing the link clickable */ a.param-doc-link::before {
position: absolute; content: ""; inset: 0; } .param-doc-description { display: none; position:
absolute; z-index: 9999; left: 0; padding: .5ex; margin-left: 1.5em; color: var(--sklearn-
color-text); box-shadow: .3em .3em .4em #999; width: max-content; text-align: left; max-
height: 10em; overflow-y: auto; /* unfitted */ background: var(--sklearn-color-unfitted-
level-0); border: thin solid var(--sklearn-color-unfitted-level-3); } @supports(position-area:
center right) { .param-doc-description { position-area: center right; position: fixed; margin-
left: 0; } } /* Fitted state for parameter tooltips */ .fitted .param-doc-description { /* fitted
*/ background: var(--sklearn-color-fitted-level-0); border: thin solid var(--sklearn-color-
fitted-level-3); } .param-doc-link:hover .param-doc-description { display: block; } .copy-
paste-icon { background-image:
url(data:image/svg+xml;base64,PHN2ZyB4bWxucz0iaHR0cDovL3d3dy53My5vcmcvMjAwMC9:
background-repeat: no-repeat; background-size: 14px 14px; background-position: 0;
display: inline-block; width: 14px; height: 14px; cursor: pointer; } .features { font-family:
monospace; cursor: pointer; background-color: var(--sklearn-color-unfitted-level-0);
border: 1px dotted var(--sklearn-color-border-box); border-radius: .20em; margin-bottom:
0.5em; font-size: inherit; /* Needed for jupyter */ } .features.fitted { background-color:
var(--sklearn-color-fitted-level-0); } .features.summary { cursor: pointer; display: flex;
margin-bottom: 0; text-align: center; align-items: center; justify-content: center; gap:
0.5em; padding: .25em; } .features.details[open] > summary { color: var(--sklearn-color-
text); background-color: var(--sklearn-color-unfitted-level-2); border-radius: .20em 0 0 0; }
.features.fitted.details[open] > summary { background-color: var(--sklearn-color-fitted-
level-2); border-radius: .20em 0 0 0; } .features.details > summary .arrow::before { content:
" ▶ "; color: grey; } .features.details[open] > summary .arrow::before { content: " ▼ "; }
.features.details:hover > summary { margin: 0; background-color: var(--sklearn-color-
unfitted-level-2); } .features.fitted.details:hover > summary { margin: 0; background-color:
var(--sklearn-color-fitted-level-2); } .features .features-container { max-width: 15em; max-
height: 10em; overflow: auto; scrollbar-width: thin; padding: .25em 0.1em; background-
color: var(--sklearn-color-unfitted-level-0); border-radius: 0 0 .5em .5em; } .features.fitted
.features-container { background-color: var(--sklearn-color-fitted-level-0); } .features
.image-container { block-size: 1em; inline-size: 1em; padding: 0; margin: 0%; display: flex;
justify-content: center; align-items: center; } .features .copy-paste-icon { background-size:
1em 1em; width: 1em; height: 1em; filter: grayscale(100%) opacity(60%); } .features
.features-container table { width: 100%; margin: 0.01em; } .features .features-container
table tr:nth-child(odd) { background-color: #fff; } .features .features-container table tr:nth-
child(even) { background-color: #f6f6f6; } .features .features-container table tr:hover {
background-color: #e0e0e0; } .features .features-container table { table-layout: inherit; }
.features .features-container table td { text-align: left; padding: 0 0.5em; border: 1px solid
rgba(106, 105, 104, 0.232); white-space: nowrap; color: var(--sklearn-color-text); }
.total_features { display: flex; justify-content: center; margin-top: 0.5em; } </style> <body>

```

LinearRegression()

**In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.**

☒ sk-estimator-id-2 sk-toggleable__label fitted sk-toggleable__label-arrow">

LinearRegression

[?Documentation for LinearRegression](#)

</label>

<details> <summary>Parameters</summary> <tbody> </tbody> <tbody> </tbody>

fit_intercept fit_intercept: bool, default=True

Whether to calculate the intercept for this model. If set to False, no intercept will be used in calculations (i.e. data is expected to be centered).

True

copy_X copy_X: bool, default=True

If True, X will be copied; else, it may be overwritten.

True

tol tol: float, default=1e-6

The precision of the solution ('coef_') is determined by 'tol' which specifies the convergence criterion of the underlying solver. 'tol' is set as 'atol' and 'btol' of :func:`scipy.sparse.linalg.lsqr` when fitting on sparse training data. 'tol' is set as 'cond' of :func:`scipy.linalg.lstsq` when fitting on dense training data.

.. versionadded:: 1.7
.. versionchanged:: 1.9
Now supported on dense data, interpreted as the 'cond' parameter.

1e-06

n_jobs n_jobs: int, default=None

The number of jobs to use for the computation. This will only provide speedup in case of sufficiently large problems, that is if firstly 'n_targets > 1' and secondly 'X' is sparse or if 'positive' is set to 'True'. 'None' means 1 unless in a :obj:`joblib.parallel\_backend` context. '-1' means using all processors. See :term:`Glossary` <n_jobs> for more details.

None

positive positive: bool, default=False

When set to "True", forces the coefficients to be positive. This option is only supported for dense arrays.

For a comparison between a linear regression model with positive constraints on the regression coefficients and a linear regression without such constraints, see :ref:`sphx\_glr\_auto\_examples\_linear\_model\_plot\_nnls.py`.
.. versionadded:: 0.24

False

</details>

<details> <summary>Fitted attributes</summary> <tbody> <th>Name</th>

<th>Type</th> <th>Value</th> </tbody> <tbody> </tbody>

coef_ coef_: array ndarray[float64] [37535.98, 1167.87, 3.38, -43887
of shape (5,) (n_features,) or (n_targets, n_features)

Estimated coefficients for the linear regression problem.
If multiple targets are passed during the fit (y 2D), this is a 2D

array of shape
(n_targets,
n_features), while
if only one
target is passed,
this is a 1D array
of length
n_features.

feature_names_in_
feature_names_in_:
ndarray of shape
(n_features_in_,)
Names
of features seen
during :term:`fit`.
Defined only when
'X' has feature
names that are all
strings.
versionadded:: 1.0

ndarray[object]
(5,) ['median_income', 'housing_median_age', 'total_rooms', 'latitude', 'population', 'median_house_value']

intercept_
intercept_: float or
array of shape
(n_targets,) Independent
term in the linear
model. Set to 0.0
if :term:`fit\_intercept`
= False.

float64

n_features_in_
n_features_in_:
int
Number of
features seen
during :term:`fit`.
versionadded::
0.24

int

rank_rank_
int Rank
of matrix 'X'. Only
available when 'X'
is dense.

int64

singular_singular_
array of shape
(min(X, y),) Singular
values of 'X'. Only
available when 'X'
is dense.

ndarray[float64]
(5,)

[279432.15, 1510.47, 368.94,

</details>

<script> /* Authors: The scikit-learn developers
SPDX-License-Identifier: BSD-3-Clause */
function copyToClipboard(text, element) { // Get the parameter prefix from the closest
toggleable content const toggleableContent = element.closest('.sk-toggleable__content');

```

const paramPrefix = toggleableContent ? toggleableContent.dataset.paramPrefix : ""; const
fullParamName = paramPrefix ? `paramPrefix{text}` : text; const originalStyle =
element.style; const computedStyle = window.getComputedStyle(element); const
originalWidth = computedStyle.width; const originalHTML =
element.innerHTML.replace('Copied!', ''); navigator.clipboard.writeText(fullParamName)
.then(() => { element.style.width = originalWidth; element.style.color = 'green';
element.innerHTML = "Copied!"; setTimeout(() => { element.innerHTML = originalHTML;
element.style = originalStyle; }, 2000); }) .catch(err => { console.error('Failed to copy:', err);
element.style.color = 'red'; element.innerHTML = "Failed!"; setTimeout(() => {
element.innerHTML = originalHTML; element.style = originalStyle; }, 2000); }); return false;
} document.querySelectorAll('.copy-paste-icon').forEach(function(element) { const
toggleableContent = element.closest('.sk-toggleable__content'); const paramPrefix =
toggleableContent ? toggleableContent.dataset.paramPrefix : ""; const parent =
element.parentElement; if (!parent || !parent.nextElementSibling) { console.warn('Expected
copy-paste icon is missing from the DOM structure'); return; } const paramName =
element.parentElement.nextElementSibling.textContent.trim().split(' ')[0]; const
fullParamName = paramPrefix ? `paramPrefix{paramName}` : paramName;
element.setAttribute('title', fullParamName); }); /** * Copy the list of feature names
formatted as a Python list. * * @param {HTMLElement} element - The copy button inside a
`.features` block; its siblings * contain a `details` element and a table containing feature
named. * @returns {boolean} Always returns `false` so callers can prevent the default click
behavior. */ function copyFeatureNamesToClipboard(element) { var detailsElem =
element.closest('.features').querySelector('details'); var wasOpen = detailsElem.open;
detailsElem.open = true; var content = element.closest('.features').querySelector('tbody')
.innerHTML.trim(); if (!wasOpen) detailsElem.open = false; const rows =
content.split('\n').map(row => `row`); const formattedText = `[
{rows.join(',\n')},\n]`; const originalHTML = element.innerHTML.replace('âœ“', ''); const
originalStyle = element.style; const copyMark = document.createElement('span');
copyMark.innerHTML = 'âœ“'; copyMark.style.color = 'blue'; copyMark.style.fontSize =
'1em'; navigator.clipboard.writeText(formattedText) .then(() => { element.style.display =
'none'; element.parentElement.appendChild(copyMark); setTimeout(() => {
copyMark.remove(); element.innerHTML = originalHTML; element.style = originalStyle; },
1000); }) .catch(err => { console.error('Failed to copy:', err); element.style.color = 'orange';
element.innerHTML = "Failed!"; setTimeout(() => { element.innerHTML = originalHTML;
element.style = originalStyle; }, 1000); }); return false; } /** * Adapted from Skrub *
https://github.com/skrub-
data/skrub/blob/403466d1d5d4dc76a7ef569b3f8228db59a31dc3/skrub/_reporting/_data/terr
* @returns "light" or "dark" */ function detectTheme(element) { const body =
document.querySelector('body'); // Check VSCode theme const themeKindAttr =
body.getAttribute('data-vscode-theme-kind'); const themeNameAttr =
body.getAttribute('data-vscode-theme-name'); if (themeKindAttr && themeNameAttr) {
const themeKind = themeKindAttr.toLowerCase(); const themeName =
themeNameAttr.toLowerCase(); if (themeKind.includes("dark") ||
themeName.includes("dark")) { return "dark"; } if (themeKind.includes("light") ||
themeName.includes("light")) { return "light"; } } // Check Jupyter theme if

```

```
(body.getAttribute('data-jp-theme-light') === 'false') { return 'dark'; } else if
(body.getAttribute('data-jp-theme-light') === 'true') { return 'light'; } // Guess based on a
parent element's color const color = window.getComputedStyle(element.parentNode,
null).getPropertyValue('color'); const match = color.match(/^rgb\s*
\s* (\d+)\s*, \s* (\d+)\s*, \s* (\d+)\s*$/i); if (match) { const [r, g, b] = [
parseFloat(match[1]), parseFloat(match[2]), parseFloat(match[3]) ]; //
https://en.wikipedia.org/wiki/HSL_and_HSV#Lightness const luma = 0.299 * r + 0.587 * g
+ 0.114 * b; if (luma > 180) { // If the text is very bright we have a dark theme return
'dark'; } if (luma < 75) { // If the text is very dark we have a light theme return 'light'; } //
Otherwise fall back to the next heuristic. } // Fallback to system preference return
window.matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' : 'light'; } function
forceTheme(elementId) { const estimatorElement =
document.querySelector(`#${elementId}`); if (estimatorElement === null) {
console.error(`Element with id ${elementId} not found.`); } else { const theme =
detectTheme(estimatorElement); estimatorElement.classList.add(theme); } }
forceTheme('sk-container-id-2');
```

```
In [95]: print(model.coef_.shape)
```

```
(5,)
```

```
In [96]: coefficients = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": model.coef_
})

print(coefficients)
```

| | Feature | Coefficient |
|---|--------------------|---------------|
| 0 | median_income | 37535.980826 |
| 1 | housing_median_age | 1167.869663 |
| 2 | total_rooms | 3.383466 |
| 3 | latitude | -43887.440531 |
| 4 | longitude | -44682.959059 |

```
In [97]: print("X shape:", X.shape)
print("Number of features:", len(X.columns))
print("Feature names:", X.columns.tolist())

print("Coefficient shape:", model.coef_.shape)
print("Coefficients:", model.coef_)

print("Intercept:", model.intercept_)
```

```
X shape: (20640, 5)
Number of features: 5
Feature names: ['median_income', 'housing_median_age', 'total_rooms', 'latitude',
'longitude']
Coefficient shape: (5,)
Coefficients: [ 3.75359808e+04  1.16786966e+03  3.38346589e+00 -4.38874405e+04
-4.46829591e+04]
Intercept: -3759827.101363706
```

```
In [98]: print(model.coef_)
```



```
[ 3.75359808e+04  1.16786966e+03  3.38346589e+00 -4.38874405e+04
 -4.46829591e+04]
```

```
In [99]: #Evaluation metrics
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
import numpy as np

y_pred = model.predict(X_test)

r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print("R² Score:", r2)
print("MAE:", mae)
print("RMSE:", rmse)
```

R² Score: 0.584448731061169

MAE: 54230.7444767925

RMSE: 73793.09681378353

```
In [100... from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
import numpy as np

# Simple Linear Regression
X_simple = df[["median_income"]]
y = df["median_house_value"]

X_train_simple, X_test_simple, y_train_simple, y_test_simple = train_test_split(
    X_simple, y, test_size=0.2, random_state=42
)

simple_model = LinearRegression()
simple_model.fit(X_train_simple, y_train_simple)

y_pred_simple = simple_model.predict(X_test_simple)

# Save metrics
simple_r2 = r2_score(y_test_simple, y_pred_simple)
simple_mae = mean_absolute_error(y_test_simple, y_pred_simple)
simple_rmse = np.sqrt(mean_squared_error(y_test_simple, y_pred_simple))

print("Simple R²:", simple_r2)
print("Simple MAE:", simple_mae)
print("Simple RMSE:", simple_rmse)
```

Simple R²: 0.45885918903846656

Simple MAE: 62990.86530093761

Simple RMSE: 84209.01241414454

```
In [101... multiple_r2 = r2
multiple_mae = mae
multiple_rmse = rmse
```

```
In [102... y_pred = model.predict(X_test)

multiple_r2 = r2_score(y_test, y_pred)
```

```
multiple_mae = mean_absolute_error(y_test, y_pred)
multiple_rmse = np.sqrt(mean_squared_error(y_test, y_pred))
```

```
In [103... comparison = pd.DataFrame({
    "Metric": ["R2 Score", "MAE", "RMSE"],
    "Simple LR": [simple_r2, simple_mae, simple_rmse],
    "Multiple LR": [multiple_r2, multiple_mae, multiple_rmse]
})

print(comparison)
```

| | Metric | Simple LR | Multiple LR |
|---|----------------------|--------------|--------------|
| 0 | R ² Score | 0.458859 | 0.584449 |
| 1 | MAE | 62990.865301 | 54230.744477 |
| 2 | RMSE | 84209.012414 | 73793.096814 |

```
In [104... comparison["Improvement"] = [
    f"{{(multiple_r2-simple_r2)*100:.2f}}%",
    f"{{((simple_mae-multiple_mae)/simple_mae)*100:.2f}}% lower",
    f"{{((simple_rmse-multiple_rmse)/simple_rmse)*100:.2f}}% lower"
]

comparison
```

```
Out[104... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>Metric</th> <th>Simple LR</th> <th>Multiple LR</th>
<th>Improvement</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th>
</tbody> <tbody></tbody>
```

	R ² Score	0.458859	0.584449	12.56%
	MAE	62990.865301	54230.744477	13.91% lower
	RMSE	84209.012414	73793.096814	12.37% lower

```
In [105... '''The Multiple Linear Regression model performs better than the Simple Linear R

- The R2 Score increased, indicating that the model explains a larger percentage
- Both MAE and RMSE decreased, meaning the prediction errors became smaller
- Adding more relevant features improved the model's predictive performance

Overall, Multiple Linear Regression provides more accurate house price predictio
```

```
Out[105... "The Multiple Linear Regression model performs better than the Simple Linear Re
gression model\n\n- The R2 Score increased, indicating that the model explains
a larger percentage of the variation in house prices\n- Both MAE and RMSE decre
ased, meaning the prediction errors became smaller\n- Adding more relevant feat
ures improved the model's predictive performance\n\nOverall, Multiple Linear Re
gression provides more accurate house price predictions than Simple Linear Regr
ession"
```

```
In [106... '''## Feature Importance

Based on the regression coefficients:

1. Median Income
2. Longitude
3. Latitude
```

These three features have the largest impact on predicting house prices

Median Income contributes positively, while Latitude and Longitude capture the g

```
Out[106... '## Feature Importance\n\nBased on the regression coefficients:\n\n1. Median In
come\n2. Longitude\n3. Latitude\n\nThese three features have the largest impact
on predicting house prices\n\nMedian Income contributes positively, while Latit
ude and Longitude capture the geographical influence on property values'
```

```
In [107... ## If Performance Did Not Improve

'''If the Multiple Linear Regression model does not perform better than the Simp

- The selected features may not be very useful for predicting house prices
- Some features may contain similar information, so adding them does not improve
- House prices may depend on complex factors that cannot be explained by a simpl
- The dataset may contain outliers or unusual values that affect the model's acc
- Using a more advanced model, such as Random Forest, may produce better predict
```

```
Out[107... "If the Multiple Linear Regression model does not perform better than the Simpl
e Linear Regression model, the possible reasons are:\n\n- The selected features
may not be very useful for predicting house prices\n- Some features may contain
similar information, so adding them does not improve the model much\n- House pr
ices may depend on complex factors that cannot be explained by a simple linear
relationship\n- The dataset may contain outliers or unusual values that affect
the model's accuracy\n- Using a more advanced model, such as Random Forest, may
produce better predictions because it can capture more complex patterns in the
data"
```

```
In [108... #PART-7(Visualization Portfolio )
```

```
In [109... #histogram
import matplotlib.pyplot as plt

features = ["median_house_value", "median_income", "housing_median_age"]

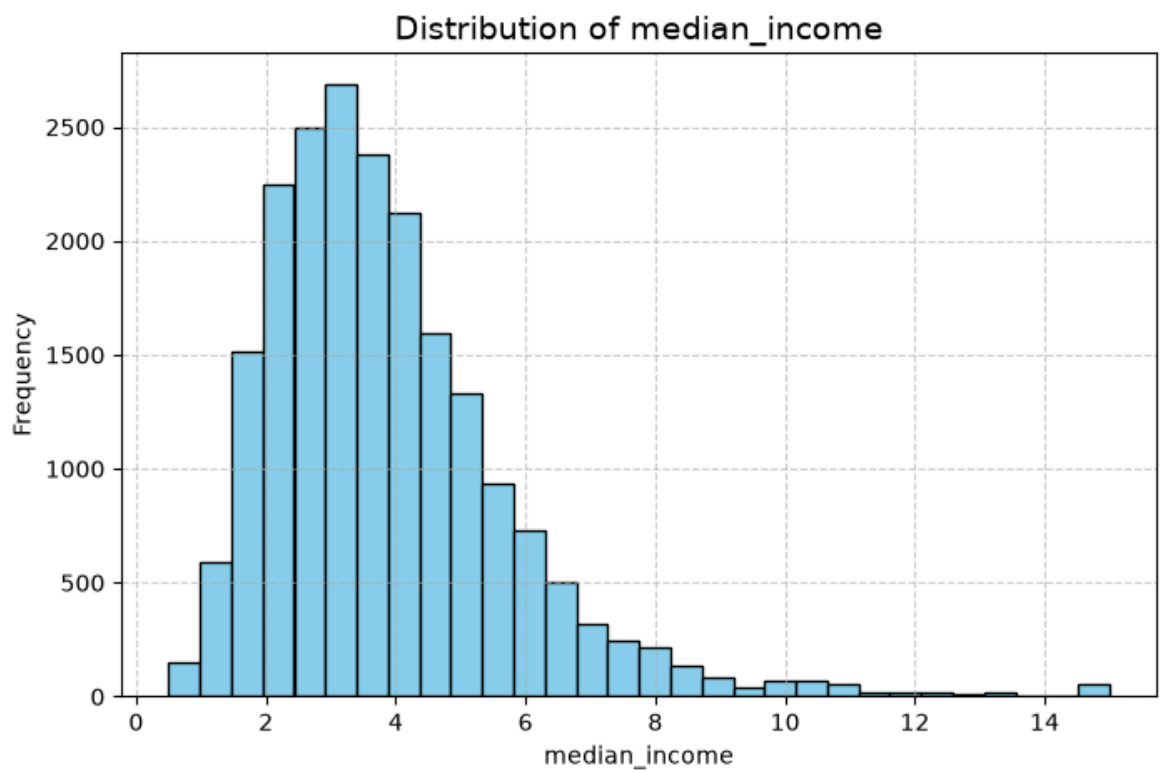
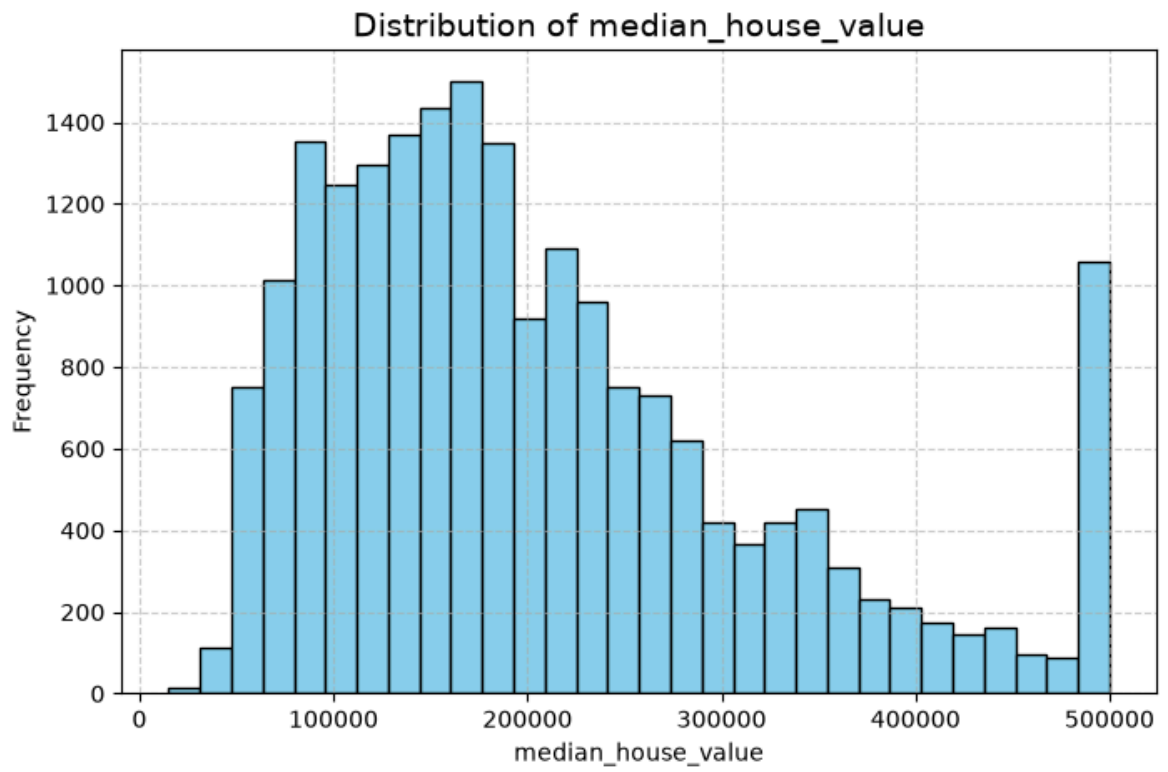
for feature in features:
    plt.figure(figsize=(8,5))

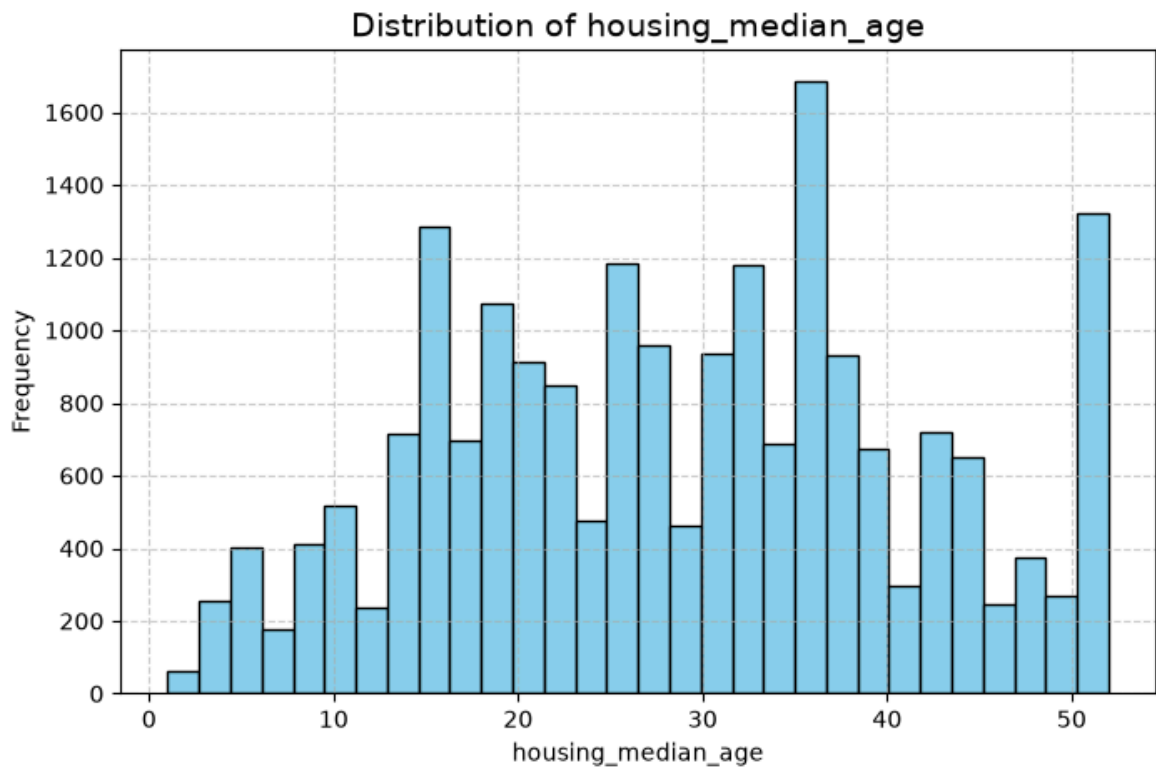
    plt.hist(df[feature],
             bins=30,
             color="skyblue",
             edgecolor="black")

    plt.title(f"Distribution of {feature}", fontsize=14)
    plt.xlabel(feature)
    plt.ylabel("Frequency")

    plt.grid(True, linestyle="--", alpha=0.6)

    plt.show()
```





In [110... `'''Median House Value: Most house values are concentrated in the middle range, with a cap at the upper limit.
Median Income: Slightly right-skewed, indicating fewer high-income areas.
Housing Median Age: Most houses are between 15-35 years old'''`

Out[110... `'Median House Value: Most house values are concentrated in the middle range, with a cap at the upper limit.
Median Income: Slightly right-skewed, indicating fewer high-income areas.
Housing Median Age: Most houses are between 15-35 years old'`

In []:

```
#boxplot
import seaborn as sns
import matplotlib.pyplot as plt

features = ["total_rooms", "population", "median_income"]

for feature in features:

    plt.figure(figsize=(8,5))

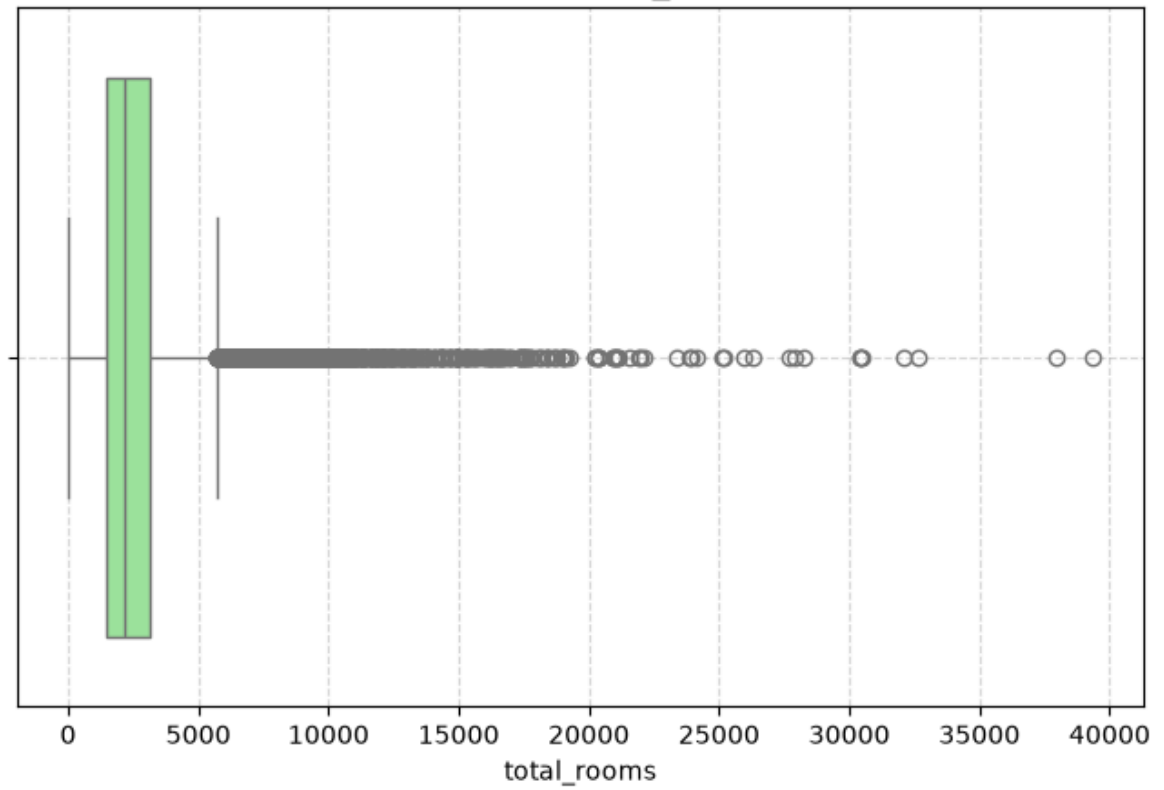
    sns.boxplot(
        x=df[feature],
        color="lightgreen"
    )

    plt.title(f"Box Plot of {feature}", fontsize=14)
    plt.xlabel(feature)

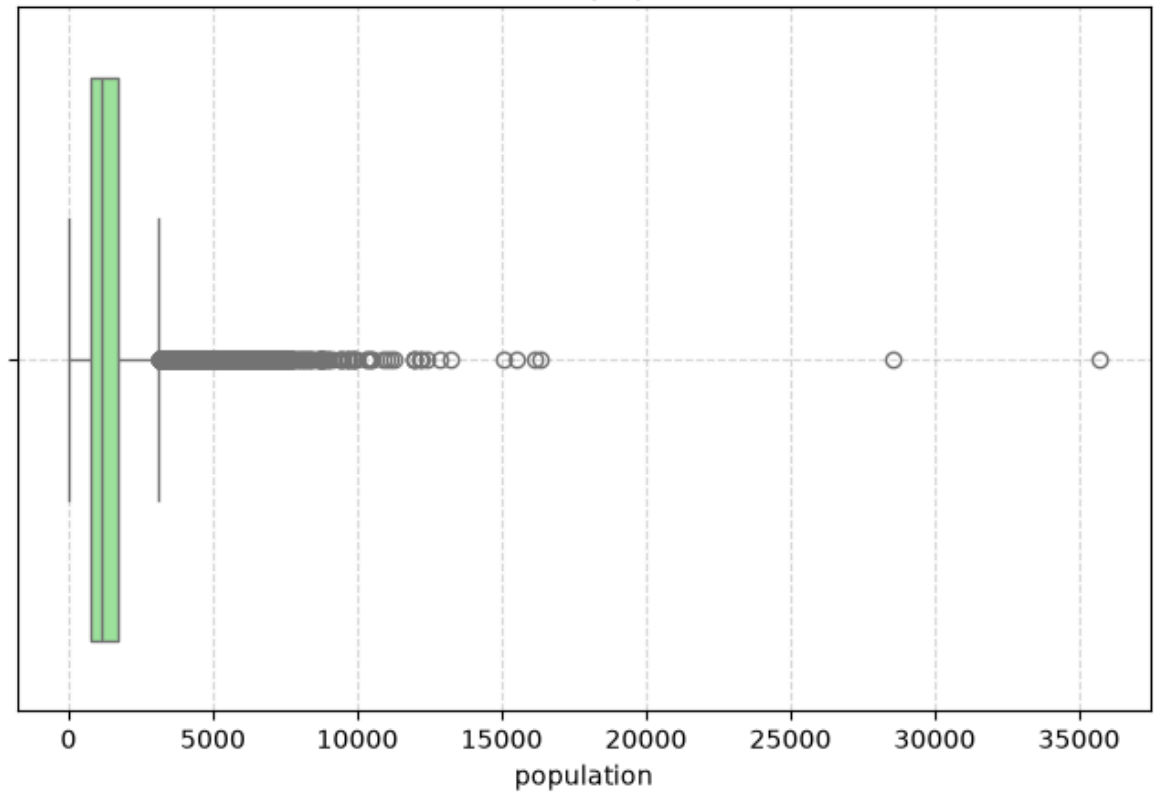
    plt.grid(True, linestyle="--", alpha=0.5)

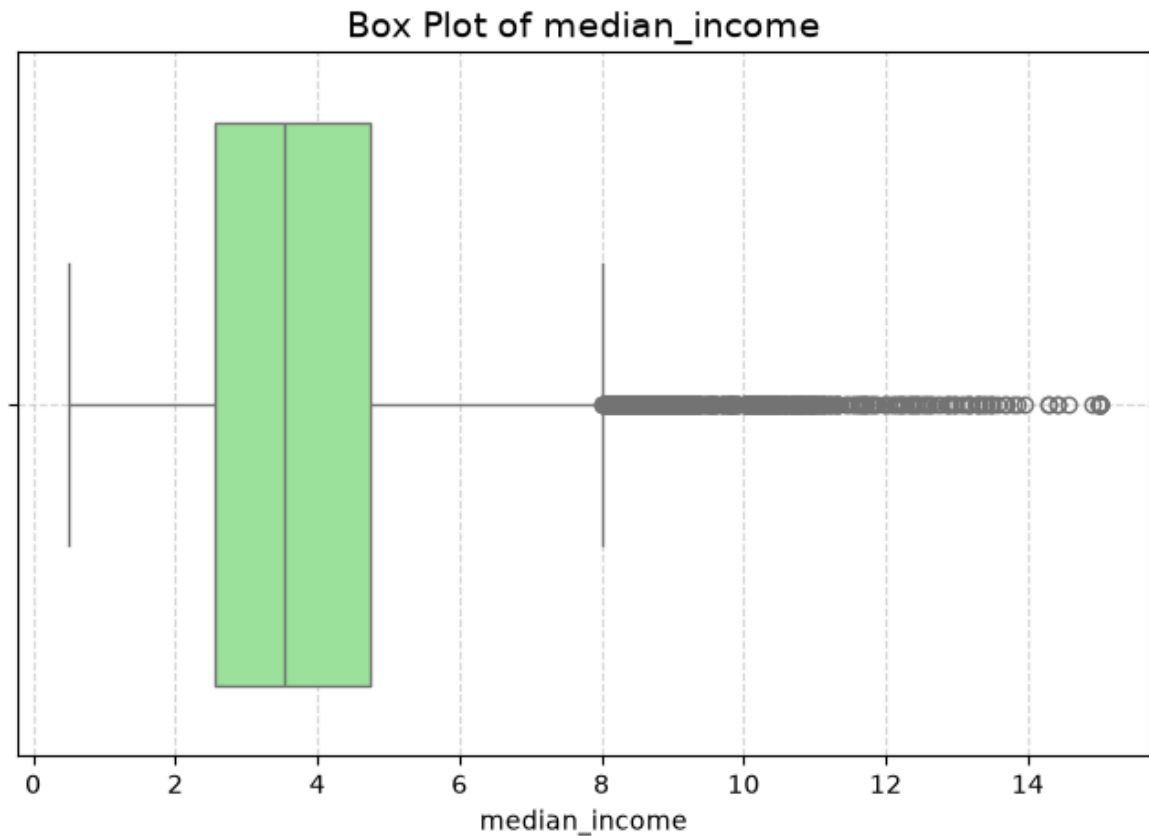
    plt.show()
```

Box Plot of total_rooms



Box Plot of population





```
In [112... '''total_rooms contains several high-value outliers.
population has many extreme observations.
median_income contains fewer outliers than the other variables.'''
```

```
Out[112... 'total_rooms contains several high-value outliers.\npopulation has many extreme
observations.\nmedian_income contains fewer outliers than the other variables.'
```

```
In [113... #Scatterplot
plt.figure(figsize=(8,6))

sns.regplot(
    x="median_income",
    y="median_house_value",
    data=df,
    scatter_kws={"color":"blue","alpha":0.4},
    line_kws={"color":"red"}
)

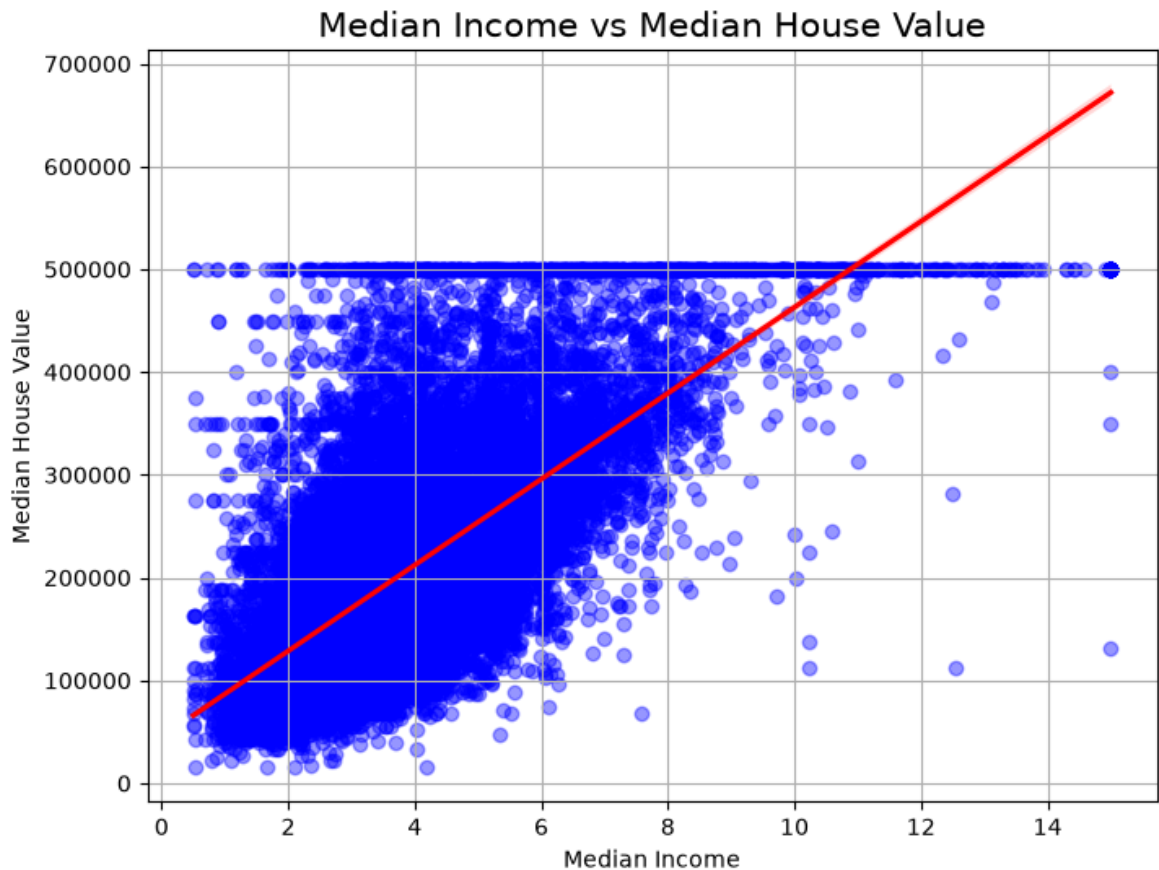
plt.title("Median Income vs Median House Value", fontsize=15)

plt.xlabel("Median Income")

plt.ylabel("Median House Value")

plt.grid(True)

plt.show()
```



In [114... `'''A positive relationship exists between income and house value.
As income increases, house prices generally increase.
The regression line confirms the positive trend.'''`

Out[114... `'A positive relationship exists between income and house value.\nAs income increases, house prices generally increase.\n\nThe regression line confirms the positive trend.'`

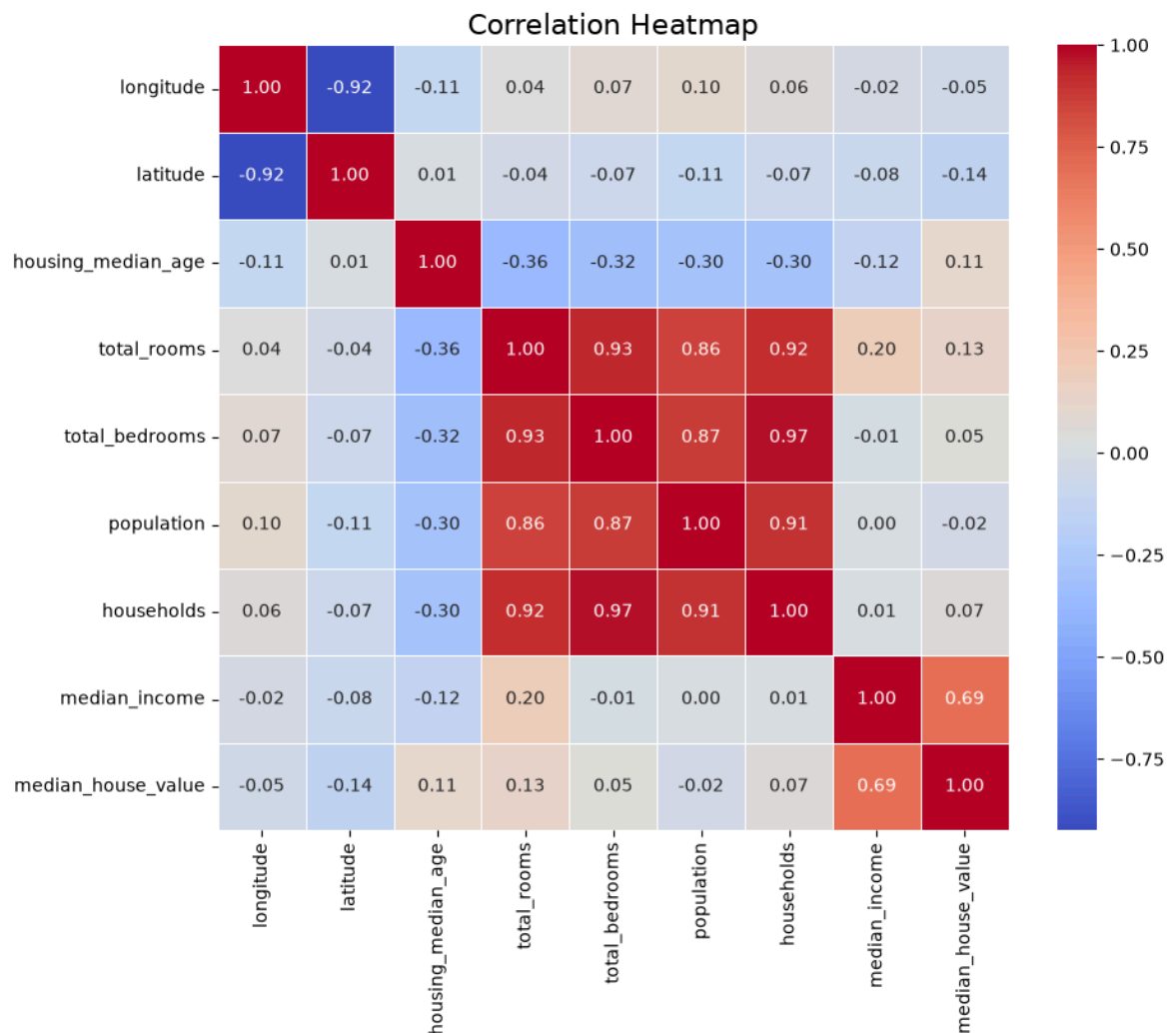
In [115... `#correlationmap
plt.figure(figsize=(10,8))

corr = df.select_dtypes(include="number").corr()

sns.heatmap(
 corr,
 annot=True,
 cmap="coolwarm",
 fmt=".2f",
 linewidths=0.5
)

plt.title("Correlation Heatmap", fontsize=16)

plt.show()`



In [116...

```
'''
Analysis
median_income has the strongest positive correlation with median_house_value.
total_rooms and total_bedrooms are highly correlated.
population and households are also strongly correlated.'''
```

Out[116...

```
'\nAnalysis\nmedian_income has the strongest positive correlation with median_h
ouse_value.\ntotal_rooms and total_bedrooms are highly correlated.\npopulation
and households are also strongly correlated.'
```

In [117...

```
#Regression line plot
plt.figure(figsize=(8,6))

plt.scatter(
    X_test_simple,
    y_test_simple,
    color="blue",
    alpha=0.5,
    label="Actual Data"
)

plt.plot(
    X_test_simple,
    y_pred_simple,
    color="red",
    linewidth=3,
    label="Regression Line"
)
```

```
plt.title("Simple Linear Regression", fontsize=15)

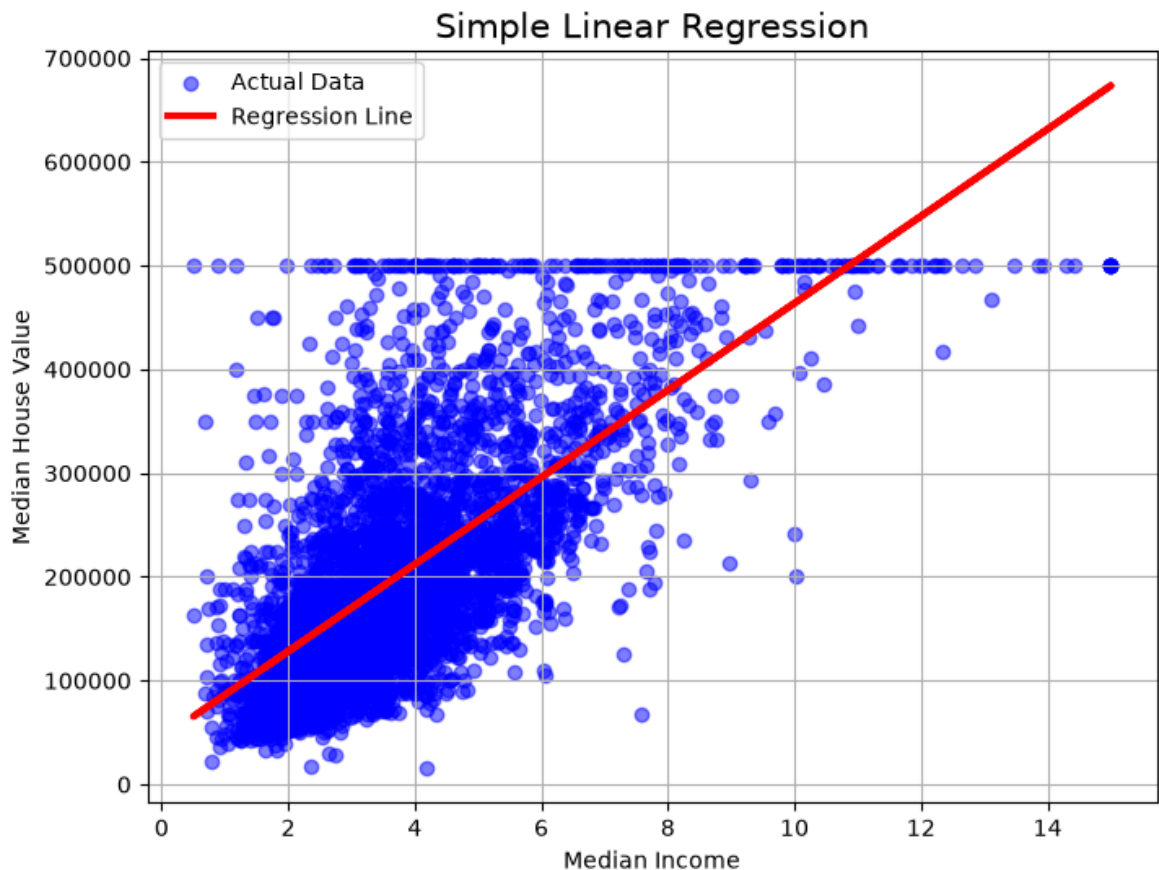
plt.xlabel("Median Income")

plt.ylabel("Median House Value")

plt.legend()

plt.grid(True)

plt.show()
```



```
In [118... '''The regression line fits the general upward trend.
Some observations lie far from the line, indicating prediction errors'''
```

```
Out[118... 'The regression line fits the general upward trend.\nSome observations lie far
from the line, indicating prediction errors'
```

```
In [119... #Residual Plot
residuals = y_test_simple - y_pred_simple

plt.figure(figsize=(8,6))

plt.scatter(
    y_pred_simple,
    residuals,
    color="purple",
    alpha=0.5
)

plt.axhline(
    y=0,
```

```

        color="red",
        linestyle="--",
        linewidth=2
    )

plt.title("Residual Plot", fontsize=15)

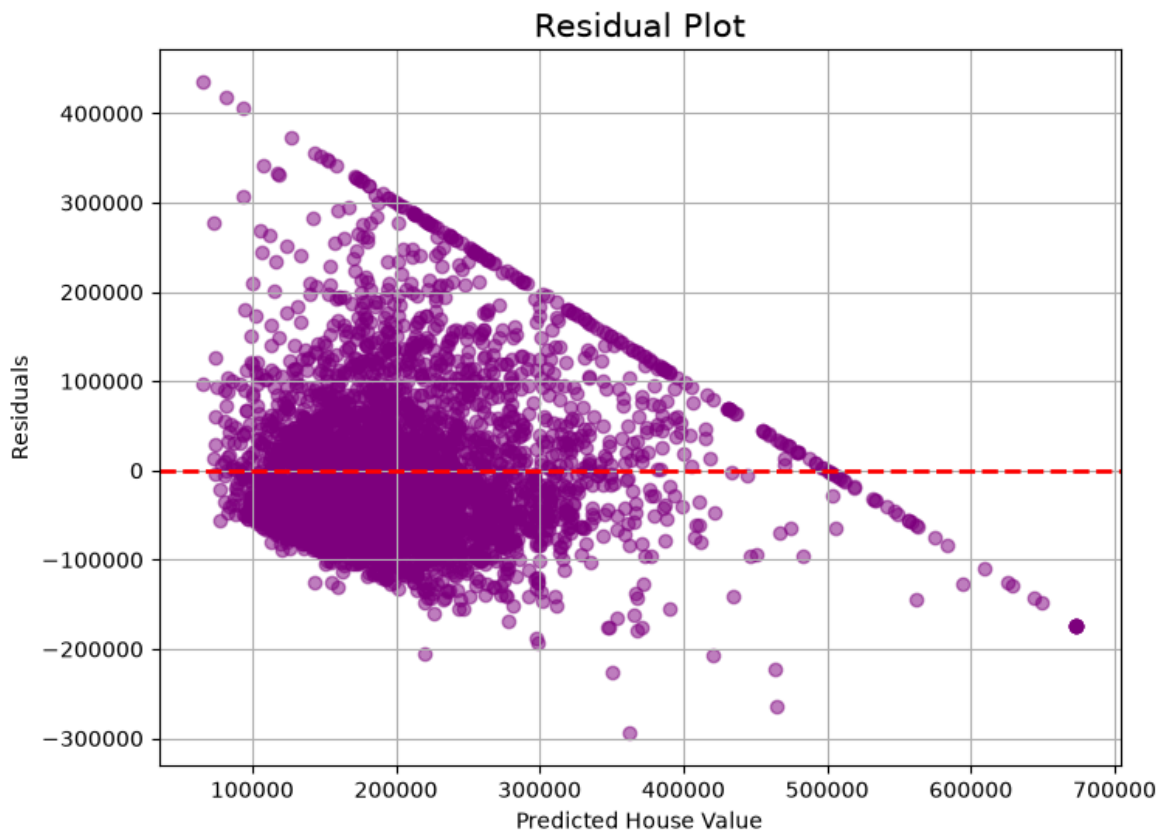
plt.xlabel("Predicted House Value")

plt.ylabel("Residuals")

plt.grid(True)

plt.show()

```



In [120...] `'''Residuals are scattered around zero, suggesting the model captures the overall trend\nSome large residuals remain, indicating prediction errors for certain observations'''`

Out[120...] `'Residuals are scattered around zero, suggesting the model captures the overall trend\nSome large residuals remain, indicating prediction errors for certain observations'`

In [121...] `'''The visualizations provide a clear understanding of the California Housing data set.'''`

Out[121...] `'The visualizations provide a clear understanding of the California Housing data set. Histograms show that several numerical features are right-skewed, while box plots reveal the presence of outliers in variables such as total_rooms and population. The scatter plot and regression line indicate a strong positive relationship between median_income and median_house_value, making it the most influential predictor. The correlation heatmap confirms this relationship and highlights correlations among other features. Finally, the residual plot shows that the linear regression model captures the overall trend, although some prediction errors remain, suggesting that additional features or more complex models could further improve predictive performance.'`

In [122...

#PART-8(Conclusion & Reflection)

In [123...

```
'''  
  
1. What did you learn from this assignment?  
  
- I learned how to import and inspect a dataset using Python.  
- I learned how to clean data by handling missing values and checking for duplic  
- I learned how to perform Exploratory Data Analysis (EDA) using summary statist  
- I learned how to build Simple Linear Regression and Multiple Linear Regression  
- I learned how to evaluate model performance using R2 Score, MAE, and RMSE.  
  
2. What was the most surprising finding in your data?  
  
- The most surprising finding was that Median Income had the strongest relations  
- I expected features like Total Rooms or Housing Median Age to have a stronger  
- I also found that geographical features such as Latitude and Longitude signifi  
  
3. What challenges did you face and how did you overcome them?  
  
- I faced missing library errors while running Python code.  
- I encountered errors while creating regression models and comparison tables.  
- I carefully read the error messages, corrected the code, and installed the req  
- After debugging the code step by step, I was able to complete the analysis suc  
  
4. How would this analysis help in a real business context?  
  
- Real estate companies can use this analysis to estimate house prices.  
- Buyers and sellers can make better decisions based on predicted house prices.  
- Businesses can identify the most important factors affecting property prices.  
  
5. What would you do differently with more time or data?  
  
- I would include additional features such as nearby schools, hospitals, transpo  
- I would perform more feature engineering to improve prediction accuracy.  
  
6. How do these statistical concepts connect to machine learning?  
  
- Statistics helps us understand the dataset before building a machine learning  
- Correlation helps in selecting the most important features.  
- Mean, median, and standard deviation help summarize the data.  
- Regression is one of the basic machine learning algorithms used for prediction  
- Evaluation metrics such as R2 Score, MAE, and RMSE help measure model performa  
  
7. What questions do you still have?  
  
- How do advanced machine learning models improve prediction accuracy?  
- What are the best techniques for handling large real-world datasets?'''
```

Out[123... '\n\n1. What did you learn from this assignment?\n\n- I learned how to import a nd inspect a dataset using Python.\n- I learned how to clean data by handling m issing values and checking for duplicates.\n- I learned how to perform Explorat ory Data Analysis (EDA) using summary statistics and visualizations.\n- I learn ed how to build Simple Linear Regression and Multiple Linear Regression model s.\n- I learned how to evaluate model performance using R² Score, MAE, and RMS E.\n\n2. What was the most surprising finding in your data?\n\n- The most sur prising finding was that Median Income had the strongest relationship with Medi an House Value.\n- I expected features like Total Rooms or Housing Median Age t o have a stronger impact, but income was the best predictor.\n- I also found th at geographical features such as Latitude and Longitude significantly influence house prices.\n\n3. What challenges did you face and how did you overcome th em?\n\n- I faced missing library errors while running Python code.\n- I encount ered errors while creating regression models and comparison tables.\n- I carefu lly read the error messages, corrected the code, and installed the required lib raries\n- After debugging the code step by step, I was able to complete the ana lysis successfully\n\n4. How would this analysis help in a real business cont ext?\n\n- Real estate companies can use this analysis to estimate house price s.\n- Buyers and sellers can make better decisions based on predicted house pri ces.\n- Businesses can identify the most important factors affecting property p rices.\n\n5. What would you do differently with more time or data?\n\n- I would include additional features such as nearby schools, hospitals, transportation, and crime rates.\n- I would perform more feature engineering to improve predict ion accuracy.\n\n6. How do these statistical concepts connect to machine lear ning?\n\n- Statistics helps us understand the dataset before building a machine learning model.\n- Correlation helps in selecting the most important feature s.\n- Mean, median, and standard deviation help summarize the data.\n- Regressi on is one of the basic machine learning algorithms used for prediction.\n- Eval uation metrics such as R² Score, MAE, and RMSE help measure model performance\n\n7. What questions do you still have?\n\n- How do advanced machine learning m odels improve prediction accuracy?\n- What are the best techniques for handling large real-world datasets?'

In [124... *#Compare with random foreset*

In [125... *#Compare with random foreset*
from sklearn.ensemble **import** RandomForestRegressor
from sklearn.metrics **import** r2_score, mean_absolute_error, mean_squared_error
import numpy **as** np

In [126... **from** sklearn.ensemble **import** RandomForestRegressor

 rf_model = RandomForestRegressor(
 n_estimators=100,
 random_state=42
)

 rf_model.fit(X_train, y_train)

```

Out[126... <style>.sk-global { /* Definition of color scheme common for light and dark mode */ --
sklearn-color-text: #000; --sklearn-color-text-muted: #666; --sklearn-color-line: gray; /*
Definition of color scheme for unfitted estimators */ --sklearn-color-unfitted-level-0:
#fff5e6; --sklearn-color-unfitted-level-1: #f6e4d2; --sklearn-color-unfitted-level-2:
#ffe0b3; --sklearn-color-unfitted-level-3: chocolate; /* Definition of color scheme for
fitted estimators */ --sklearn-color-fitted-level-0: #f0f8ff; --sklearn-color-fitted-level-1:
#d4ebff; --sklearn-color-fitted-level-2: #b3dbfd; --sklearn-color-fitted-level-3:
cornflowerblue; } .sk-global.light { /* Specific color for light theme */ --sklearn-color-text-
on-default-background: black; --sklearn-color-background: white; --sklearn-color-border-
box: black; --sklearn-color-icon: #696969; } .sk-global.dark { --sklearn-color-text-on-
default-background: white; --sklearn-color-background: #111; --sklearn-color-border-
box: white; --sklearn-color-icon: #878787; } .sk-global { color: var(--sklearn-color-text); }
.sk-global pre { padding: 0; } .sk-global input.sk-hidden--visually { border: 0; clip-path:
inset(100%); height: 1px; margin: -1px; overflow: hidden; padding: 0; position: absolute;
width: 1px; } .sk-global div.sk-dashed-wrapped { border: 1px dashed var(--sklearn-color-
line); margin: 0 0.4em 0.5em 0.4em; box-sizing: border-box; padding-bottom: 0.4em;
background-color: var(--sklearn-color-background); } .sk-global div.sk-container { /*
jupyter's `normalize.less` sets `[hidden] { display: none; }` but bootstrap.min.css set
`[hidden] { display: none !important; }` so we also need the `!important` here to be able to
override the default hidden behavior on the sphinx rendered scikit-learn.org. See:
https://github.com/scikit-learn/scikit-learn/issues/21755 */ display: inline-block
!important; position: relative; } .sk-global div.sk-text-repr-fallback { display: none; } div.sk-
parallel-item, div.sk-serial, div.sk-item { /* draw centered vertical line to link estimators */
background-image: linear-gradient(var(--sklearn-color-text-on-default-background), var(-
sklearn-color-text-on-default-background)); background-size: 2px 100%; background-
repeat: no-repeat; background-position: center center; } /* Parallel-specific style estimator
block */ .sk-global div.sk-parallel-item::after { content: ""; width: 100%; border-bottom:
2px solid var(--sklearn-color-text-on-default-background); flex-grow: 1; } .sk-global div.sk-
parallel { display: flex; align-items: stretch; justify-content: center; background-color: var(--
sklearn-color-background); position: relative; } .sk-global div.sk-parallel-item { display: flex;
flex-direction: column; } .sk-global div.sk-parallel-item:first-child::after { align-self: flex-
end; width: 50%; } .sk-global div.sk-parallel-item:last-child::after { align-self: flex-start;
width: 50%; } .sk-global div.sk-parallel-item:only-child::after { width: 0; } /* Serial-specific
style estimator block */ .sk-global div.sk-serial { display: flex; flex-direction: column; align-
items: center; background-color: var(--sklearn-color-background); padding-right: 1em;
padding-left: 1em; } /* Toggleable style: style used for
estimator/Pipeline/ColumnTransformer box that is clickable and can be
expanded/collapsed. - Pipeline and ColumnTransformer use this feature and define the
default style - Estimators will overwrite some part of the style using the `sk-estimator`
class */ /* Pipeline and ColumnTransformer style (default) */ .sk-global div.sk-toggleable {
/* Default theme specific background. It is overwritten whether we have a specific
estimator or a Pipeline/ColumnTransformer */ background-color: var(--sklearn-color-
background); } /* Toggleable label */ .sk-global label.sk-toggleable__label { cursor: pointer;
display: flex; width: 100%; margin-bottom: 0; padding: 0.5em; box-sizing: border-box;
text-align: center; align-items: center; justify-content: center; gap: 0.5em; } .sk-global

```

```

label.sk-toggleable__label .caption { font-size: 0.6rem; font-weight: lighter; color: var(--sklearn-color-text-muted); } .sk-global label.sk-toggleable__label-arrow:before { /* Arrow on the left of the label */ content: " ▶ "; float: left; margin-right: 0.25em; color: var(--sklearn-color-icon); } .sk-global label.sk-toggleable__label-arrow:hover:before { color: var(--sklearn-color-text); } /* Toggleable content - dropdown */ .sk-global div.sk-toggleable__content { display: none; text-align: left; /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-toggleable__content.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); } .sk-global div.sk-toggleable__content pre { margin: 0.2em; border-radius: 0.25em; color: var(--sklearn-color-text); /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-toggleable__content.fitted pre { /* unfitted */ background-color: var(--sklearn-color-fitted-level-0); } .sk-global input.sk-toggleable__control:checked~div.sk-toggleable__content { /* Expand drop-down */ display: block; width: 100%; overflow: visible; } .sk-global input.sk-toggleable__control:checked~label.sk-toggleable__label-arrow:before { content: " ▼ "; } /* Pipeline/ColumnTransformer-specific style */ .sk-global div.sk-label input.sk-toggleable__control:checked~label.sk-toggleable__label { color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-label.fitted input.sk-toggleable__control:checked~label.sk-toggleable__label { background-color: var(--sklearn-color-fitted-level-2); } /* Estimator-specific style */ /* Colorize estimator box */ .sk-global div.sk-estimator input.sk-toggleable__control:checked~label.sk-toggleable__label { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-estimator.fitted input.sk-toggleable__control:checked~label.sk-toggleable__label { /* fitted */ background-color: var(--sklearn-color-fitted-level-2); } .sk-global div.sk-label label.sk-toggleable__label, .sk-global div.sk-label label { /* The background is the default theme color */ color: var(--sklearn-color-text-on-default-background); } /* On hover, darken the color of the background */ .sk-global div.sk-label:hover label.sk-toggleable__label { color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); } /* Label box, darken color on hover, fitted */ .sk-global div.sk-label.fitted:hover label.sk-toggleable__label.fitted { color: var(--sklearn-color-text); background-color: var(--sklearn-color-fitted-level-2); } /* Estimator label */ .sk-global div.sk-label label { font-family: monospace; font-weight: bold; line-height: 1.2em; } .sk-global div.sk-label-container { text-align: center; } /* Estimator-specific */ .sk-global div.sk-estimator { font-family: monospace; border: 1px dotted var(--sklearn-color-border-box); border-radius: 0.25em; box-sizing: border-box; margin-bottom: 0.5em; /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-estimator.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); } /* on hover */ .sk-global div.sk-estimator:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-estimator.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-2); } /* Specification for estimator info (e.g. "i" and "?") */ /* Common style for "i" and "?" */ .sk-estimator-doc-link, a:link.sk-estimator-doc-link, a:visited.sk-estimator-doc-link { float: right; font-size: smaller; line-height: 1em; font-family: monospace; background-color: var(--sklearn-color-unfitted-level-0); border-radius: 1em; height: 1em; width: 1em; text-decoration: none !important; margin-left: 0.5em; text-align: center; /* unfitted */ border: var(--sklearn-color-unfitted-level-3) 1pt solid; color: var(--sklearn-

```

```

color-unfitted-level-3); } .sk-estimator-doc-link.fitted, a:link.sk-estimator-doc-link.fitted,
a:visited.sk-estimator-doc-link.fitted { /* fitted */ background-color: var(--sklearn-color-
fitted-level-0); border: var(--sklearn-color-fitted-level-3) 1pt solid; color: var(--sklearn-
color-fitted-level-3); } /* On hover */ div.sk-estimator: hover .sk-estimator-doc-link: hover,
.sk-estimator-doc-link: hover, div.sk-label-container: hover .sk-estimator-doc-link: hover,
.sk-estimator-doc-link: hover { /* unfitted */ background-color: var(--sklearn-color-
unfitted-level-3); border: var(--sklearn-color-fitted-level-0) 1pt solid; color: var(--sklearn-
color-unfitted-level-0); text-decoration: none; } div.sk-estimator.fitted: hover .sk-estimator-
doc-link.fitted: hover, .sk-estimator-doc-link.fitted: hover, div.sk-label-container: hover .sk-
estimator-doc-link.fitted: hover, .sk-estimator-doc-link.fitted: hover { /* fitted */
background-color: var(--sklearn-color-fitted-level-3); border: var(--sklearn-color-fitted-
level-0) 1pt solid; color: var(--sklearn-color-fitted-level-0); text-decoration: none; } /* Span,
style for the box shown on hovering the info icon */ .sk-estimator-doc-link span { display:
none; z-index: 9999; position: relative; font-weight: normal; right: .2ex; padding: .5ex;
margin: .5ex; width: min-content; min-width: 20ex; max-width: 50ex; color: var(--sklearn-
color-text); box-shadow: 2pt 2pt 4pt #999; /* unfitted */ background: var(--sklearn-color-
unfitted-level-0); border: .5pt solid var(--sklearn-color-unfitted-level-3); } .sk-estimator-
doc-link.fitted span { /* fitted */ background: var(--sklearn-color-fitted-level-0); border:
var(--sklearn-color-fitted-level-3); } .sk-estimator-doc-link: hover span { display: block; } /*
"-specific style due to the `` HTML tag */ .sk-global a.estimator_doc_link { float: right;
font-size: 1rem; line-height: 1em; font-family: monospace; background-color: var(--
sklearn-color-unfitted-level-0); border-radius: 1rem; height: 1rem; width: 1rem; text-
decoration: none; /* unfitted */ color: var(--sklearn-color-unfitted-level-1); border: var(--
sklearn-color-unfitted-level-1) 1pt solid; } .sk-global a.estimator_doc_link.fitted { /* fitted
*/ background-color: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-
level-1) 1pt solid; color: var(--sklearn-color-fitted-level-1); } /* On hover */ .sk-global
a.estimator_doc_link: hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-
level-3); color: var(--sklearn-color-background); text-decoration: none; } .sk-global
a.estimator_doc_link.fitted: hover { /* fitted */ background-color: var(--sklearn-color-fitted-
level-3); } .sk-top-container.sk-global { /* pydata-sphinx-theme hides overflow, so
scrolling is disabled. We need to set it to !important and add tabindex="0" in the HTML to
allow keyboard-only users to navigate the display. */ overflow-x: scroll !important; max-
width: 100%; } .estimator-table { font-family: monospace; } .estimator-table summary {
padding: .5rem; cursor: pointer; } .estimator-table summary::marker { font-size: 0.7rem; }
.estimator-table details[open] { padding-left: 0.1rem; padding-right: 0.1rem; padding-
bottom: 0.3rem; } .estimator-table .parameters-table { margin-left: auto !important;
margin-right: auto !important; margin-top: 0; } .estimator-table .parameters-table tr:nth-
child(odd) { background-color: #fff; } .estimator-table .parameters-table tr:nth-child(even)
{ background-color: #f6f6f6; } .estimator-table .parameters-table tr: hover td { background-
color: #e0e0e0; } .estimator-table table :is(td, th) { border: 1px solid rgba(106, 105, 104,
0.232); } /* `table td` is set in notebook with right text-align. We need to overwrite it. */
.estimator-table table td.param { text-align: left; position: relative; padding: 0; } .user-set td
{ color: rgb(255, 94, 0); text-align: left !important; } .user-set td.value { color: rgb(255, 94, 0);
background-color: transparent; } .default td, .estimator-table th { color: black; text-align:
left !important; } .user-set td i, .default td i { color: black; } td.fitted-att-type { white-space:

```



```

preserve nowrap; } /* Styles for parameter documentation links We need styling for visited
so jupyter doesn't overwrite it */ a.param-doc-link, a.param-doc-link:link, a.param-doc-
link:visited { text-decoration: underline dashed; text-underline-offset: .3em; color: inherit;
display: block; padding: .5em; } @supports(anchor-name: --doc-link) { a.param-doc-link,
a.param-doc-link:link, a.param-doc-link:visited { anchor-name: --doc-link; } } /* "hack" to
make the entire area of the cell containing the link clickable */ a.param-doc-link::before {
position: absolute; content: ""; inset: 0; } .param-doc-description { display: none; position:
absolute; z-index: 9999; left: 0; padding: .5ex; margin-left: 1.5em; color: var(--sklearn-
color-text); box-shadow: .3em .3em .4em #999; width: max-content; text-align: left; max-
height: 10em; overflow-y: auto; /* unfitted */ background: var(--sklearn-color-unfitted-
level-0); border: thin solid var(--sklearn-color-unfitted-level-3); } @supports(position-area:
center right) { .param-doc-description { position-area: center right; position: fixed; margin-
left: 0; } } /* Fitted state for parameter tooltips */ .fitted .param-doc-description { /* fitted
*/ background: var(--sklearn-color-fitted-level-0); border: thin solid var(--sklearn-color-
fitted-level-3); } .param-doc-link:hover .param-doc-description { display: block; } .copy-
paste-icon { background-image:
url(data:image/svg+xml;base64,PHN2ZyB4bWxucz0iaHR0cDovL3d3dy53My5vcmcvMjAwMC9:
background-repeat: no-repeat; background-size: 14px 14px; background-position: 0;
display: inline-block; width: 14px; height: 14px; cursor: pointer; } .features { font-family:
monospace; cursor: pointer; background-color: var(--sklearn-color-unfitted-level-0);
border: 1px dotted var(--sklearn-color-border-box); border-radius: .20em; margin-bottom:
0.5em; font-size: inherit; /* Needed for jupyter */ } .features.fitted { background-color:
var(--sklearn-color-fitted-level-0); } .features.summary { cursor: pointer; display: flex;
margin-bottom: 0; text-align: center; align-items: center; justify-content: center; gap:
0.5em; padding: .25em; } .features.details[open] > summary { color: var(--sklearn-color-
text); background-color: var(--sklearn-color-unfitted-level-2); border-radius: .20em 0 0 0; }
.features.fitted.details[open] > summary { background-color: var(--sklearn-color-fitted-
level-2); border-radius: .20em 0 0 0; } .features.details > summary .arrow::before { content:
" ▶ "; color: grey; } .features.details[open] > summary .arrow::before { content: " ▼ "; }
.features.details:hover > summary { margin: 0; background-color: var(--sklearn-color-
unfitted-level-2); } .features.fitted.details:hover > summary { margin: 0; background-color:
var(--sklearn-color-fitted-level-2); } .features .features-container { max-width: 15em; max-
height: 10em; overflow: auto; scrollbar-width: thin; padding: .25em 0.1em; background-
color: var(--sklearn-color-unfitted-level-0); border-radius: 0 0 .5em .5em; } .features.fitted
.features-container { background-color: var(--sklearn-color-fitted-level-0); } .features
.image-container { block-size: 1em; inline-size: 1em; padding: 0; margin: 0%; display: flex;
justify-content: center; align-items: center; } .features .copy-paste-icon { background-size:
1em 1em; width: 1em; height: 1em; filter: grayscale(100%) opacity(60%); } .features
.features-container table { width: 100%; margin: 0.01em; } .features .features-container
table tr:nth-child(odd) { background-color: #fff; } .features .features-container table tr:nth-
child(even) { background-color: #f6f6f6; } .features .features-container table tr:hover {
background-color: #e0e0e0; } .features .features-container table { table-layout: inherit; }
.features .features-container table td { text-align: left; padding: 0 0.5em; border: 1px solid
rgba(106, 105, 104, 0.232); white-space: nowrap; color: var(--sklearn-color-text); }
.total_features { display: flex; justify-content: center; margin-top: 0.5em; } </style> <body>

```

RandomForestRegressor(random_state=42)

**In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.**

```
<input class="sk-toggleable__control sk-hidden--visually sk-global" id="sk-estimator-id-3" type="checkbox" checked=""><label for="sk-estimator-id-3" class="sk-toggleable__label fitted sk-toggleable__label-arrow">
```

RandomForestRegressor

[?Documentation for RandomForestRegressor](#)

```
</label>
```

```
<details> <summary>Parameters</summary> <tbody> </tbody> <tbody></tbody>
```

```
    random_state random_state: int, RandomState instance or None,
default=None<br><br>Controls both the randomness of the bootstrapping of the
samples used<br>when building trees (if ``bootstrap=True``) and the sampling of
the<br>features to consider when looking for the best split at each node<br>(if
``max_features < n_features``).<br>See :term:`Glossary` <random_state>` for details.
```

```
    n_estimators n_estimators: int, default=100<br><br>The number of trees in the
forest.<br><br>.. versionchanged:: 0.22<br>The default value of ``n_estimators``
changed from 10 to 100<br>in 0.22.
```

```
    criterion criterion: {"squared_error", "absolute_error", "poisson"},
default="squared_error"<br><br>The function to measure the quality of a split.
Supported criteria<br>are "squared_error" for the mean squared error, which is
equal to<br>variance reduction as feature selection criterion and minimizes the
L2<br>loss using the mean of each terminal node, "absolute_error" for the
mean<br>absolute error, which minimizes the L1 loss using the median of each
terminal<br>node, and "poisson" which uses reduction in Poisson deviance to find
splits,<br>also using the mean of each terminal node.<br><br>.. versionadded::
0.18<br>Mean Absolute Error (MAE) criterion.<br><br>.. versionadded:: 1.0<br>Poisson criterion.<br><br>.. versionchanged:: 1.9<br>Criterion ``friedman_mse``
was deprecated.
```

```
    max_depth max_depth: int, default=None<br><br>The maximum depth of the
tree. If None, then nodes are expanded until<br>all leaves are pure or until all
leaves contain less than<br>min_samples_split samples.
```

```
    min_samples_split min_samples_split: int or float, default=2<br><br>The minimum
number of samples required to split an internal node:<br><br>- If int, then
consider ``min_samples_split`` as the minimum number.<br>- If float, then
``min_samples_split`` is a fraction and<br>``ceil(min_samples_split * n_samples)`` are
the minimum<br>number of samples for each split.<br><br>.. versionchanged::
0.18<br>Added float values for fractions.
```

```
    min_samples_leaf min_samples_leaf: int or float, default=1<br><br>The minimum
number of samples required to be at a leaf node.<br>A split point at any depth will
only be considered if it leaves at<br>least ``min_samples_leaf`` training samples in
each of the left and<br>right branches. This may have the effect of smoothing the
model,<br>especially in regression.<br><br>- If int, then consider
``min_samples_leaf`` as the minimum number.<br>- If float, then ``min_samples_leaf``
is a fraction and<br>``ceil(min_samples_leaf * n_samples)`` are the minimum<br>number of samples for each node.<br><br>.. versionchanged:: 0.18<br>Added
float values for fractions.
```

```
    min_weight_fraction_leaf min_weight_fraction_leaf: float, default=0.0<br><br>The
minimum weighted fraction of the sum total of weights (of all<br>the input
```

samples) required to be at a leaf node. Samples have equal weight when `sample_weight` is not provided.

`max_features` `max_features`: {"sqrt", "log2", None}, int or float, default=1.0
 The number of features to consider when looking for the best split:
 - If int, then consider `max_features` features at each split.
 - If float, then `max_features` is a fraction and `max(1, int(max_features * n_features_in_))` features are considered at each split.
 - If "sqrt", then `max_features=sqrt(n_features)`.
 - If "log2", then `max_features=log2(n_features)`.
 - If None or 1.0, then `max_features=n_features`.
 .. note:: The default of 1.0 is equivalent to bagged trees and more randomness can be achieved by setting smaller values, e.g. 0.3.
 .. versionchanged:: 1.1 The default of `max_features` changed from "auto" to 1.0.
 Note: the search for a split does not stop until at least one valid partition of the node samples is found, even if it requires to effectively inspect more than `max_features` features.

`max_leaf_nodes` `max_leaf_nodes`: int, default=None
 Grow trees with `max_leaf_nodes` in best-first fashion. Best nodes are defined as relative reduction in impurity.
 If None then unlimited number of leaf nodes.

N

`min_impurity_decrease` `min_impurity_decrease`: float, default=0.0
 A node will be split if this split induces a decrease of the impurity greater than or equal to this value.
 The weighted impurity decrease equation is the following:

$$N_t / N * (impurity - N_{t_R} / N_t * right_impurity - N_{t_L} / N_t * left_impurity)$$
 where `N` is the total number of samples, `Nt` is the number of samples at the current node, `Nt_L` is the number of samples in the left child, and `Nt_R` is the number of samples in the right child.
`N`, `Nt`, `Nt_R` and `Nt_L` all refer to the weighted sum, if `sample_weight` is passed.
 .. versionadded:: 0.19

`bootstrap` `bootstrap`: bool, default=True
 Whether bootstrap samples are used when building trees. If False, the whole dataset is used to build each tree.

`oob_score` `oob_score`: bool or callable, default=False
 Whether to use out-of-bag samples to estimate the generalization score.
 By default, `func: ~sklearn.metrics.r2_score` is used.
 Provide a callable with signature `metric(y_true, y_pred)` to use a custom metric. Only available if `bootstrap=True`.
 For an illustration of out-of-bag (OOB) error estimation, see the example
 :ref:`sphx\_glr\_auto\_examples\_ensemble\_plot\_ensemble\_oob.py`.

F

`n_jobs` `n_jobs`: int, default=None
 The number of jobs to run in parallel.
 :meth:`fit`, :meth:`predict`, :meth:`decision\_path` and :meth:`apply` are all parallelized over the trees. `None` means 1 unless in a `joblib.parallel_backend` context. `-1` means using all processors. See :term:`Glossary` `<n_jobs>` for more details.

N

`verbose` `verbose`: int, default=0
 Controls the verbosity when fitting and predicting.

`warm_start` `warm_start`: bool, default=False
 When set to `True`, reuse the solution of the previous call to `fit` and add more estimators to the ensemble, otherwise, just fit a whole new forest. See :term:`Glossary` `<warm_start>` and :ref:`tree\_ensemble\_warm\_start` for details.

F

`ccp_alpha` `ccp_alpha`: non-negative float, default=0.0
 Complexity parameter used for Minimal Cost-Complexity Pruning. The subtree with the largest cost complexity that is smaller than `ccp_alpha` will be chosen. By default, no pruning is performed. See :ref:`minimal\_cost\_complexity\_pruning` for details.

See
:ref:`sphx\_glr\_auto\_examples\_tree\_plot\_cost\_complexity\_pruning.py`
for an example of such pruning.

.. versionadded:: 0.22

max_samples max_samples: int or float, default=None

If bootstrap is True, the number of samples to draw from X
to train each base estimator.

- If None (default), then draw `X.shape[0]` samples irrespective of
`sample\_weight`.
- If int, then draw `max\_samples` samples.
- If float, then draw `max\_samples \* X.shape[0]` unweighted samples
or `max\_samples \* sample\_weight.sum()` weighted samples.

.. versionadded:: 0.22

.. versionchanged:: 1.9
Float `max\_samples` is relative to `sample\_weight.sum()` instead of
`X.shape[0]` for weighted samples.

monotonic_cst monotonic_cst: array-like of int of shape (n_features), default=None

Indicates the monotonicity constraint to enforce on each feature.
- 1: monotonically increasing
- 0: no constraint
- -1: monotonically decreasing

If monotonic_cst is None, no constraints are applied.

Monotonicity constraints are not supported for:
- multioutput regressions (i.e. when `n\_outputs\_` > 1).

Read more in the :ref:`User Guide <monotonic\_cst\_gbdt>`.

.. versionadded:: 1.4

</details>

<details> <summary>Fitted attributes</summary> <tbody> <th>Name</th>
<th>Type</th> <th>Value</th> </tbody> <tbody> </tbody>

estimator_ estimator_:
:class:`~sklearn.tree.DecisionTreeRegressor`

The child estimator template used to create the collection of fitted
sub-estimators.

.. versionadded:: 1.2
`base\_estimator\_` was renamed to `estimator\_`.

DecisionTreeRegressor

estimators_ estimators_: list of DecisionTreeRegressor

The collection of fitted sub-estimators.

list

[Decis
Decision

estimators_samples_ estimators_samples_: list of arrays

The subset of drawn samples (i.e., the in-bag samples) for each base
estimator. Each subset is defined by an array of the indices selected.

.. versionadded:: 1.4

list

[array([3113,

feature_importances_ feature_importances_: ndarray of shape (n_features,)

The impurity-based feature importances.
The higher, the more important the feature.
The importance of a feature is computed as the (normalized)
total reduction of the criterion brought by that feature. It is also
known as the Gini importance.

Warning: impurity-based feature importances can be misleading for
high cardinality features (many unique values).
See
:func:`sklearn.inspection.permutation\_importance` as an alternative.

ndarray[float64](5,)

feature_names_in_ feature_names_in_: ndarray of shape (n_features_in_,)

Names of features seen during :term:`fit`. Defined only when `X`
has feature names that are all strings.

.. versionadded:: 1.0

ndarray[object](5,) ['median_inco

n_features_in_ n_features_in_: int

Number of features seen during :term:`fit`.

.. versionadded:: 0.24

int

`n_outputs_`: int

The number of
outputs when ``fit`` is performed.

int

</details>

```
<script>/* Authors: The scikit-learn developers SPDX-License-Identifier: BSD-3-Clause */
function copyToClipboard(text, element) { // Get the parameter prefix from the closest
toggleable content const toggleableContent = element.closest('.sk-toggleable__content');
const paramPrefix = toggleableContent ? toggleableContent.dataset.paramPrefix : ''; const
fullParamName = paramPrefix ? `paramPrefix{text}` : text; const originalStyle =
element.style; const computedStyle = window.getComputedStyle(element); const
originalWidth = computedStyle.width; const originalHTML =
element.innerHTML.replace('Copied!', ''); navigator.clipboard.writeText(fullParamName)
.then(() => { element.style.width = originalWidth; element.style.color = 'green';
element.innerHTML = "Copied!"; setTimeout(() => { element.innerHTML = originalHTML;
element.style = originalStyle; }, 2000); }) .catch(err => { console.error('Failed to copy:', err);
element.style.color = 'red'; element.innerHTML = "Failed!"; setTimeout(() => {
element.innerHTML = originalHTML; element.style = originalStyle; }, 2000); }); return false;
} document.querySelectorAll('.copy-paste-icon').forEach(function(element) { const
toggleableContent = element.closest('.sk-toggleable__content'); const paramPrefix =
toggleableContent ? toggleableContent.dataset.paramPrefix : ''; const parent =
element.parentElement; if (!parent || !parent.nextElementSibling) { console.warn('Expected
copy-paste icon is missing from the DOM structure'); return; } const paramName =
element.parentElement.nextElementSibling.textContent.trim().split(' ')[0]; const
fullParamName = paramPrefix ? `paramPrefix{paramName}` : paramName;
element.setAttribute('title', fullParamName); }); /** * Copy the list of feature names
formatted as a Python list. * * @param {HTMLElement} element - The copy button inside a
`.features` block; its siblings * contain a `details` element and a table containing feature
named. * @returns {boolean} Always returns `false` so callers can prevent the default click
behavior. */ function copyFeatureNamesToClipboard(element) { var detailsElem =
element.closest('.features').querySelector('details'); var wasOpen = detailsElem.open;
detailsElem.open = true; var content = element.closest('.features').querySelector('tbody')
.innerHTML.trim(); if (!wasOpen) detailsElem.open = false; const rows =
content.split('\n').map(row => `row` ); const formattedText = `[\n
{rows.join(',\n')},\n]`; const originalHTML = element.innerHTML.replace('âœ“', ''); const
originalStyle = element.style; const copyMark = document.createElement('span');
copyMark.innerHTML = 'âœ“'; copyMark.style.color = 'blue'; copyMark.style.fontSize =
'1em'; navigator.clipboard.writeText(formattedText) .then(() => { element.style.display =
'none'; element.parentElement.appendChild(copyMark); setTimeout(() => {
copyMark.remove(); element.innerHTML = originalHTML; element.style = originalStyle; },
1000); }) .catch(err => { console.error('Failed to copy:', err); element.style.color = 'orange';
element.innerHTML = "Failed!"; setTimeout(() => { element.innerHTML = originalHTML;
element.style = originalStyle; }, 1000); }); return false; } /** * Adapted from Skrub *
https://github.com/skrub-
data/skrub/blob/403466d1d5d4dc76a7ef569b3f8228db59a31dc3/skrub/_reporting/_data/terr
* @returns "light" or "dark" */ function detectTheme(element) { const body =
document.querySelector('body'); // Check VSCode theme const themeKindAttr =
```

```
body.getAttribute('data-vscode-theme-kind'); const themeNameAttr =
body.getAttribute('data-vscode-theme-name'); if (themeKindAttr && themeNameAttr) {
const themeKind = themeKindAttr.toLowerCase(); const themeName =
themeNameAttr.toLowerCase(); if (themeKind.includes("dark") ||
themeName.includes("dark")) { return "dark"; } if (themeKind.includes("light") ||
themeName.includes("light")) { return "light"; } } // Check Jupyter theme if
(body.getAttribute('data-jp-theme-light') === 'false') { return 'dark'; } else if
(body.getAttribute('data-jp-theme-light') === 'true') { return 'light'; } // Guess based on a
parent element's color const color = window.getComputedStyle(element.parentNode,
null).getPropertyValue('color'); const match = color.match(/^rgb\s*
\s* (\d+)\s*, \s* (\d+)\s*, \s* (\d+)\s*$/i); if (match) { const [r, g, b] = [
parseFloat(match[1]), parseFloat(match[2]), parseFloat(match[3]) ]; //
https://en.wikipedia.org/wiki/HSL_and_HSV#Lightness const luma = 0.299 * r + 0.587 * g
+ 0.114 * b; if (luma > 180) { // If the text is very bright we have a dark theme return
'dark'; } if (luma < 75) { // If the text is very dark we have a light theme return 'light'; } //
Otherwise fall back to the next heuristic. } // Fallback to system preference return
window.matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' : 'light'; } function
forceTheme(elementId) { const estimatorElement =
document.querySelector(`#${elementId}`); if (estimatorElement === null) {
console.error(`Element with id ${elementId} not found.`); } else { const theme =
detectTheme(estimatorElement); estimatorElement.classList.add(theme); } }
forceTheme('sk-container-id-3');
```

In [128...

#prediction

rf_pred = rf_model.predict(X_test)

In [129...

rf_r2 = r2_score(y_test, rf_pred)

rf_mae = mean_absolute_error(y_test, rf_pred)

rf_rmse = np.sqrt(mean_squared_error(y_test, rf_pred))

print("Random Forest Results")

print("R² Score:", rf_r2)

print("MAE:", rf_mae)

print("RMSE:", rf_rmse)

Random Forest Results

R² Score: 0.8110495190488294

MAE: 32028.281543120156

RMSE: 49759.66304963792

In [137...

#Comparison table

comparison = pd.DataFrame({

"Model":[

"Simple Linear Regression",

"Multiple Linear Regression",

"Random Forest"

],

 "R² Score":[

```

        simple_r2,
        multiple_r2,
        rf_r2
    ],

    "MAE": [
        simple_mae,
        multiple_mae,
        rf_mae
    ],

    "RMSE": [
        simple_rmse,
        multiple_rmse,
        rf_rmse
    ]
})

comparison

```

Out[137...] <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Model</th> <th>R² Score</th> <th>MAE</th> <th>RMSE</th> </thead> <tbody> <tr><td>0</td> <td>1</td> <td>2</td> </tbody> </tbody>

Simple Linear Regression	0.458859	62990.865301	84209.012414
Multiple Linear Regression	0.584449	54230.744477	73793.096814
Random Forest	0.811050	32028.281543	49759.663050

In [138...] `'''The Random Forest model generally performs better than Linear Regression beca`

Out[138...] `'The Random Forest model generally performs better than Linear Regression because it can capture complex and non-linear relationships between features and house prices. Unlike Linear Regression, Random Forest builds multiple decision trees and combines their predictions, reducing overfitting and improving accuracy. This often results in a higher R2 score and lower MAE and RMSE values.'`

In [139...] `# Feature Importance`

In [140...] `# Feature Importance`

```

importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": rf_model.feature_importances_
})

importance = importance.sort_values(
    by="Importance",
    ascending=False
)

print(importance)

```

	Feature	Importance
0	median_income	0.530068
4	longitude	0.176120
3	latitude	0.168436
1	housing_median_age	0.072429
2	total_rooms	0.052947

```
In [141... #barchat
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8,5))

sns.barplot(
    data=importance,
    x="Importance",
    y="Feature",
    color="steelblue"
)

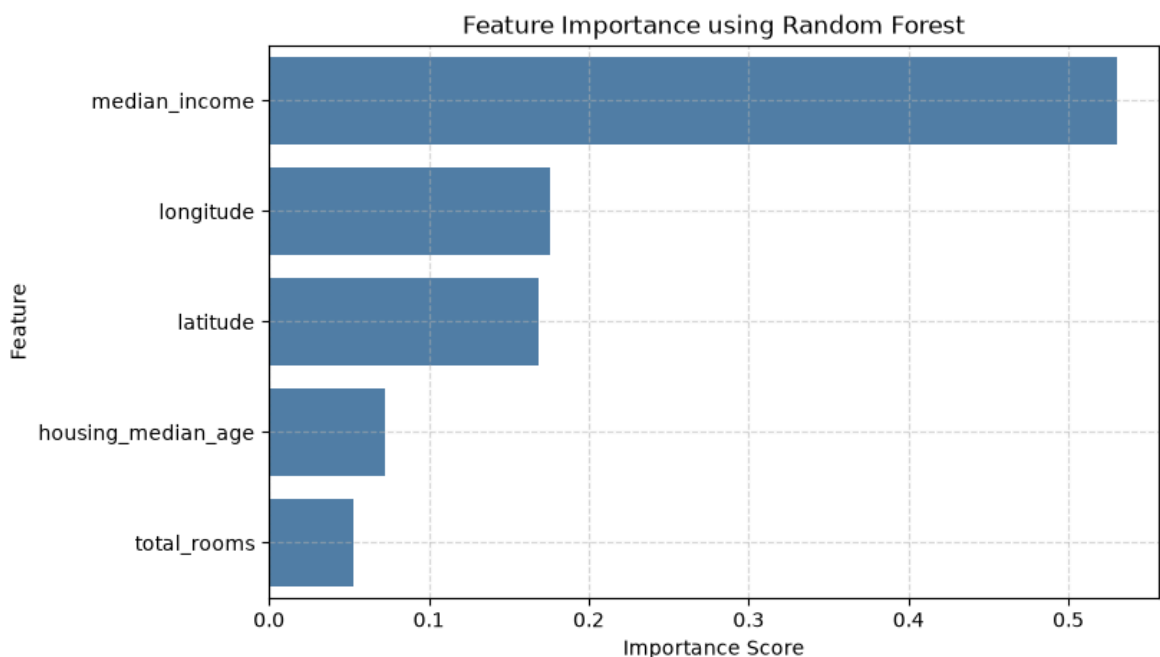
plt.title("Feature Importance using Random Forest")

plt.xlabel("Importance Score")

plt.ylabel("Feature")

plt.grid(True, linestyle="--", alpha=0.5)

plt.show()
```



```
In [142... '''The feature importance analysis shows the contribution of each feature to the
```

```
Out[142... 'The feature importance analysis shows the contribution of each feature to the
Random Forest model. Features with higher importance scores have a greater impact
on predicting house prices. In this dataset, median_income is expected to be the
most influential predictor, followed by geographical features such as latitude and
longitude.'
```



```
In [143... import joblib

joblib.dump(rf_model, "house_model.pkl")

print("Model saved successfully!")
```

Model saved successfully!

```
In [144... pip install plotly
```

Requirement already satisfied: plotly in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (6.9.0)
 Requirement already satisfied: narwhals>=1.15.1 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from plotly) (2.24.0)
 Requirement already satisfied: packaging in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from plotly) (26.2)
 Note: you may need to restart the kernel to use updated packages.

```
In [145... '''import plotly.express as px

fig = px.histogram(
    df,
    x="median_house_value",
    title="Distribution of House Prices"
)

fig.show()'''
```

```
Out[145... 'import plotly.express as px\n\nfig = px.histogram(\n    df,\n    x="median_house_value",\n    title="Distribution of House Prices"\n)\n\nfig.show()'
```

```
In [146... !pip install statsmodels
```

Requirement already satisfied: statsmodels in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (0.14.6)
 Requirement already satisfied: numpy<3,>=1.22.3 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from statsmodels) (2.5.1)
 Requirement already satisfied: scipy!=1.9.2,>=1.8 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from statsmodels) (1.18.0)
 Requirement already satisfied: pandas!=2.1.0,>=1.4 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from statsmodels) (3.0.3)
 Requirement already satisfied: patsy>=0.5.6 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from statsmodels) (1.0.2)
 Requirement already satisfied: packaging>=21.3 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from statsmodels) (26.2)
 Requirement already satisfied: python-dateutil>=2.8.2 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from pandas!=2.1.0,>=1.4->statsmodels) (2.9.0.post0)
 Requirement already satisfied: tzdata in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from pandas!=2.1.0,>=1.4->statsmodels) (2026.3)
 Requirement already satisfied: six>=1.5 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from python-dateutil>=2.8.2->pandas!=2.1.0,>=1.4->statsmodels) (1.17.0)

```
In [147... import plotly.express as px

fig = px.scatter(
    df,
    x="median_income",
    y="median_house_value",
    trendline="ols",
```

```
    title="Median Income vs Median House Value"  
)  
  
fig.show()
```

Median Income vs Median House Value



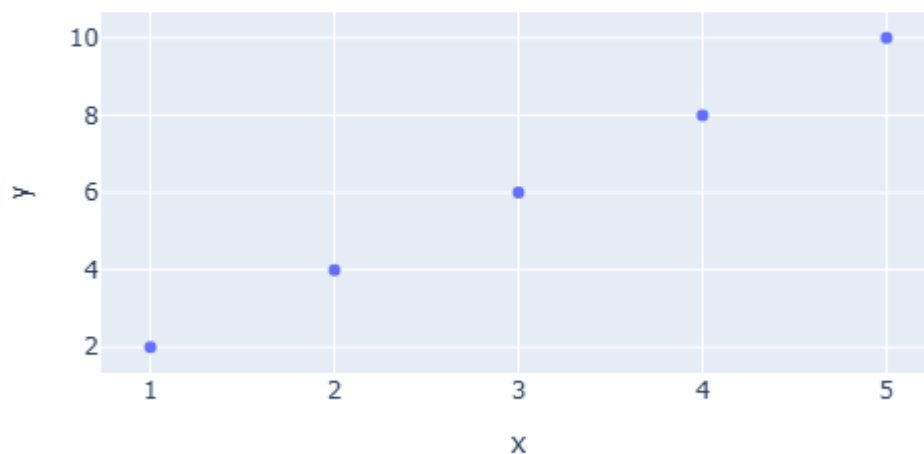
```
In [148... import plotly  
print(plotly.__version__)
```

6.9.0

```
In [149... import plotly.express as px
```

```
In [150... import plotly.express as px  
  
fig = px.scatter(  
    x=[1, 2, 3, 4, 5],  
    y=[2, 4, 6, 8, 10],  
    title="Test Scatter Plot"  
)  
  
fig.show()
```

Test Scatter Plot



```
In [151... import plotly.io as pio
pio.renderers.default = "browser"
```

```
In [152... import plotly.express as px

fig = px.scatter(
    df,
    x="median_income",
    y="median_house_value",
    title="Median Income vs Median House Value"
)

fig.show()
```

```
In [153... import plotly.express as px

fig = px.histogram(
    df,
    x="median_house_value",
    title="Distribution of House Prices"
)

fig.show()
```

```
In [154... fig = px.box(
    df,
    y="median_income",
    title="Median Income Box Plot"
)

fig.show()
```

```
In [155... !pip install streamlit
```

Requirement already satisfied: streamlit in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (1.59.2)

Requirement already satisfied: altair!=5.4.0,!5.4.1,<7,>=4.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from streamlit) (6.2.2)

Requirement already satisfied: blinker<2,>=1.5.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from streamlit) (1.9.0)

Requirement already satisfied: cachetools<8,>=5.5 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from streamlit) (7.1.4)

Requirement already satisfied: click<9,>=7.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from streamlit) (8.4.2)

Requirement already satisfied: gitpython!=3.1.19,<4,>=3.0.7 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from streamlit) (3.1.52)

Requirement already satisfied: numpy<3,>=1.23 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from streamlit) (2.5.1)

Requirement already satisfied: packaging>=20 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from streamlit) (26.2)

Requirement already satisfied: pandas<4,>=1.4.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from streamlit) (3.0.3)

Requirement already satisfied: pillow<13,>=7.1.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from streamlit) (12.3.0)

Requirement already satisfied: pydeck<1,>=0.8.0b4 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from streamlit) (0.9.3)

Requirement already satisfied: protobuf<8,>=3.20 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from streamlit) (7.35.1)

Requirement already satisfied: pyarrow<25,>=7.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from streamlit) (24.0.0)

Requirement already satisfied: requests<3,>=2.27 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from streamlit) (2.34.2)

Requirement already satisfied: tenacity<10,>=8.1.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from streamlit) (9.1.4)

Requirement already satisfied: toml<2,>=0.10.1 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from streamlit) (0.10.2)

Requirement already satisfied: typing-extensions<5,>=4.10.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from streamlit) (4.16.0)

Requirement already satisfied: starlette>=0.40.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from streamlit) (1.3.1)

Requirement already satisfied: uvicorn>=0.30.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from streamlit) (0.51.0)

Requirement already satisfied: httptools>=0.6.3 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from streamlit) (0.8.0)

Requirement already satisfied: anyio>=4.0.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from streamlit) (4.14.2)

Requirement already satisfied: python-multipart>=0.0.10 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from streamlit) (0.0.32)

Requirement already satisfied: websockets>=12.0.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from streamlit) (16.1.1)

Requirement already satisfied: itsdangerous>=2.1.2 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from streamlit) (2.2.0)

Requirement already satisfied: watchdog<7,>=2.1.5 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from streamlit) (6.0.0)

Requirement already satisfied: jinja2 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from altair!=5.4.0,!5.4.1,<7,>=4.0->streamlit) (3.1.6)

Requirement already satisfied: jsonschema>=3.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from altair!=5.4.0,!5.4.1,<7,>=4.0->streamlit) (4.26.0)

Requirement already satisfied: narwhals>=2.4.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from altair!=5.4.0,!5.4.1,<7,>=4.0->streamlit) (2.4.0)

Requirement already satisfied: colorama in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from click<9,>=7.0->streamlit) (0.4.6)

Requirement already satisfied: gitdb<5,>=4.0.1 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from gitpython!=3.1.19,<4,>=3.0.7->streamlit) (4.0.12)

Requirement already satisfied: smmap<6,>=3.0.1 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from gitdb<5,>=4.0.1->gitpython!=3.1.19,<4,>=3.0.7->streamlit) (5.0.3)

Requirement already satisfied: python-dateutil>=2.8.2 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from pandas<4,>=1.4.0->streamlit) (2.9.0.post0)

Requirement already satisfied: tzdata in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from pandas<4,>=1.4.0->streamlit) (2026.3)

Requirement already satisfied: charset_normalizer<4,>=2 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from requests<3,>=2.27->streamlit) (3.4.9)

Requirement already satisfied: idna<4,>=2.5 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from requests<3,>=2.27->streamlit) (3.18)

Requirement already satisfied: urllib3<3,>=1.26 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from requests<3,>=2.27->streamlit) (2.7.0)

Requirement already satisfied: certifi>=2023.5.7 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from requests<3,>=2.27->streamlit) (2026.6.17)

Requirement already satisfied: MarkupSafe>=2.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from jinja2->altair!=5.4.0,!5.4.1,<7,>=4.0->streamlit) (3.0.3)

Requirement already satisfied: attrs>=22.2.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from jsonschema>=3.0->altair!=5.4.0,!5.4.1,<7,>=4.0->streamlit) (26.1.0)

Requirement already satisfied: jsonschema-specifications>=2023.03.6 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from jsonschema>=3.0->altair!=5.4.0,!5.4.1,<7,>=4.0->streamlit) (2025.9.1)

Requirement already satisfied: referencing>=0.28.4 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from jsonschema>=3.0->altair!=5.4.0,!5.4.1,<7,>=4.0->streamlit) (0.37.0)

Requirement already satisfied: rpds-py>=0.25.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from jsonschema>=3.0->altair!=5.4.0,!5.4.1,<7,>=4.0->streamlit) (2026.6.3)

Requirement already satisfied: six>=1.5 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from python-dateutil>=2.8.2->pandas<4,>=1.4.0->streamlit) (1.17.0)

Requirement already satisfied: h11>=0.8 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from uvicorn>=0.30.0->streamlit) (0.16.0)

In [156... `model.fit(X_train, y_train)`

```

Out[156... <style>.sk-global { /* Definition of color scheme common for light and dark mode */ --
sklearn-color-text: #000; --sklearn-color-text-muted: #666; --sklearn-color-line: gray; /*
Definition of color scheme for unfitted estimators */ --sklearn-color-unfitted-level-0:
#fff5e6; --sklearn-color-unfitted-level-1: #f6e4d2; --sklearn-color-unfitted-level-2:
#ffe0b3; --sklearn-color-unfitted-level-3: chocolate; /* Definition of color scheme for
fitted estimators */ --sklearn-color-fitted-level-0: #f0f8ff; --sklearn-color-fitted-level-1:
#d4ebff; --sklearn-color-fitted-level-2: #b3dbfd; --sklearn-color-fitted-level-3:
cornflowerblue; } .sk-global.light { /* Specific color for light theme */ --sklearn-color-text-
on-default-background: black; --sklearn-color-background: white; --sklearn-color-border-
box: black; --sklearn-color-icon: #696969; } .sk-global.dark { --sklearn-color-text-on-
default-background: white; --sklearn-color-background: #111; --sklearn-color-border-
box: white; --sklearn-color-icon: #878787; } .sk-global { color: var(--sklearn-color-text); }
.sk-global pre { padding: 0; } .sk-global input.sk-hidden--visually { border: 0; clip-path:
inset(100%); height: 1px; margin: -1px; overflow: hidden; padding: 0; position: absolute;
width: 1px; } .sk-global div.sk-dashed-wrapped { border: 1px dashed var(--sklearn-color-
line); margin: 0 0.4em 0.5em 0.4em; box-sizing: border-box; padding-bottom: 0.4em;
background-color: var(--sklearn-color-background); } .sk-global div.sk-container { /*
jupyter's `normalize.less` sets `[hidden] { display: none; }` but bootstrap.min.css set
`[hidden] { display: none !important; }` so we also need the `!important` here to be able to
override the default hidden behavior on the sphinx rendered scikit-learn.org. See:
https://github.com/scikit-learn/scikit-learn/issues/21755 */ display: inline-block
!important; position: relative; } .sk-global div.sk-text-repr-fallback { display: none; } div.sk-
parallel-item, div.sk-serial, div.sk-item { /* draw centered vertical line to link estimators */
background-image: linear-gradient(var(--sklearn-color-text-on-default-background), var(-
sklearn-color-text-on-default-background)); background-size: 2px 100%; background-
repeat: no-repeat; background-position: center center; } /* Parallel-specific style estimator
block */ .sk-global div.sk-parallel-item::after { content: ""; width: 100%; border-bottom:
2px solid var(--sklearn-color-text-on-default-background); flex-grow: 1; } .sk-global div.sk-
parallel { display: flex; align-items: stretch; justify-content: center; background-color: var(--
sklearn-color-background); position: relative; } .sk-global div.sk-parallel-item { display: flex;
flex-direction: column; } .sk-global div.sk-parallel-item:first-child::after { align-self: flex-
end; width: 50%; } .sk-global div.sk-parallel-item:last-child::after { align-self: flex-start;
width: 50%; } .sk-global div.sk-parallel-item:only-child::after { width: 0; } /* Serial-specific
style estimator block */ .sk-global div.sk-serial { display: flex; flex-direction: column; align-
items: center; background-color: var(--sklearn-color-background); padding-right: 1em;
padding-left: 1em; } /* Toggleable style: style used for
estimator/Pipeline/ColumnTransformer box that is clickable and can be
expanded/collapsed. - Pipeline and ColumnTransformer use this feature and define the
default style - Estimators will overwrite some part of the style using the `sk-estimator`
class */ /* Pipeline and ColumnTransformer style (default) */ .sk-global div.sk-toggleable {
/* Default theme specific background. It is overwritten whether we have a specific
estimator or a Pipeline/ColumnTransformer */ background-color: var(--sklearn-color-
background); } /* Toggleable label */ .sk-global label.sk-toggleable__label { cursor: pointer;
display: flex; width: 100%; margin-bottom: 0; padding: 0.5em; box-sizing: border-box;
text-align: center; align-items: center; justify-content: center; gap: 0.5em; } .sk-global

```

```

label.sk-toggleable__label .caption { font-size: 0.6rem; font-weight: lighter; color: var(--
sklearn-color-text-muted); } .sk-global label.sk-toggleable__label-arrow:before { /* Arrow
on the left of the label */ content: " ▶ "; float: left; margin-right: 0.25em; color: var(--
sklearn-color-icon); } .sk-global label.sk-toggleable__label-arrow:hover:before { color: var(--
sklearn-color-text); } /* Toggleable content - dropdown */ .sk-global div.sk-
toggleable__content { display: none; text-align: left; /* unfitted */ background-color: var(--
sklearn-color-unfitted-level-0); } .sk-global div.sk-toggleable__content.fitted { /* fitted */
background-color: var(--sklearn-color-fitted-level-0); } .sk-global div.sk-
toggleable__content pre { margin: 0.2em; border-radius: 0.25em; color: var(--sklearn-
color-text); /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-
global div.sk-toggleable__content.fitted pre { /* unfitted */ background-color: var(--
sklearn-color-fitted-level-0); } .sk-global input.sk-toggleable__control:checked~div.sk-
toggleable__content { /* Expand drop-down */ display: block; width: 100%; overflow:
visible; } .sk-global input.sk-toggleable__control:checked~label.sk-toggleable__label-
arrow:before { content: " ▼ "; } /* Pipeline/ColumnTransformer-specific style */ .sk-global
div.sk-label input.sk-toggleable__control:checked~label.sk-toggleable__label { color: var(--
sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); } .sk-global
div.sk-label.fitted input.sk-toggleable__control:checked~label.sk-toggleable__label {
background-color: var(--sklearn-color-fitted-level-2); } /* Estimator-specific style */ /*
Colorize estimator box */ .sk-global div.sk-estimator input.sk-
toggleable__control:checked~label.sk-toggleable__label { /* unfitted */ background-color:
var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-estimator.fitted input.sk-
toggleable__control:checked~label.sk-toggleable__label { /* fitted */ background-color:
var(--sklearn-color-fitted-level-2); } .sk-global div.sk-label label.sk-toggleable__label, .sk-
global div.sk-label label { /* The background is the default theme color */ color: var(--
sklearn-color-text-on-default-background); } /* On hover, darken the color of the
background */ .sk-global div.sk-label:hover label.sk-toggleable__label { color: var(--
sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); } /* Label box,
darken color on hover, fitted */ .sk-global div.sk-label.fitted:hover label.sk-
toggleable__label.fitted { color: var(--sklearn-color-text); background-color: var(--sklearn-
color-fitted-level-2); } /* Estimator label */ .sk-global div.sk-label label { font-family:
monospace; font-weight: bold; line-height: 1.2em; } .sk-global div.sk-label-container {
text-align: center; } /* Estimator-specific */ .sk-global div.sk-estimator { font-family:
monospace; border: 1px dotted var(--sklearn-color-border-box); border-radius: 0.25em;
box-sizing: border-box; margin-bottom: 0.5em; /* unfitted */ background-color: var(--
sklearn-color-unfitted-level-0); } .sk-global div.sk-estimator.fitted { /* fitted */
background-color: var(--sklearn-color-fitted-level-0); } /* on hover */ .sk-global div.sk-
estimator:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-2); }
.sk-global div.sk-estimator.fitted:hover { /* fitted */ background-color: var(--sklearn-color-
fitted-level-2); } /* Specification for estimator info (e.g. "i" and "?") */ /* Common style for
"i" and "?" */ .sk-estimator-doc-link, a:link.sk-estimator-doc-link, a:visited.sk-estimator-
doc-link { float: right; font-size: smaller; line-height: 1em; font-family: monospace;
background-color: var(--sklearn-color-unfitted-level-0); border-radius: 1em; height: 1em;
width: 1em; text-decoration: none !important; margin-left: 0.5em; text-align: center; /*
unfitted */ border: var(--sklearn-color-unfitted-level-3) 1pt solid; color: var(--sklearn-

```

```

color-unfitted-level-3); } .sk-estimator-doc-link.fitted, a:link.sk-estimator-doc-link.fitted,
a:visited.sk-estimator-doc-link.fitted { /* fitted */ background-color: var(--sklearn-color-
fitted-level-0); border: var(--sklearn-color-fitted-level-3) 1pt solid; color: var(--sklearn-
color-fitted-level-3); } /* On hover */ div.sk-estimator: hover .sk-estimator-doc-link: hover,
.sk-estimator-doc-link: hover, div.sk-label-container: hover .sk-estimator-doc-link: hover,
.sk-estimator-doc-link: hover { /* unfitted */ background-color: var(--sklearn-color-
unfitted-level-3); border: var(--sklearn-color-fitted-level-0) 1pt solid; color: var(--sklearn-
color-unfitted-level-0); text-decoration: none; } div.sk-estimator.fitted: hover .sk-estimator-
doc-link.fitted: hover, .sk-estimator-doc-link.fitted: hover, div.sk-label-container: hover .sk-
estimator-doc-link.fitted: hover, .sk-estimator-doc-link.fitted: hover { /* fitted */
background-color: var(--sklearn-color-fitted-level-3); border: var(--sklearn-color-fitted-
level-0) 1pt solid; color: var(--sklearn-color-fitted-level-0); text-decoration: none; } /* Span,
style for the box shown on hovering the info icon */ .sk-estimator-doc-link span { display:
none; z-index: 9999; position: relative; font-weight: normal; right: .2ex; padding: .5ex;
margin: .5ex; width: min-content; min-width: 20ex; max-width: 50ex; color: var(--sklearn-
color-text); box-shadow: 2pt 2pt 4pt #999; /* unfitted */ background: var(--sklearn-color-
unfitted-level-0); border: .5pt solid var(--sklearn-color-unfitted-level-3); } .sk-estimator-
doc-link.fitted span { /* fitted */ background: var(--sklearn-color-fitted-level-0); border:
var(--sklearn-color-fitted-level-3); } .sk-estimator-doc-link: hover span { display: block; } /*
"-specific style due to the `` HTML tag */ .sk-global a.estimator_doc_link { float: right;
font-size: 1rem; line-height: 1em; font-family: monospace; background-color: var(--
sklearn-color-unfitted-level-0); border-radius: 1rem; height: 1rem; width: 1rem; text-
decoration: none; /* unfitted */ color: var(--sklearn-color-unfitted-level-1); border: var(--
sklearn-color-unfitted-level-1) 1pt solid; } .sk-global a.estimator_doc_link.fitted { /* fitted
*/ background-color: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-
level-1) 1pt solid; color: var(--sklearn-color-fitted-level-1); } /* On hover */ .sk-global
a.estimator_doc_link: hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-
level-3); color: var(--sklearn-color-background); text-decoration: none; } .sk-global
a.estimator_doc_link.fitted: hover { /* fitted */ background-color: var(--sklearn-color-fitted-
level-3); } .sk-top-container.sk-global { /* pydata-sphinx-theme hides overflow, so
scrolling is disabled. We need to set it to !important and add tabindex="0" in the HTML to
allow keyboard-only users to navigate the display. */ overflow-x: scroll !important; max-
width: 100%; } .estimator-table { font-family: monospace; } .estimator-table summary {
padding: .5rem; cursor: pointer; } .estimator-table summary::marker { font-size: 0.7rem; }
.estimator-table details[open] { padding-left: 0.1rem; padding-right: 0.1rem; padding-
bottom: 0.3rem; } .estimator-table .parameters-table { margin-left: auto !important;
margin-right: auto !important; margin-top: 0; } .estimator-table .parameters-table tr:nth-
child(odd) { background-color: #fff; } .estimator-table .parameters-table tr:nth-child(even)
{ background-color: #f6f6f6; } .estimator-table .parameters-table tr: hover td { background-
color: #e0e0e0; } .estimator-table table :is(td, th) { border: 1px solid rgba(106, 105, 104,
0.232); } /* `table td` is set in notebook with right text-align. We need to overwrite it. */
.estimator-table table td.param { text-align: left; position: relative; padding: 0; } .user-set td
{ color: rgb(255, 94, 0); text-align: left !important; } .user-set td.value { color: rgb(255, 94, 0);
background-color: transparent; } .default td, .estimator-table th { color: black; text-align:
left !important; } .user-set td i, .default td i { color: black; } td.fitted-att-type { white-space:

```



```

preserve nowrap; } /* Styles for parameter documentation links We need styling for visited
so jupyter doesn't overwrite it */ a.param-doc-link, a.param-doc-link:link, a.param-doc-
link:visited { text-decoration: underline dashed; text-underline-offset: .3em; color: inherit;
display: block; padding: .5em; } @supports(anchor-name: --doc-link) { a.param-doc-link,
a.param-doc-link:link, a.param-doc-link:visited { anchor-name: --doc-link; } } /* "hack" to
make the entire area of the cell containing the link clickable */ a.param-doc-link::before {
position: absolute; content: ""; inset: 0; } .param-doc-description { display: none; position:
absolute; z-index: 9999; left: 0; padding: .5ex; margin-left: 1.5em; color: var(--sklearn-
color-text); box-shadow: .3em .3em .4em #999; width: max-content; text-align: left; max-
height: 10em; overflow-y: auto; /* unfitted */ background: var(--sklearn-color-unfitted-
level-0); border: thin solid var(--sklearn-color-unfitted-level-3); } @supports(position-area:
center right) { .param-doc-description { position-area: center right; position: fixed; margin-
left: 0; } } /* Fitted state for parameter tooltips */ .fitted .param-doc-description { /* fitted
*/ background: var(--sklearn-color-fitted-level-0); border: thin solid var(--sklearn-color-
fitted-level-3); } .param-doc-link:hover .param-doc-description { display: block; } .copy-
paste-icon { background-image:
url(data:image/svg+xml;base64,PHN2ZyB4bWxucz0iaHR0cDovL3d3dy53My5vcmcvMjAwMC9:
background-repeat: no-repeat; background-size: 14px 14px; background-position: 0;
display: inline-block; width: 14px; height: 14px; cursor: pointer; } .features { font-family:
monospace; cursor: pointer; background-color: var(--sklearn-color-unfitted-level-0);
border: 1px dotted var(--sklearn-color-border-box); border-radius: .20em; margin-bottom:
0.5em; font-size: inherit; /* Needed for jupyter */ } .features.fitted { background-color:
var(--sklearn-color-fitted-level-0); } .features.summary { cursor: pointer; display: flex;
margin-bottom: 0; text-align: center; align-items: center; justify-content: center; gap:
0.5em; padding: .25em; } .features.details[open] > summary { color: var(--sklearn-color-
text); background-color: var(--sklearn-color-unfitted-level-2); border-radius: .20em 0 0 0; }
.features.fitted.details[open] > summary { background-color: var(--sklearn-color-fitted-
level-2); border-radius: .20em 0 0 0; } .features.details > summary .arrow::before { content:
" ▶ "; color: grey; } .features.details[open] > summary .arrow::before { content: " ▼ "; }
.features.details:hover > summary { margin: 0; background-color: var(--sklearn-color-
unfitted-level-2); } .features.fitted.details:hover > summary { margin: 0; background-color:
var(--sklearn-color-fitted-level-2); } .features .features-container { max-width: 15em; max-
height: 10em; overflow: auto; scrollbar-width: thin; padding: .25em 0.1em; background-
color: var(--sklearn-color-unfitted-level-0); border-radius: 0 0 .5em .5em; } .features.fitted
.features-container { background-color: var(--sklearn-color-fitted-level-0); } .features
.image-container { block-size: 1em; inline-size: 1em; padding: 0; margin: 0%; display: flex;
justify-content: center; align-items: center; } .features .copy-paste-icon { background-size:
1em 1em; width: 1em; height: 1em; filter: grayscale(100%) opacity(60%); } .features
.features-container table { width: 100%; margin: 0.01em; } .features .features-container
table tr:nth-child(odd) { background-color: #fff; } .features .features-container table tr:nth-
child(even) { background-color: #f6f6f6; } .features .features-container table tr:hover {
background-color: #e0e0e0; } .features .features-container table { table-layout: inherit; }
.features .features-container table td { text-align: left; padding: 0 0.5em; border: 1px solid
rgba(106, 105, 104, 0.232); white-space: nowrap; color: var(--sklearn-color-text); }
.total_features { display: flex; justify-content: center; margin-top: 0.5em; } </style> <body>

```

LinearRegression()

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

☒>

LinearRegression

[?Documentation for LinearRegression](#)

</label>

<details> <summary>Parameters</summary> <tbody> </tbody> <tbody> </tbody>

fit_intercept fit_intercept: bool, default=True

Whether to calculate the intercept for this model. If set to False, no intercept will be used in calculations (i.e. data is expected to be centered). True

copy_X copy_X: bool, default=True

If True, X will be copied; else, it may be overwritten. True

tol tol: float, default=1e-6

The precision of the solution ('coef_') is determined by 'tol' which specifies the convergence criterion of the underlying solver. 'tol' is set as 'atol' and 'btol' of :func:'scipy.sparse.linalg.lsqr' when fitting on sparse training data. 'tol' is set as 'cond' of :func:'scipy.linalg.lstsq' when fitting on dense training data.

.. versionadded:: 1.7
.. versionchanged:: 1.9
Now supported on dense data, interpreted as the 'cond' parameter. 1e-06

n_jobs n_jobs: int, default=None

The number of jobs to use for the computation. This will only provide speedup in case of sufficiently large problems, that is if firstly 'n_targets > 1' and secondly 'X' is sparse or if 'positive' is set to 'True'. 'None' means 1 unless in a :obj:'joblib.parallel_backend' context. '-1' means using all processors. See :term:'Glossary <n_jobs>' for more details. None

positive positive: bool, default=False

When set to "True", forces the coefficients to be positive. This option is only supported for dense arrays.

For a comparison between a linear regression model with positive constraints on the regression coefficients and a linear regression without such constraints, see :ref:'sphx_glr_auto_examples_linear_model_plot_nnls.py'.
.. versionadded:: 0.24 False

</details>

<details> <summary>Fitted attributes</summary> <tbody> <th>Name</th>

<th>Type</th> <th>Value</th> </tbody> <tbody> </tbody>

coef_ coef_: array ndarray[float64] [37535.98, 1167.87, 3.38, -43887
of shape (5,) (n_features,) or (n_targets, n_features)

Estimated coefficients for the linear regression problem.
If multiple targets are passed during the fit (y 2D), this is a 2D

array of shape
(n_targets,
n_features), while
if only one
target is passed,
this is a 1D array
of length
n_features.

feature_names_in_
feature_names_in_:
ndarray of shape
(n_features_in_,)
Names
of features seen
during :term:`fit`.
Defined only when
'X' has feature
names that are all
strings.
versionadded:: 1.0

ndarray[object]
(5,) ['median_income', 'housing_median_age', 'total_rooms', 'latitude', 'population', 'median_house_value']

intercept_
intercept_: float or
array of shape
(n_targets,) Independent
term in the linear
model. Set to 0.0
if :term:`fit\_intercept`
= False.

float64

n_features_in_
n_features_in_:
int
Number of
features seen
during :term:`fit`.
versionadded::
0.24

int

rank_rank_
int Rank
of matrix 'X'. Only
available when 'X'
is dense.

int64

singular_singular_
array of shape
(min(X, y),) Singular
values of 'X'. Only
available when 'X'
is dense.

ndarray[float64]
(5,)

[279432.15, 1510.47, 368.94,

</details>

<script> /* Authors: The scikit-learn developers
SPDX-License-Identifier: BSD-3-Clause */
function copyToClipboard(text, element) { // Get the parameter prefix from the closest
toggleable content const toggleableContent = element.closest('.sk-toggleable__content');

```

const paramPrefix = toggleableContent ? toggleableContent.dataset.paramPrefix : ""; const
fullParamName = paramPrefix ? `paramPrefix{text}` : text; const originalStyle =
element.style; const computedStyle = window.getComputedStyle(element); const
originalWidth = computedStyle.width; const originalHTML =
element.innerHTML.replace('Copied!', ''); navigator.clipboard.writeText(fullParamName)
.then(() => { element.style.width = originalWidth; element.style.color = 'green';
element.innerHTML = "Copied!"; setTimeout(() => { element.innerHTML = originalHTML;
element.style = originalStyle; }, 2000); }) .catch(err => { console.error('Failed to copy:', err);
element.style.color = 'red'; element.innerHTML = "Failed!"; setTimeout(() => {
element.innerHTML = originalHTML; element.style = originalStyle; }, 2000); }); return false;
} document.querySelectorAll('.copy-paste-icon').forEach(function(element) { const
toggleableContent = element.closest('.sk-toggleable__content'); const paramPrefix =
toggleableContent ? toggleableContent.dataset.paramPrefix : ""; const parent =
element.parentElement; if (!parent || !parent.nextElementSibling) { console.warn('Expected
copy-paste icon is missing from the DOM structure'); return; } const paramName =
element.parentElement.nextElementSibling.textContent.trim().split(' ')[0]; const
fullParamName = paramPrefix ? `paramPrefix{paramName}` : paramName;
element.setAttribute('title', fullParamName); }); /** * Copy the list of feature names
formatted as a Python list. * * @param {HTMLElement} element - The copy button inside a
`.features` block; its siblings * contain a `details` element and a table containing feature
named. * @returns {boolean} Always returns `false` so callers can prevent the default click
behavior. */ function copyFeatureNamesToClipboard(element) { var detailsElem =
element.closest('.features').querySelector('details'); var wasOpen = detailsElem.open;
detailsElem.open = true; var content = element.closest('.features').querySelector('tbody')
.innerHTML.trim(); if (!wasOpen) detailsElem.open = false; const rows =
content.split('\n').map(row => `row `); const formattedText = `[
{rows.join(',\n')},n]`; const originalHTML = element.innerHTML.replace('âœ“', ''); const
originalStyle = element.style; const copyMark = document.createElement('span');
copyMark.innerHTML = 'âœ“'; copyMark.style.color = 'blue'; copyMark.style.fontSize =
'1em'; navigator.clipboard.writeText(formattedText) .then(() => { element.style.display =
'none'; element.parentElement.appendChild(copyMark); setTimeout(() => {
copyMark.remove(); element.innerHTML = originalHTML; element.style = originalStyle; },
1000); }) .catch(err => { console.error('Failed to copy:', err); element.style.color = 'orange';
element.innerHTML = "Failed!"; setTimeout(() => { element.innerHTML = originalHTML;
element.style = originalStyle; }, 1000); }); return false; } /** * Adapted from Skrub *
https://github.com/skrub-
data/skrub/blob/403466d1d5d4dc76a7ef569b3f8228db59a31dc3/skrub/_reporting/_data/terr
* @returns "light" or "dark" */ function detectTheme(element) { const body =
document.querySelector('body'); // Check VSCode theme const themeKindAttr =
body.getAttribute('data-vscode-theme-kind'); const themeNameAttr =
body.getAttribute('data-vscode-theme-name'); if (themeKindAttr && themeNameAttr) {
const themeKind = themeKindAttr.toLowerCase(); const themeName =
themeNameAttr.toLowerCase(); if (themeKind.includes("dark") ||
themeName.includes("dark")) { return "dark"; } if (themeKind.includes("light") ||
themeName.includes("light")) { return "light"; } } // Check Jupyter theme if

```

```
(body.getAttribute('data-jp-theme-light') === 'false') { return 'dark'; } else if
(body.getAttribute('data-jp-theme-light') === 'true') { return 'light'; } // Guess based on a
parent element's color const color = window.getComputedStyle(element.parentNode,
null).getPropertyValue('color'); const match = color.match(/^rgb\s*
\s* (\d+)\s*, \s* (\d+)\s*, \s* (\d+)\s*$/i); if (match) { const [r, g, b] = [
parseFloat(match[1]), parseFloat(match[2]), parseFloat(match[3]) ]; //
https://en.wikipedia.org/wiki/HSL_and_HSV#Lightness const luma = 0.299 * r + 0.587 * g
+ 0.114 * b; if (luma > 180) { // If the text is very bright we have a dark theme return
'dark'; } if (luma < 75) { // If the text is very dark we have a light theme return 'light'; } //
Otherwise fall back to the next heuristic. } // Fallback to system preference return
window.matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' : 'light'; } function
forceTheme(elementId) { const estimatorElement =
document.querySelector(`#${elementId}`); if (estimatorElement === null) {
console.error(`Element with id ${elementId} not found.`); } else { const theme =
detectTheme(estimatorElement); estimatorElement.classList.add(theme); } }
forceTheme('sk-container-id-4');
```

In [157...

```
import joblib

joblib.dump(model, "house_model.pkl")

print("Model saved successfully!")
```

Model saved successfully!

In [158...

```
import os

print(os.getcwd())
print(os.listdir())
```

```
C:\Users\deekshitha.kotian
['.bash_history', '.gitconfig', '.idlerc', '.ipynb_checkpoints', '.ipython', '.ju
pyter', '.lessht', '.python_history', '.streamlit', '.vscode', '.vscode-shared',
'app.py', 'AppData', 'Application Data', 'Contacts', 'Cookies', 'Documents', 'Dow
nloads', 'Favorites', 'house_model.pkl', 'housing.csv', 'Links', 'Local Setting
s', 'Music', 'My Documents', 'NetHood', 'NTUSER.DAT', 'ntuser.dat.LOG1', 'ntuser.
dat.LOG2', 'NTUSER.DAT{c4d96ec7-771b-11f1-85ca-ac91a10ae34c}.TM.blf', 'NTUSER.DAT
{c4d96ec7-771b-11f1-85ca-ac91a10ae34c}.TMContainer000000000000000001.regtrans-m
s', 'NTUSER.DAT{c4d96ec7-771b-11f1-85ca-ac91a10ae34c}.TMContainer00000000000000
0002.regtrans-ms', 'ntuser.ini', 'OneDrive', 'OneDrive - VCTI', 'PrintHood', 'Pyt
hon-Training', 'Recent', 'Saved Games', 'Searches', 'SendTo', 'source', 'Start Me
nu', 'Templates', 'Untitled-1.py', 'Untitled.ipynb', 'Untitled1.ipynb', 'Untitled
1.PDF', 'Videos']
```

In [110...

```
import joblib

model = joblib.load("house_model.pkl")
```

In []: